Arun S 241801027

EX:1 DATA PREPROCESSING AND CLEANING

Aim:

To preprocess the Telco Customer Churn dataset by cleaning the data, handling missing values, removing duplicate records, encoding categorical features, standardizing numerical features, and visualizing important relationships to prepare the dataset for machine learning.

Learning objectives:

*To load the customer churn dataset into a Pandas DataFrame.

*To explore the dataset using summary statistics and dataset information.

*To identify and remove duplicate records.

*To handle missing values using the Median (numerical columns) and Mode (categorical columns).

*To encode categorical features into numerical values.

*To standardize numerical features using StandardScaler.

*To visualize data using histograms, count plots, box plots, and correlation heatmaps.

*To prepare a clean dataset suitable for machine learning algorithms.

Built-in Methods / Functions Used

Pandas

pd.read_csv() – Load the dataset.

head() – Display the first 10 records.

tail() – Display the last 10 records.

shape – Get the dimensions of the dataset.

columns – Display column names.

info() – Display dataset information.

describe() – Generate summary statistics.

isnull() – Check missing values.

sum() – Count missing values and duplicates.

duplicated() – Identify duplicate records.

drop_duplicates() – Remove duplicate records.

fillna() – Replace missing values.

median() – Calculate the median for numerical columns.

mode() – Calculate the mode for categorical columns.

get_dummies() – Perform one-hot encoding.

corr() – Compute the correlation matrix.

to_numeric() – Convert data to numeric type (if required).

NumPy

np.nan – Represent missing values.

Scikit-learn

LabelEncoder() – Encode binary categorical variables.

StandardScaler() – Standardize numerical features.

Seaborn

histplot() – Plot the distribution of numerical features.

countplot() – Create count plots for categorical variables.

boxplot() – Display box plots.

heatmap() – Display the correlation heatmap.

Matplotlib

figure() – Create a figure.

title() – Add a title to the plot.

xlabel() – Add an x-axis label.

ylabel() – Add a y-axis label.

show() – Display the plot.

1.Import the required Python libraries.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import LabelEncoder, OneHotEncoder, StandardScaler
```

2. Load the Telco Customer Churn Dataset into a Pandas DataFrame. And display First 10 records, Last 10 records, Shape of the dataset, Column names.

```
df = pd.read_csv("/content/customer_churn_dataset-testing-master.csv.zip")

print("First 10 Records")
print(df.head(10))

print("\nLast 10 Records")
print(df.tail(10))

print("\nShape of Dataset")
print(df.shape)

print("\nColumn Names")
print(df.columns.tolist())
```

```
            8             Basic         Monthly          687
           23             Basic          Annual          995

Last Interaction  Churn
               9      1
              20      0
              21      0
              18      0
              18      0
              29      0
              14      1
              16      0
               8      0
              10      0

t 10 Records
    CustomerID  Age  Gender  Tenure  Usage Frequency  Support Calls  \
64       64365   48  Female      36               22              4
65       64366   26  Female      54               24              2
```

```
64                9      1
65               23      1
66               21      1
67               24      1
68               15      1
69               14      1
70                9      1
71               20      1
72               13      1
73               22      1

pe of Dataset
374, 12)

umn Names
```

3. Explore the Dataset and display Dataset information, Summary statistics, Number of missing values, Number of duplicate records

```
print("\nDataset Information")
print(df.info())

print("\nSummary Statistics")
print(df.describe(include='all'))

print("\nMissing Values")
print(df.isnull().sum())

print("\nDuplicate Records")
print(df.duplicated().sum())
```

```
Summary Statistics
          CustomerID           Age  Gender        Tenure  Usage Frequency  \
count   64374.000000  64374.000000   64374  64374.000000     64374.000000
unique           NaN           NaN       2           NaN              NaN
top              NaN           NaN  Female           NaN              NaN
freq             NaN           NaN   34353           NaN              NaN
mean    32187.500000     41.970982     NaN     31.994827        15.080234
std     18583.317451     13.924911     NaN     17.098234         8.816470
min         1.000000     18.000000     NaN      1.000000         1.000000
25%     16094.250000     30.000000     NaN     18.000000         7.000000
50%     32187.500000     42.000000     NaN     33.000000        15.000000
75%     48280.750000     54.000000     NaN     47.000000        23.000000
max     64374.000000     65.000000     NaN     60.000000        30.000000

        Support Calls  Payment Delay Subscription Type Contract Length  \
count    64374.000000   64374.000000              64374           64374
unique            NaN            NaN                  3               3
top               NaN            NaN           Standard         Monthly
freq              NaN            NaN              21502           22130
mean         5.400690      17.133952                NaN             NaN
std          3.114005       8.852211                NaN             NaN
min          0.000000       0.000000                NaN             NaN
25%          3.000000      10.000000                NaN             NaN
50%          6.000000      19.000000                NaN             NaN
75%          8.000000      25.000000                NaN             NaN
max         10.000000      30.000000                NaN             NaN

         Total Spend  Last Interaction         Churn
count   64374.000000      64374.000000  64374.000000
unique           NaN               NaN           NaN
top              NaN               NaN           NaN
freq             NaN               NaN           NaN
mean      541.023379         15.498850      0.473685
```

```
Duplicate Records
0
```

```
print("Duplicate Records Before:", df.duplicated().sum())

df = df.drop_duplicates()

print("Duplicate Records After:", df.duplicated().sum())
```

```
Duplicate Records Before: 0
Duplicate Records After: 0
```

```
df.loc[10, 'Age'] = np.nan
df.loc[50, 'Gender'] = np.nan
df.loc[100, 'Tenure'] = np.nan
df.loc[200, 'Subscription Type'] = np.nan
df.loc[500, 'Total Spend'] = np.nan
```

```
print(df.isnull().sum())
```

```
CustomerID           0
Age                  1
Gender               1
Tenure               1
Usage Frequency      0
Support Calls        0
Payment Delay        0
Subscription Type    1
Contract Length      0
Total Spend          1
Last Interaction     0
Churn                0
dtype: int64
```

```
num_cols = df.select_dtypes(include=['int64', 'float64']).columns
for col in num_cols:
    df[col].fillna(df[col].median(), inplace=True)


cat_cols = df.select_dtypes(include='object').columns
for col in cat_cols:
    df[col].fillna(df[col].mode()[0], inplace=True)


print(df.isnull().sum())
```

```
CustomerID           0
Age                  0
Gender               0
Tenure               0
Usage Frequency      0
Support Calls        0
Payment Delay        0
Subscription Type    0
Contract Length      0
Total Spend          0
Last Interaction     0
Churn                0
dtype: int64
/tmp/ipykernel_568/3334642829.py:3: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through ch
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col]


  df[col].fillna(df[col].median(), inplace=True)
/tmp/ipykernel_568/3334642829.py:8: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through ch
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col]


  df[col].fillna(df[col].mode()[0], inplace=True)
```

```
print("Duplicate Records Before:", df.duplicated().sum())

df = df.drop_duplicates()

print("Duplicate Records After:", df.duplicated().sum())
```

```
Duplicate Records Before: 0
Duplicate Records After: 0
```

```
import numpy as np

df.loc[10, 'Age'] = np.nan
df.loc[50, 'Gender'] = np.nan
df.loc[100, 'Tenure'] = np.nan
df.loc[150, 'Subscription Type'] = np.nan
df.loc[200, 'Total Spend'] = np.nan
```

```
print(df.isnull().sum())
```

```
CustomerID           0
Age                  1
Gender               1
Tenure               1
Usage Frequency      0
Support Calls        0
Payment Delay        0
Subscription Type    1
Contract Length      0
Total Spend          1
Last Interaction     0
Churn                0
dtype: int64
```

```
num_cols = df.select_dtypes(include=['int64','float64']).columns

for col in num_cols:
    df[col] = df[col].fillna(df[col].median())
```

```
cat_cols = df.select_dtypes(include='object').columns

for col in cat_cols:
    df[col] = df[col].fillna(df[col].mode()[0])
```

```
print(df.isnull().sum())
```

```
CustomerID           0
Age                  0
Gender               0
Tenure               0
Usage Frequency      0
Support Calls        0
Payment Delay        0
Subscription Type    0
Contract Length      0
Total Spend          0
Last Interaction     0
Churn                0
dtype: int64
```

6. Standardise the numerical features: Tenure, MonthlyCharges, TotalCharges, using StandardScaler.

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

df['Gender'] = le.fit_transform(df['Gender'])
```

```
df = pd.get_dummies(df,
                    columns=['Subscription Type'],
                    drop_first=True)
```

```
df = pd.get_dummies(df,
                    columns=['Contract Length'],
                    drop_first=True)
```

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
```

```
df[['Tenure','Total Spend']] = scaler.fit_transform(
    df[['Tenure','Total Spend']]
)
```

7. Create a Count Plot showing Contract vs Churn.

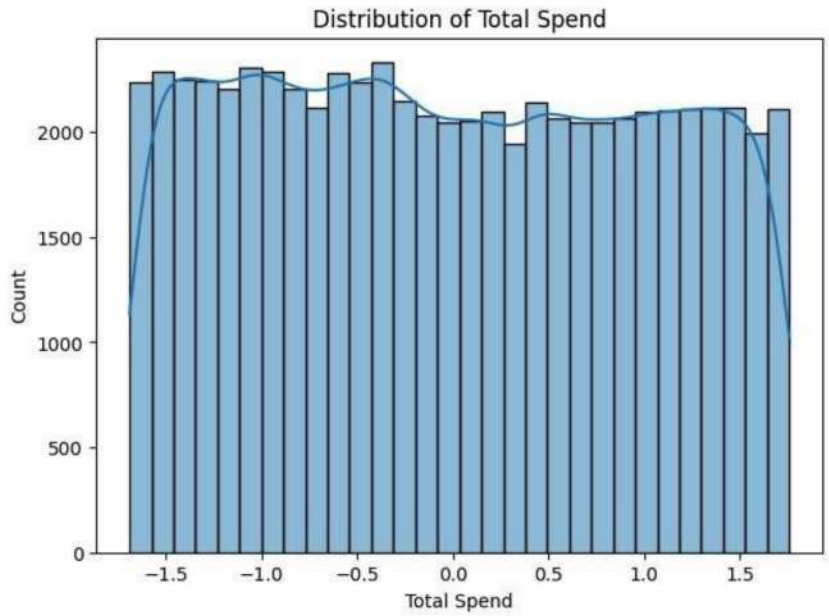
```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(7,5))

sns.histplot(df['Total Spend'],
             bins=30,
             kde=True)

plt.title("Distribution of Total Spend")

plt.show()
```



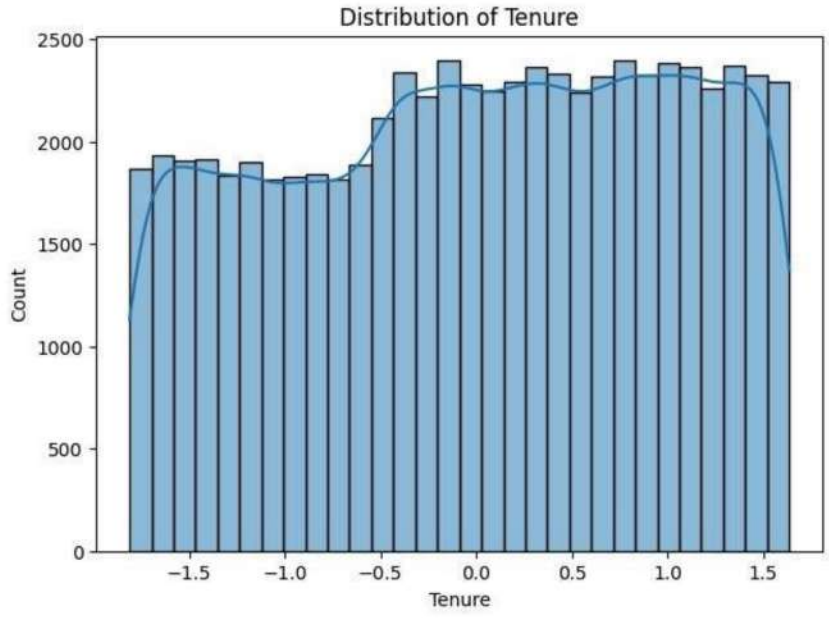
8. Create a Count Plot showing Contract vs Churn.

```
plt.figure(figsize=(7,5))

sns.histplot(df['Tenure'],
             bins=30,
             kde=True)

plt.title("Distribution of Tenure")

plt.show()
```

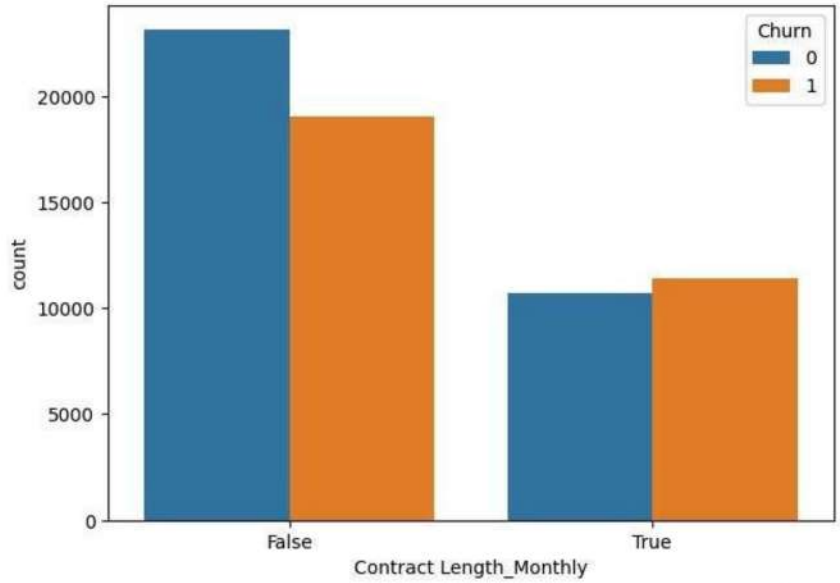


9. Draw a Box Plot of MonthlyCharges grouped by Churn.

```
plt.figure(figsize=(7,5))

sns.countplot(data=df,
              x='Contract Length_Monthly',
              hue='Churn')

plt.show()
```

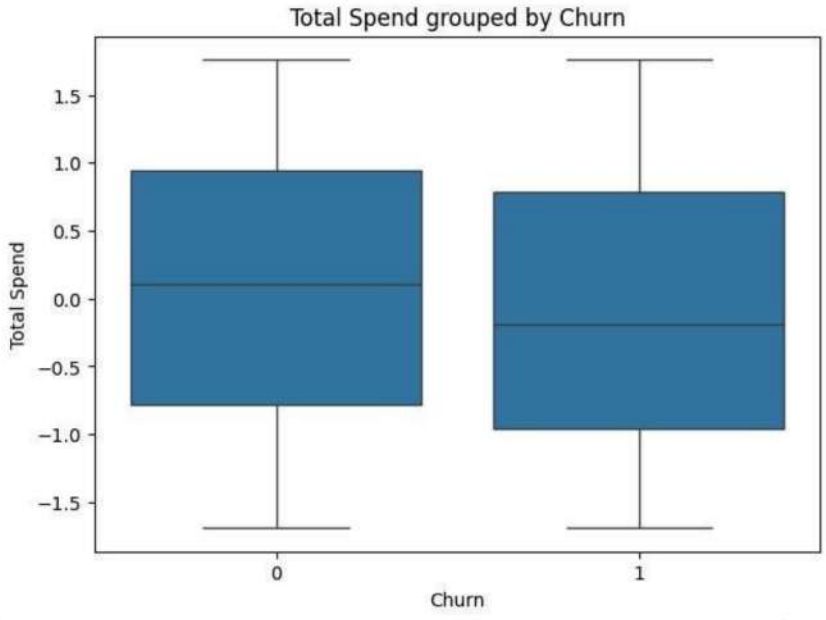



```
plt.figure(figsize=(7,5))

sns.boxplot(data=df,
            x='Churn',
            y='Total Spend')

plt.title("Total Spend grouped by Churn")

plt.show()
```



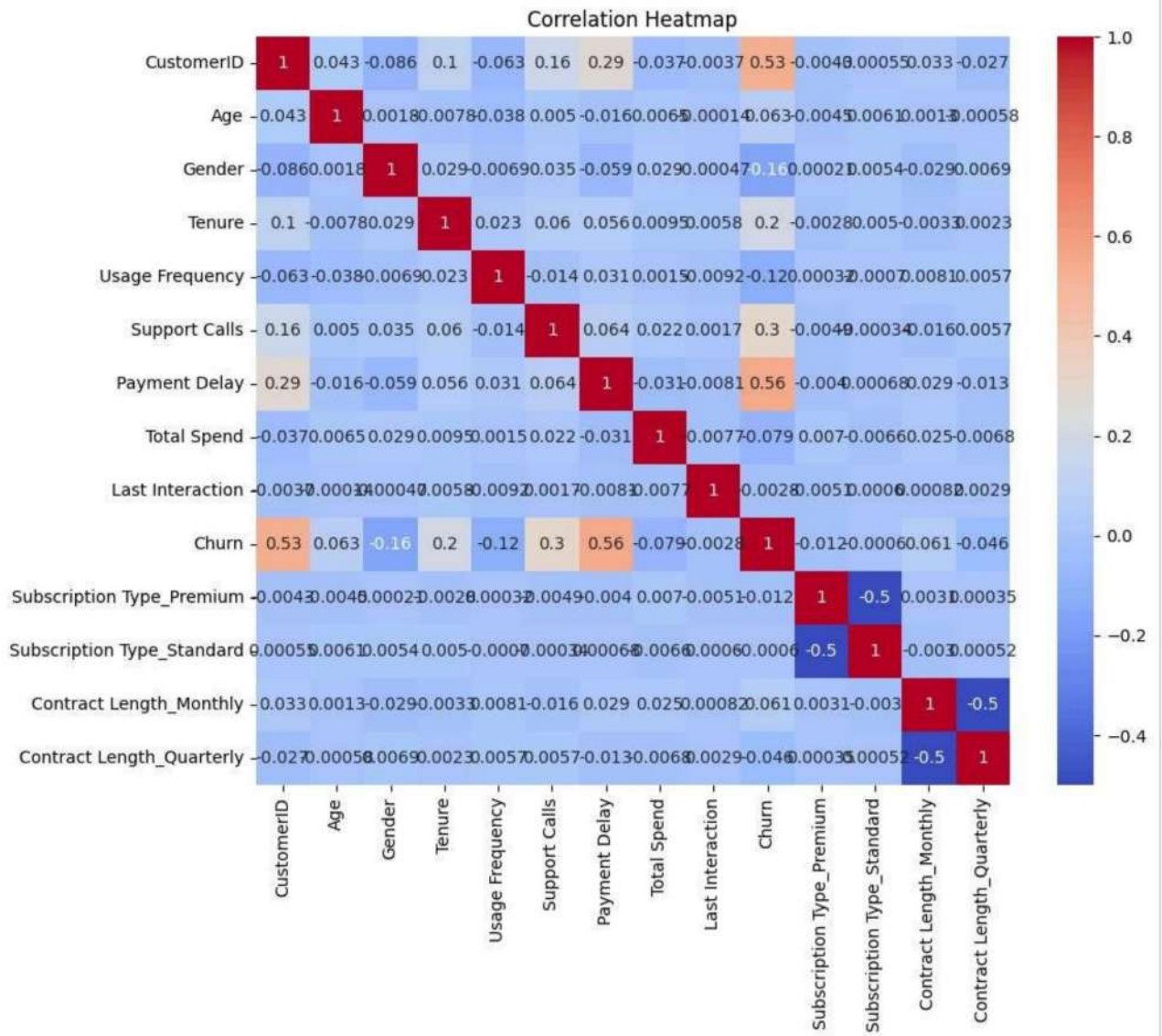
10. Display the Correlation Heatmap of all numerical features.

```
plt.figure(figsize=(10,8))

sns.heatmap(df.corr(),
            annot=True,
            cmap='coolwarm')

plt.title("Correlation Heatmap")
```

```
plt.show()
```



Learning Outcomes:

After completing this experiment, I was able to:

Load and explore the Telco Customer Churn dataset.

Identify and clean missing and duplicate data.

Encode categorical variables for machine learning.

Standardize numerical features using StandardScaler.

Visualize data distributions and relationships using different plots.

Understand feature correlations through a heatmap.

Prepare a clean and preprocessed dataset for predictive modeling

EX:2 Data Discovery and Preparation

Arun S (241801027)

AIM:

To perform data discovery, exploratory data analysis, outlier detection, and data preparation on the customer churn dataset to identify important customer behavior patterns and prepare the dataset for predictive analytics.

LEARNING OBJECTIVES:

*To explore and understand the customer churn dataset.

*To analyze the distribution of customer churn, gender, subscription type, and contract length.

*To visualize numerical features such as Age, Tenure, and Total Spend using histograms and box plots.

*To detect outliers in numerical features using the Interquartile Range (IQR) method.

*To study the relationship between contract length, subscription type, tenure, total spend, and customer churn.

*To identify important features that may influence customer churn.

*To split the processed dataset into 80% training and 20% testing data.

*To save the final processed dataset as CustomerChurn_Preprocessed.csv

FUNCTIONS / METHODS USED:

Pandas Functions:

Function Purpose

pd.read_csv() Load the CSV/ZIP dataset

head() Display first records

tail() Display last records

shape Find rows and columns

columns Display column names

info() Display dataset information

describe() Generate summary statistics

isnull() Check missing values

duplicated() Find duplicate records

drop_duplicates() Remove duplicate records

fillna() Fill missing values

median() Calculate median

mode() Find the most frequent value

groupby() Group data for analysis

value_counts() Count categorical values

corr() Calculate correlation

to_csv() Save processed datase

NumPy Functions:

Function Purpose

np.nan Represent missing values

Double-click (or enter) to edit

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split

sns.set_style("whitegrid")
```

```
file_path = "/content/customer_churn_dataset-testing-master.csv.zip"

df = pd.read_csv(file_path)

print("Dataset loaded successfully!")
```

```
Dataset loaded successfully!
```

```
print(df.head())
```

```
   CustomerID  Age  Gender  Tenure  Usage Frequency  Support Calls  \
0           1   22  Female      25               14              4
1           2   41  Female      28               28              7
2           3   47    Male      27               10              2
3           4   35    Male       9               12              5
4           5   53  Female      58               24              9

   Payment Delay Subscription Type Contract Length  Total Spend  \
0             27              Basic         Monthly          598
1             13           Standard         Monthly          584
2             29            Premium          Annual          757
3             17            Premium       Quarterly          232
4              2           Standard          Annual          533

   Last Interaction  Churn
0                 9      1
1                20      0
2                21      0
3                18      0
4                18      0
```

```
print("First 10 Records:")
display(df.head(10))
```

First 10 Records:

	CustomerID	Age	Gender	Tenure	Usage Frequency	Support Calls	Payment Delay	Subscription Type	Contract Length	Total Spend	Las Interacti
0	1	22	Female	25	14	4	27	Basic	Monthly	598	
1	2	41	Female	28	28	7	13	Standard	Monthly	584	2
2	3	47	Male	27	10	2	29	Premium	Annual	757	2
3	4	35	Male	9	12	5	17	Premium	Quarterly	232	1
4	5	53	Female	58	24	9	2	Standard	Annual	533	1
5	6	30	Male	41	14	10	10	Premium	Monthly	500	2
6	7	47	Female	37	15	9	28	Basic	Quarterly	574	1
7	8	54	Female	36	11	0	18	Standard	Monthly	323	1
8	9	36	Male	20	5	10	8	Basic	Monthly	687	
9	10	65	Male	8	4	2	23	Basic	Annual	995	1

```
print("Last 10 Records:")
display(df.tail(10))
```

Last 10 Records:

	CustomerID	Age	Gender	Tenure	Usage Frequency	Support Calls	Payment Delay	Subscription Type	Contract Length	Total Spend	Inter
64364	64365	48	Female	36	22	4	22	Premium	Annual	342	
64365	64366	26	Female	54	24	2	21	Basic	Annual	165	
64366	64367	64	Male	30	27	9	28	Premium	Annual	884	
64367	64368	51	Male	47	17	10	26	Basic	Annual	544	
64368	64369	45	Male	3	1	5	11	Standard	Annual	218	
64369	64370	45	Female	33	12	6	21	Basic	Quarterly	947	
64370	64371	37	Male	6	1	5	22	Standard	Annual	923	
64371	64372	25	Male	39	14	8	30	Premium	Monthly	327	
64372	64373	50	Female	18	19	7	22	Standard	Monthly	540	
64373	64374	52	Female	45	15	9	25	Standard	Monthly	696	

```
print("Shape of Dataset:")
print(df.shape)
```

```
Shape of Dataset:
(64374, 12)
```

```
print("Column Names:")
print(df.columns.tolist())
```

```
Column Names:
['CustomerID', 'Age', 'Gender', 'Tenure', 'Usage Frequency', 'Support Calls', 'Payment Delay', 'Subscript
```

```
print("Dataset Information:")
df.info()
```

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 64374 entries, 0 to 64373
Data columns (total 12 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   CustomerID         64374 non-null  int64
 1   Age                64374 non-null  int64
 2   Gender             64374 non-null  object
 3   Tenure             64374 non-null  int64
 4   Usage Frequency    64374 non-null  int64
 5   Support Calls      64374 non-null  int64
 6   Payment Delay      64374 non-null  int64
 7   Subscription Type  64374 non-null  object
 8   Contract Length    64374 non-null  object
 9   Total Spend        64374 non-null  int64
 10  Last Interaction   64374 non-null  int64
 11  Churn              64374 non-null  int64
dtypes: int64(9), object(3)
memory usage: 5.9+ MB
```

```
print("Summary Statistics:")
display(df.describe(include='all'))
```

Summary Statistics:

	CustomerID	Age	Gender	Tenure	Usage Frequency	Support Calls	Payment Delay	Subscri
count	64374.000000	64374.000000	64374	64374.000000	64374.000000	64374.000000	64374.000000	
unique	NaN	NaN	2	NaN	NaN	NaN	NaN	
top	NaN	NaN	Female	NaN	NaN	NaN	NaN	Sta
freq	NaN	NaN	34353	NaN	NaN	NaN	NaN	
mean	32187.500000	41.970982	NaN	31.994827	15.080234	5.400690	17.133952	
std	18583.317451	13.924911	NaN	17.098234	8.816470	3.114005	8.852211	
min	1.000000	18.000000	NaN	1.000000	1.000000	0.000000	0.000000	
25%	16094.250000	30.000000	NaN	18.000000	7.000000	3.000000	10.000000	
50%	32187.500000	42.000000	NaN	33.000000	15.000000	6.000000	19.000000	
75%	48280.750000	54.000000	NaN	47.000000	23.000000	8.000000	25.000000	
max	64374.000000	65.000000	NaN	60.000000	30.000000	10.000000	30.000000	

```
print("Missing Values:")
print(df.isnull().sum())
```

```
Missing Values:
CustomerID           0
Age                  0
Gender               0
Tenure               0
Usage Frequency      0
Support Calls        0
Payment Delay        0
Subscription Type    0
Contract Length      0
Total Spend          0
Last Interaction     0
Churn                0
dtype: int64
```

```
print("Duplicate Records:")
print(df.duplicated().sum())
```

```
Duplicate Records:
0
```

```
print("Duplicates Before Removal:",
      df.duplicated().sum())

df = df.drop_duplicates()

print("Duplicates After Removal:",
      df.duplicated().sum())
```

```
Duplicates Before Removal: 0
Duplicates After Removal: 0
```

```
print(df.isnull().sum())
```

```
CustomerID           0
Age                  0
Gender               0
Tenure               0
Usage Frequency      0
Support Calls        0
Payment Delay        0
Subscription Type    0
Contract Length      0
Total Spend          0
Last Interaction     0
Churn                0
dtype: int64
```

```
numerical_columns = df.select_dtypes(
    include=['int64', 'float64']
).columns

for col in numerical_columns:
    df[col] = df[col].fillna(df[col].median())
```

```
categorical_columns = df.select_dtypes(
    include=['object']
).columns

for col in categorical_columns:
    df[col] = df[col].fillna(df[col].mode()[0])
```

```
print("Missing Values After Cleaning:")
print(df.isnull().sum())
```

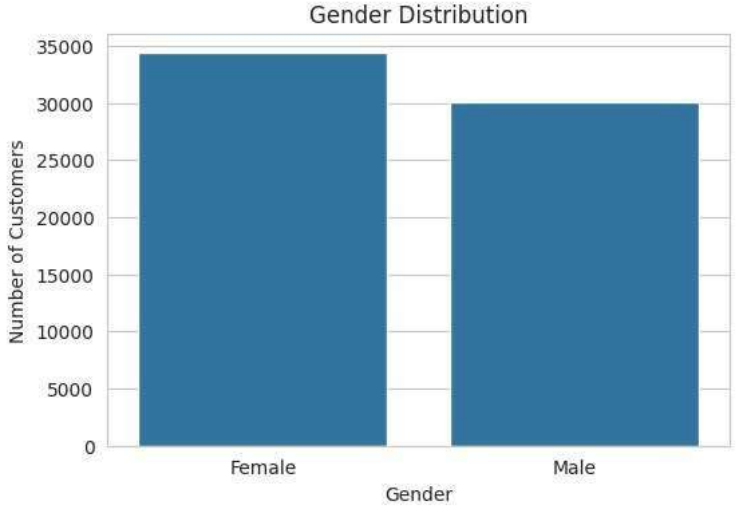
```
Missing Values After Cleaning:
CustomerID           0
Age                  0
Gender               0
Tenure               0
Usage Frequency      0
Support Calls        0
Payment Delay        0
Subscription Type    0
Contract Length      0
Total Spend          0
Last Interaction     0
Churn                0
dtype: int64
```

```
plt.figure(figsize=(6,4))

sns.countplot(data=df, x='Gender')

plt.title("Gender Distribution")
plt.xlabel("Gender")
plt.ylabel("Number of Customers")

plt.show()
```

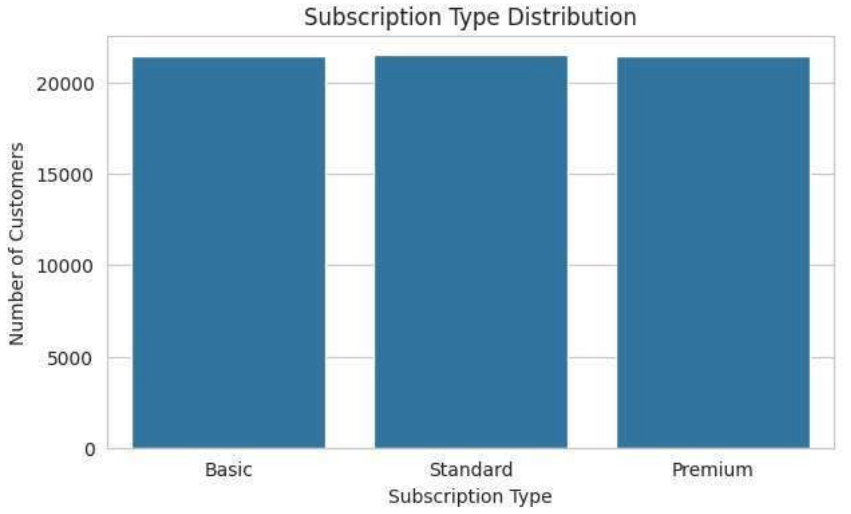



```
plt.figure(figsize=(7,4))

sns.countplot(data=df, x='Subscription Type')

plt.title("Subscription Type Distribution")
plt.xlabel("Subscription Type")
plt.ylabel("Number of Customers")
```

```
plt.show()
```

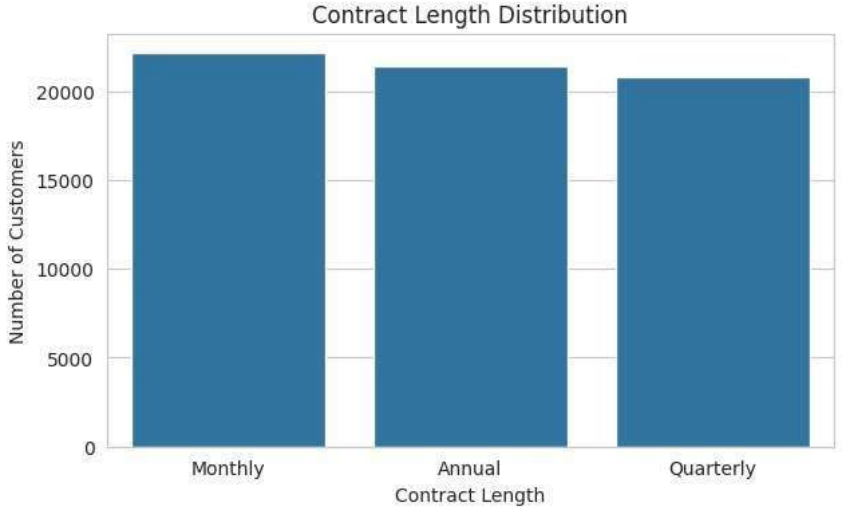



```
plt.figure(figsize=(7,4))

sns.countplot(data=df, x='Contract Length')

plt.title("Contract Length Distribution")
plt.xlabel("Contract Length")
plt.ylabel("Number of Customers")

plt.show()
```

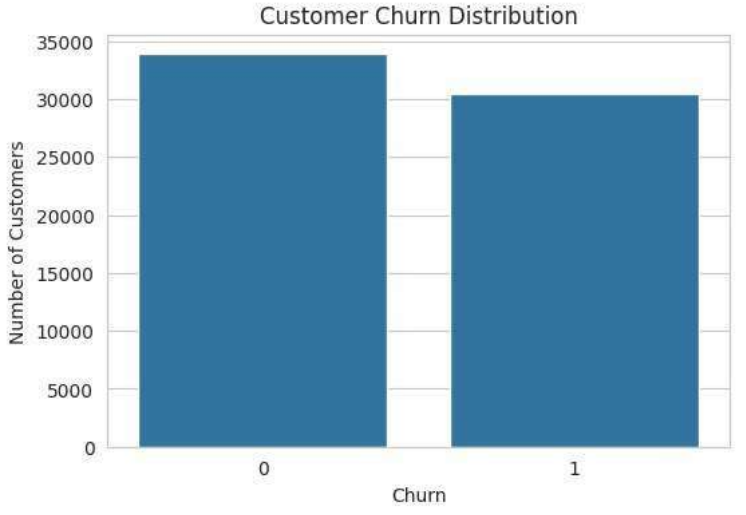



```
plt.figure(figsize=(6,4))

sns.countplot(data=df, x='Churn')

plt.title("Customer Churn Distribution")
plt.xlabel("Churn")
plt.ylabel("Number of Customers")

plt.show()
```

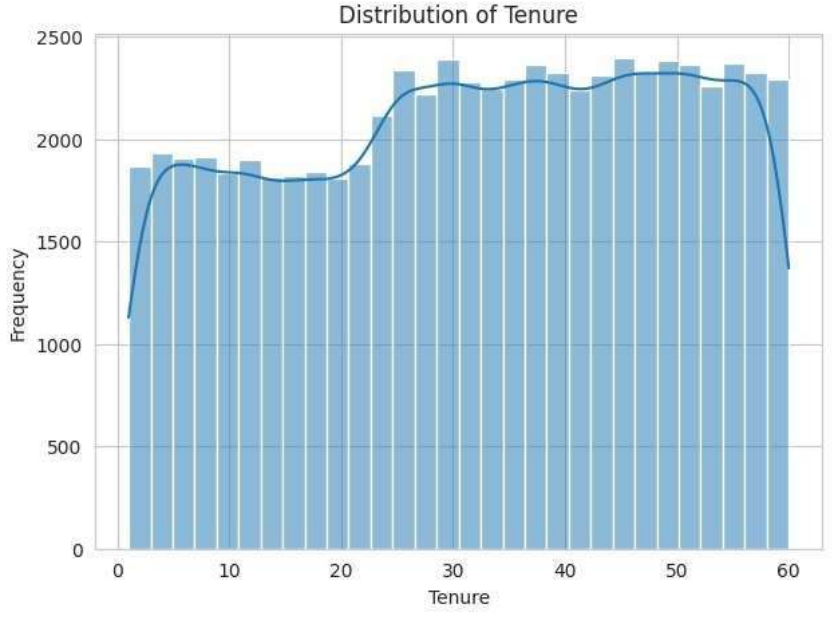



```
plt.figure(figsize=(7,5))

sns.histplot(
    data=df,
    x='Tenure',
    bins=30,
    kde=True
)

plt.title("Distribution of Tenure")
plt.xlabel("Tenure")
plt.ylabel("Frequency")

plt.show()
```

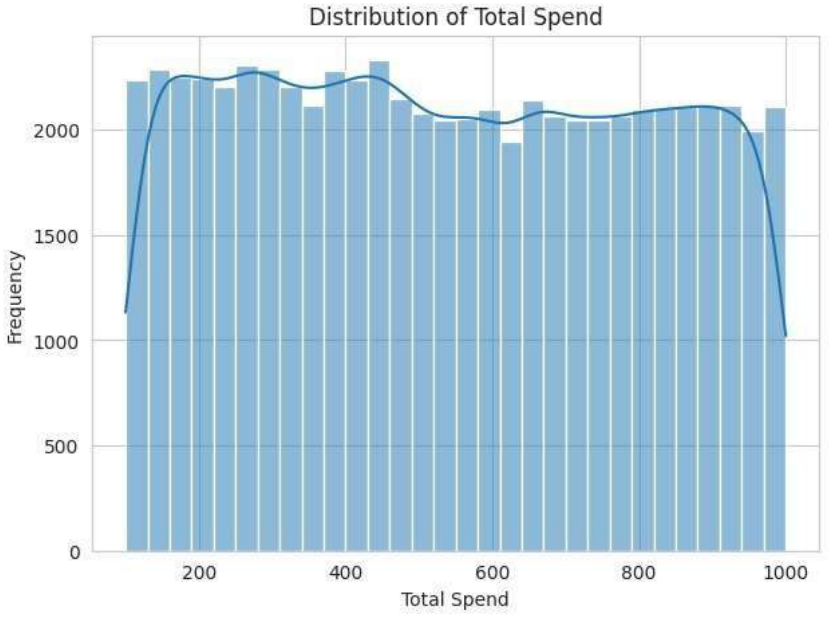



```
plt.figure(figsize=(7,5))

sns.histplot(
    data=df,
    x='Total Spend',
    bins=30,
    kde=True
)
```

```
plt.title("Distribution of Total Spend")
plt.xlabel("Total Spend")
plt.ylabel("Frequency")

plt.show()
```

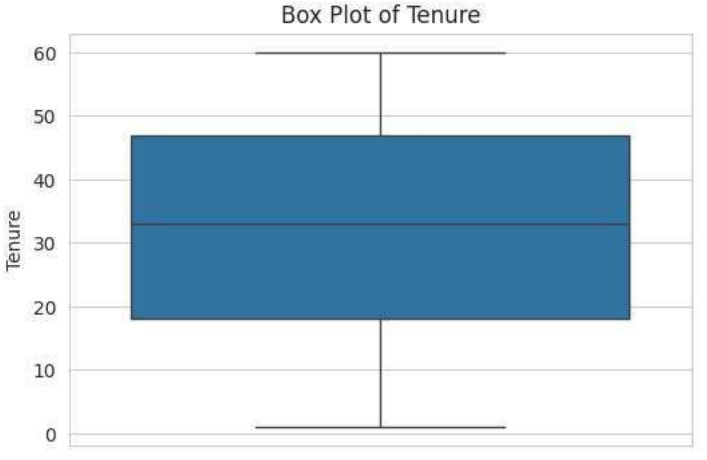



```
plt.figure(figsize=(6,4))

sns.boxplot(data=df, y='Tenure')

plt.title("Box Plot of Tenure")

plt.show()
```

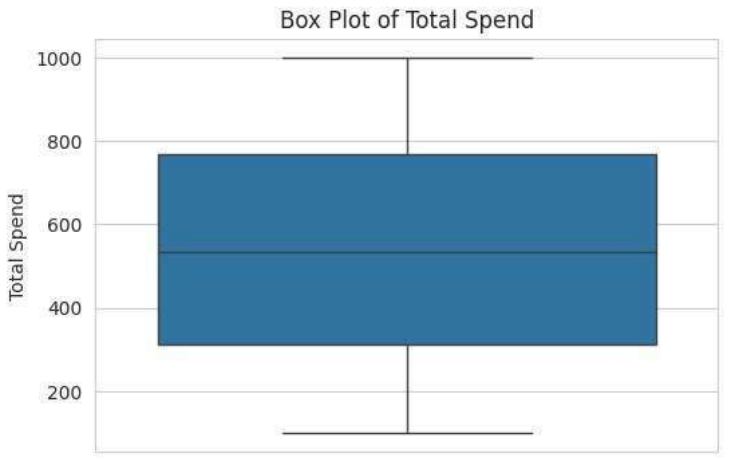



```
plt.figure(figsize=(6,4))

sns.boxplot(data=df, y='Total Spend')

plt.title("Box Plot of Total Spend")

plt.show()
```




```
numerical_features = [
    'Age',
    'Tenure',
    'Usage Frequency',
    'Support Calls',
    'Payment Delay',
    'Total Spend',
    'Last Interaction'
]

for col in numerical_features:

    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)

    IQR = Q3 - Q1

    lower_limit = Q1 - 1.5 * IQR
    upper_limit = Q3 + 1.5 * IQR

    outliers = df[
        (df[col] < lower_limit) |
        (df[col] > upper_limit)
    ]

    print("Feature:", col)
    print("Q1:", Q1)
    print("Q3:", Q3)
    print("IQR:", IQR)
    print("Lower Limit:", lower_limit)
    print("Upper Limit:", upper_limit)
    print("Number of Outliers:", len(outliers))
    print("-" * 50)
```

```
Feature: Age
Q1: 30.0
Q3: 54.0
IQR: 24.0
Lower Limit: -6.0
Upper Limit: 90.0
Number of Outliers: 0
--------------------------------------------------
Feature: Tenure
Q1: 18.0
Q3: 47.0
IQR: 29.0
Lower Limit: -25.5
Upper Limit: 90.5
Number of Outliers: 0
--------------------------------------------------
Feature: Usage Frequency
Q1: 7.0
Q3: 23.0
IQR: 16.0
```

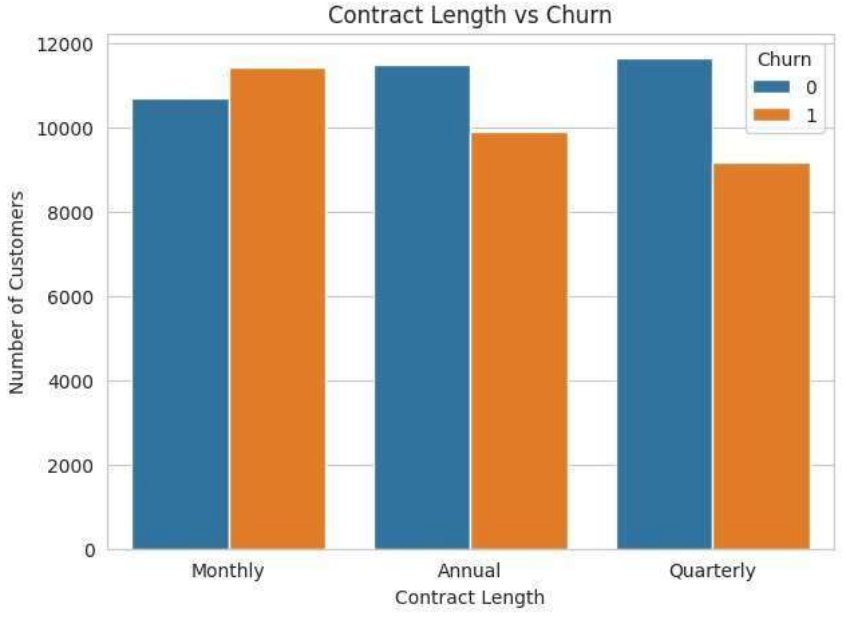
```
Lower Limit: -17.0
Upper Limit: 47.0
Number of Outliers: 0
--------------------------------------------------
Feature: Support Calls
Q1: 3.0
Q3: 8.0
IQR: 5.0
Lower Limit: -4.5
Upper Limit: 15.5
Number of Outliers: 0
--------------------------------------------------
Feature: Payment Delay
Q1: 10.0
Q3: 25.0
IQR: 15.0
Lower Limit: -12.5
Upper Limit: 47.5
Number of Outliers: 0
--------------------------------------------------
Feature: Total Spend
Q1: 313.0
Q3: 768.0
IQR: 455.0
Lower Limit: -369.5
Upper Limit: 1450.5
Number of Outliers: 0
--------------------------------------------------
Feature: Last Interaction
Q1: 8.0
Q3: 23.0
IQR: 15.0
Lower Limit: -14.5
Upper Limit: 45.5
Number of Outliers: 0
--------------------------------------------------
```

```
plt.figure(figsize=(7,5))

sns.countplot(
    data=df,
    x='Contract Length',
    hue='Churn'
)

plt.title("Contract Length vs Churn")
plt.xlabel("Contract Length")
plt.ylabel("Number of Customers")

plt.show()
```

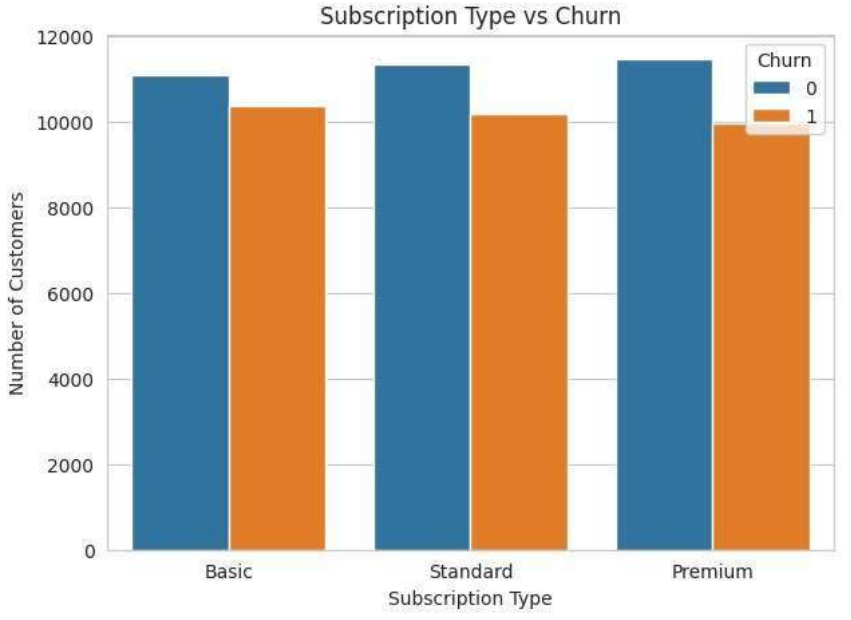


```
plt.figure(figsize=(7,5))

sns.countplot(
    data=df,
    x='Subscription Type',
    hue='Churn'
)

plt.title("Subscription Type vs Churn")
plt.xlabel("Subscription Type")
plt.ylabel("Number of Customers")

plt.show()
```

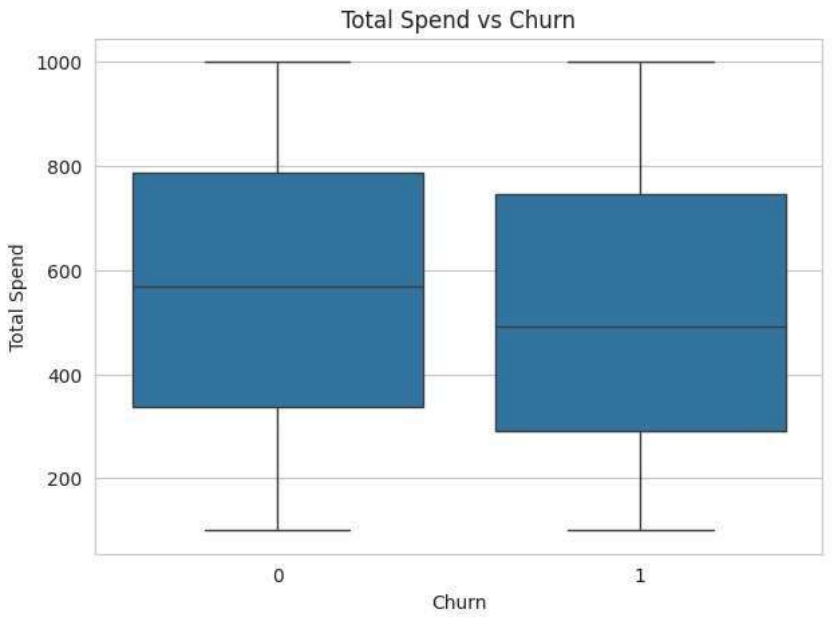



```
plt.figure(figsize=(7,5))

sns.boxplot(
    data=df,
    x='Churn',
    y='Total Spend'
)

plt.title("Total Spend vs Churn")
plt.xlabel("Churn")
plt.ylabel("Total Spend")

plt.show()
```

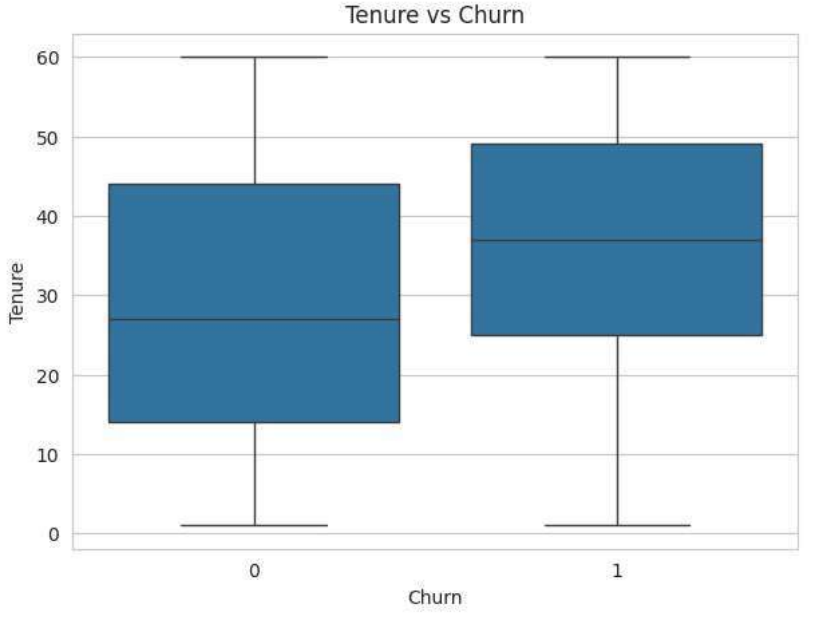



```
plt.figure(figsize=(7,5))

sns.boxplot(
    data=df,
    x='Churn',
    y='Tenure'
)

plt.title("Tenure vs Churn")
plt.xlabel("Churn")
plt.ylabel("Tenure")

plt.show()
```




```
contract_churn = df[df['Churn'] == 1]['Contract Length'].value_counts()

print(contract_churn)

print(
    "Contract with highest churn:",
```

```
    contract_churn.idxmax()
)
```

```
Contract Length
Monthly      11421
Annual        9895
Quarterly     9177
Name: count, dtype: int64
Contract with highest churn: Monthly
```

```
tenure_churn = df.groupby('Churn')['Tenure'].mean()

print(tenure_churn)
```

```
Churn
0    28.826481
1    35.515200
Name: Tenure, dtype: float64
```

```
print("Average Tenure of Non-Churn Customers:",
      df[df['Churn'] == 0]['Tenure'].mean())

print("Average Tenure of Churn Customers:",
      df[df['Churn'] == 1]['Tenure'].mean())
```

```
Average Tenure of Non-Churn Customers: 28.8264809185089
Average Tenure of Churn Customers: 35.515200209884235
```

```
spending = df.groupby(
    'Subscription Type'
)['Total Spend'].mean()

print(spending)

print(
    "Customer group with highest average spending:",
    spending.idxmax()
)
```

```
Subscription Type
Basic       540.908536
Premium     543.581579
Standard    538.589387
Name: Total Spend, dtype: float64
Customer group with highest average spending: Premium
```

```
subscription_churn = df[
    df['Churn'] == 1
]['Subscription Type'].value_counts()

print(subscription_churn)

print(
    "Subscription Type with highest churn:",
    subscription_churn.idxmax()
)
```

```
Subscription Type
Basic       10356
Standard    10177
Premium      9960
Name: count, dtype: int64
Subscription Type with highest churn: Basic
```

```
print(
    df.groupby('Churn')['Total Spend'].mean()
)
```

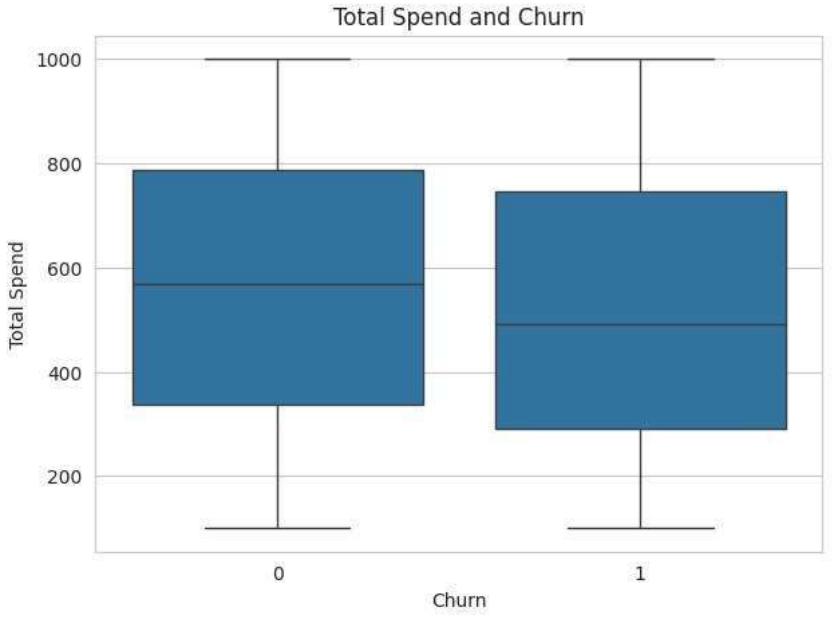
```
Churn
0    560.541956
1    519.336143
Name: Total Spend, dtype: float64
```

```
plt.figure(figsize=(7,5))

sns.boxplot(
    data=df,
    x='Churn',
    y='Total Spend'
)

plt.title("Total Spend and Churn")

plt.show()
```




```
encoded_df = df.copy()
```

```
le = LabelEncoder()

encoded_df['Gender'] = le.fit_transform(
    encoded_df['Gender']
)
```

```
encoded_df['Subscription Type'] = le.fit_transform(
    encoded_df['Subscription Type']
)
```

```
encoded_df['Contract Length'] = le.fit_transform(
    encoded_df['Contract Length']
)
```

```
scale_columns = [
    'Age',
    'Tenure',
    'Usage Frequency',
    'Support Calls',
    'Payment Delay',
    'Total Spend',
    'Last Interaction'
]

scaler = StandardScaler()

encoded_df[scale_columns] = scaler.fit_transform(
    encoded_df[scale_columns]
)
```

```
   CustomerID       Age  Gender    Tenure  Usage Frequency  Support Calls  \
0           1 -1.434202       0 -0.409100        -0.122526      -0.449807
1           2 -0.069730       0 -0.233642         1.465424       0.513590
2           3  0.361155       1 -0.292128        -0.576225      -1.092072
3           4 -0.500616       1 -1.344876        -0.349375      -0.128674
4           5  0.792041       0  1.520939         1.011724       1.155855

   Payment Delay  Subscription Type  Contract Length  Total Spend  \
0       1.114538                  0                1     0.218408
1      -0.467000                  2                1     0.164742
2       1.340472                  1                0     0.827900
3      -0.015132                  1                2    -1.184575
4      -1.709637                  2                0    -0.030756

   Last Interaction  Churn
0         -0.752324      1
1          0.521065      0
2          0.636827      0
3          0.289540      0
4          0.289540      0
```

```
correlation = encoded_df.corr()['Churn'].sort_values(
    ascending=False
)

print(correlation)
```

```
Churn                1.000000
Payment Delay        0.557386
CustomerID           0.529832
Support Calls        0.304631
Tenure               0.195327
Age                  0.063457
Last Interaction    -0.002818
Subscription Type   -0.007738
Contract Length     -0.017098
Total Spend         -0.078867
Usage Frequency     -0.115098
Gender              -0.164549
Name: Churn, dtype: float64
```

```
X = encoded_df.drop('Churn', axis=1)

y = encoded_df['Churn']

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

```
Training data: (51499, 11)
Testing data: (12875, 11)
```

```
encoded_df.to_csv(
    "/content/CustomerChurn_Preprocessed.csv",
    index=False
)

print("File saved successfully!")
```

```
File saved successfully!
```

LEARNING OUTCOMES

After completing this experiment, I was able to:

- Explore and understand the customer churn dataset.
- Perform exploratory data analysis using different visualizations.
- Identify and handle data quality issues such as missing values and duplicate records.
- Detect outliers using the Interquartile Range (IQR) method.
- Analyze the relationship between customer features and churn.
- Identify important features that influence customer churn.
- Prepare the dataset for predictive analytics.
- Split the dataset into training and testing datasets.
- Save the processed dataset for future machine learning applications.

Arun S (241801027)

AIM

To build and evaluate classification models for predicting customer churn using Logistic Regression, Decision Tree, and Random Forest algorithms, and to compare their performance using suitable evaluation metrics.

LEARNING OBJECTIVES:

To prepare the customer churn dataset for classification.

To divide the dataset into 80% training and 20% testing data.

To implement Logistic Regression, Decision Tree, and Random Forest classifiers.

To evaluate the models using Confusion Matrix, Accuracy, Precision, Recall, and F1-Score.

To compare the performance of different classification models.

To identify the best-performing model for customer churn prediction.

To understand important features that influence customer churn.

To provide suitable business recommendations based on the model results.

FUNCTIONS / METHODS USED:

Function / Method Purpose

pd.read_csv() Load the dataset

drop() Remove unnecessary columns

pd.get_dummies() Encode categorical features

train_test_split() Split data into training and testing sets

StandardScaler() Standardize numerical features

fit() Train the machine learning model

predict() Predict customer churn

LogisticRegression() Build Logistic Regression model

DecisionTreeClassifier() Build Decision Tree model

RandomForestClassifier() Build Random Forest model

confusion_matrix() Calculate confusion matrix

accuracy_score() Calculate accuracy

precision_score() Calculate precision

recall_score() Calculate recall

f1_score() Calculate F1-score

feature_importances_ Find important features in Random Forest

sns.heatmap() Visualize confusion matrix

sns.barplot() Visualize feature importance

plot(kind='bar') Compare model performance

display() Display DataFrame output

1. **Import Libraries**

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import (
    confusion_matrix,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score
)

sns.set_style("whitegrid")
```

2. **Load CustomerChurn_Preprocessed.csv**

```
df = pd.read_csv("/content/customer_churn_dataset-testing-master.csv.zip")

print("Dataset Loaded Successfully")
print("Shape:", df.shape)

display(df.head())
```

```
Dataset Loaded Successfully
Shape: (64374, 12)
```

	CustomerID	Age	Gender	Tenure	Usage Frequency	Support Calls	Payment Delay	Subscription Type
0	1	22	Female	25	14	4	27	Basic
1	2	41	Female	28	28	7	13	Standard
2	3	47	Male	27	10	2	29	Premium
3	4	35	Male	9	12	5	17	Premium

3. Check Dataset

```
print("Column Names:")
print(df.columns.tolist())

print("\nMissing Values:")
print(df.isnull().sum())

print("\nDuplicate Records:")
print(df.duplicated().sum())
```

```
Column Names:
['CustomerID', 'Age', 'Gender', 'Tenure', 'Usage Frequency', 'Support Calls',

Missing Values:
CustomerID           0
Age                  0
Gender               0
Tenure               0
Usage Frequency      0
Support Calls        0
Payment Delay        0
Subscription Type    0
Contract Length      0
Total Spend          0
Last Interaction     0
Churn                0
dtype: int64

Duplicate Records:
0
```

4. Clean Missing Values

```
# Numerical columns
numerical_columns = df.select_dtypes(
    include=['int64', 'float64']
).columns

for col in numerical_columns:
```

```
        df[col] = df[col].fillna(df[col].median())

    # Categorical columns
    categorical_columns = df.select_dtypes(
        include=['object']
    ).columns

    for col in categorical_columns:
        df[col] = df[col].fillna(df[col].mode()[0])

    print("Missing values after cleaning:")
    print(df.isnull().sum())
```

```
Missing values after cleaning:
CustomerID           0
Age                  0
Gender               0
Tenure               0
Usage Frequency      0
Support Calls        0
Payment Delay        0
Subscription Type    0
Contract Length      0
Total Spend          0
Last Interaction     0
Churn                0
dtype: int64
```

```
print("Duplicates before removal:", df.duplicated().sum())

df = df.drop_duplicates()

print("Duplicates after removal:", df.duplicated().sum())
```

```
Duplicates before removal: 0
Duplicates after removal: 0
```

```
X = df.drop(['Churn', 'CustomerID'], axis=1)

y = df['Churn']

print("Features:")
print(X.columns.tolist())

print("\nTarget:")
print(y.name)
```

```
Features:
['Age', 'Gender', 'Tenure', 'Usage Frequency', 'Support Calls', 'Payment Dela

Target:
Churn
```

```
X = pd.get_dummies(
    X,
    columns=[
        'Gender',
        'Subscription Type',
        'Contract Length'
    ],
    drop_first=True
)

print("Encoded Features:")
print(X.columns.tolist())

print("\nShape after encoding:")
print(X.shape)
```

```
Encoded Features:
['Age', 'Tenure', 'Usage Frequency', 'Support Calls', 'Payment Delay', 'Total

Shape after encoding:
(64374, 12)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

```
Training data: (51499, 12)
Testing data: (12875, 12)
```

```
print(X_train.columns.tolist())
```

```
['Age', 'Tenure', 'Usage Frequency', 'Support Calls', 'Payment Delay', 'Total
```

```
from sklearn.preprocessing import StandardScaler

# Numerical columns in your dataset
scale_columns = [
    'Age',
    'Tenure',
    'Usage Frequency',
    'Support Calls',
    'Payment Delay',
    'Total Spend',
    'Last Interaction'
]
```

```
# Create scaler
scaler = StandardScaler()

# Make copies
X_train = X_train.copy()
X_test = X_test.copy()

# Standardize only numerical columns
X_train[scale_columns] = scaler.fit_transform(
    X_train[scale_columns]
)

X_test[scale_columns] = scaler.transform(
    X_test[scale_columns]
)

print("Standardization completed successfully!")
```

```
Standardization completed successfully!
```

```
logistic_model = LogisticRegression(
    max_iter=1000,
    random_state=42
)

logistic_model.fit(X_train, y_train)

y_pred_logistic = logistic_model.predict(X_test)

print("Logistic Regression completed.")
```

```
Logistic Regression completed.
```

```
decision_tree_model = DecisionTreeClassifier(
    random_state=42
)

decision_tree_model.fit(X_train, y_train)

y_pred_tree = decision_tree_model.predict(X_test)

print("Decision Tree completed.")
```

```
Decision Tree completed.
```

```
random_forest_model = RandomForestClassifier(
    n_estimators=100,
    random_state=42,
    n_jobs=-1
)

random_forest_model.fit(X_train, y_train)
```

```
y_pred_forest = random_forest_model.predict(X_test)

print("Random Forest completed.")
```

```
Random Forest completed.
```

```
def evaluate_model(model_name, y_test, y_pred):

    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(
        y_test,
        y_pred,
        zero_division=0
    )
    recall = recall_score(
        y_test,
        y_pred,
        zero_division=0
    )
    f1 = f1_score(
        y_test,
        y_pred,
        zero_division=0
    )

    cm = confusion_matrix(y_test, y_pred)

    print("\n" + "=" * 50)
    print(model_name)
    print("=" * 50)

    print("Accuracy :", round(accuracy, 4))
    print("Precision:", round(precision, 4))
    print("Recall   :", round(recall, 4))
    print("F1-Score :", round(f1, 4))

    print("\nConfusion Matrix:")
    print(cm)

    return accuracy, precision, recall, f1
```

```
logistic_results = evaluate_model(
    "Logistic Regression",
    y_test,
    y_pred_logistic
)
```

```

==================================================
Logistic Regression
==================================================
Accuracy : 0.8272
Precision: 0.8139
Recall   : 0.8234
```

```
F1-Score : 0.8186

Confusion Matrix:
[[5628 1148]
 [1077 5022]]
```

```
tree_results = evaluate_model(
    "Decision Tree",
    y_test,
    y_pred_tree
)
```

```
==================================================
Decision Tree
==================================================
Accuracy : 0.9981
Precision: 0.9974
Recall   : 0.9985
F1-Score : 0.998

Confusion Matrix:
[[6760   16]
 [   9 6090]]
```

```
forest_results = evaluate_model(
    "Random Forest",
    y_test,
    y_pred_forest
)
```

```
==================================================
Random Forest
==================================================
Accuracy : 0.9981
Precision: 0.999
Recall   : 0.997
F1-Score : 0.998

Confusion Matrix:
[[6770    6]
 [  18 6081]]
```

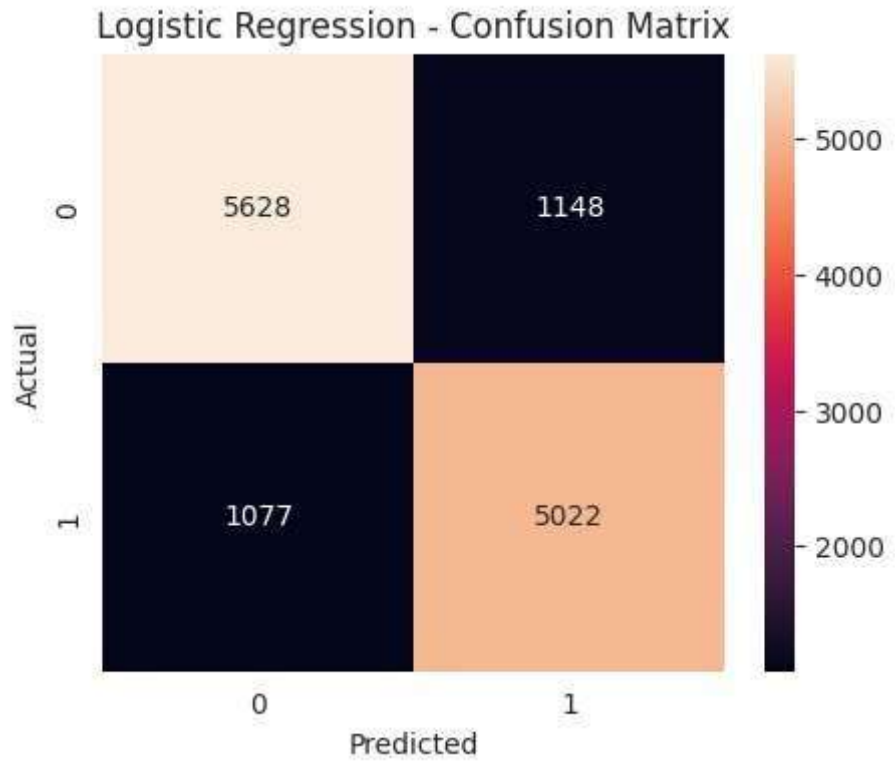
```
cm_logistic = confusion_matrix(
    y_test,
    y_pred_logistic
)

plt.figure(figsize=(5,4))

sns.heatmap(
    cm_logistic,
    annot=True,
    fmt='d'
)
```

```
plt.title("Logistic Regression - Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.show()
```



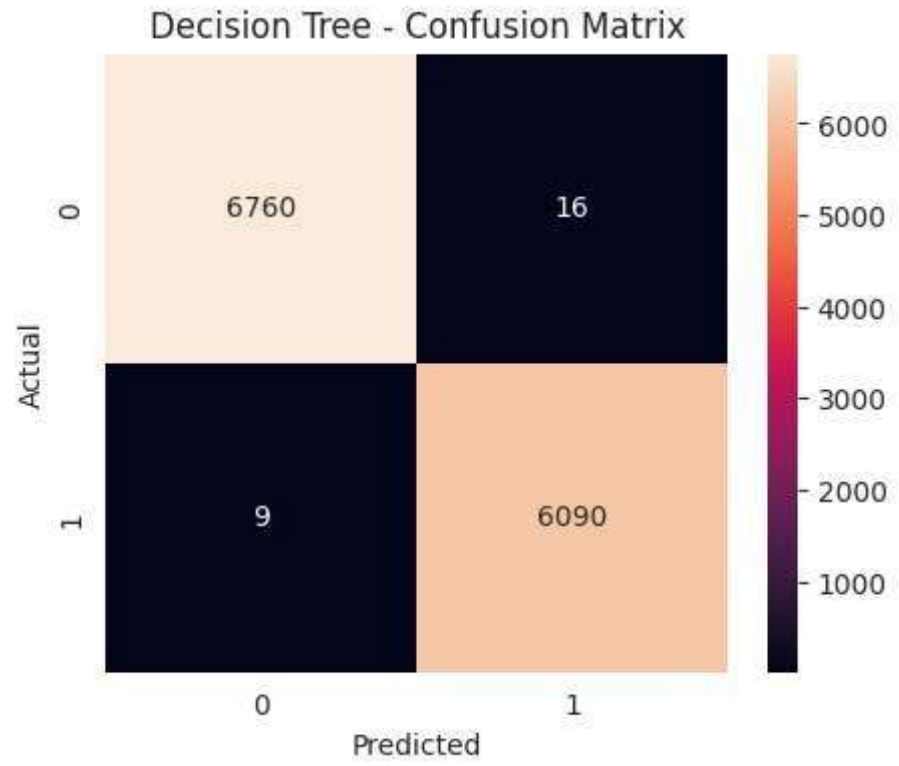

```
cm_tree = confusion_matrix(
    y_test,
    y_pred_tree
)

plt.figure(figsize=(5,4))

sns.heatmap(
    cm_tree,
    annot=True,
    fmt='d'
)

plt.title("Decision Tree - Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.show()
```



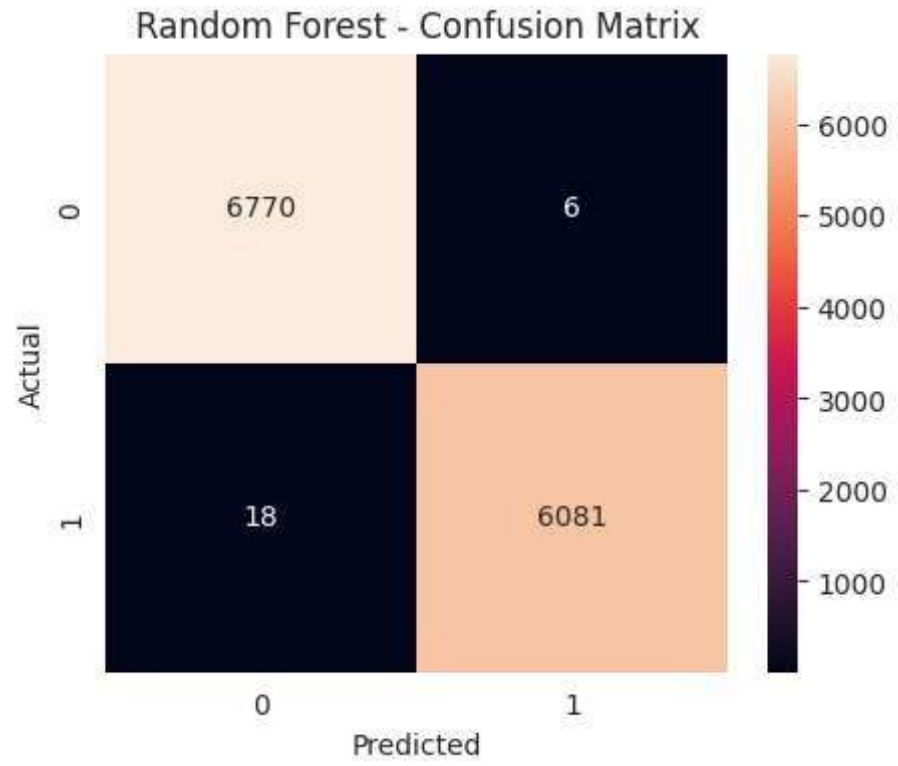

```
cm_forest = confusion_matrix(
    y_test,
    y_pred_forest
)

plt.figure(figsize=(5,4))

sns.heatmap(
    cm_forest,
    annot=True,
    fmt='d'
)

plt.title("Random Forest - Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.show()
```




```
results = pd.DataFrame({
    'Model': [
        'Logistic Regression',
        'Decision Tree',
        'Random Forest'
    ],

    'Accuracy': [
        logistic_results[0],
        tree_results[0],
        forest_results[0]
    ],

    'Precision': [
        logistic_results[1],
        tree_results[1],
        forest_results[1]
    ],

    'Recall': [
        logistic_results[2],
        tree_results[2],
        forest_results[2]
    ],

    'F1-Score': [
        logistic_results[3],
        tree_results[3],
        forest_results[3]
    ]
})
```

```
display(results)
```

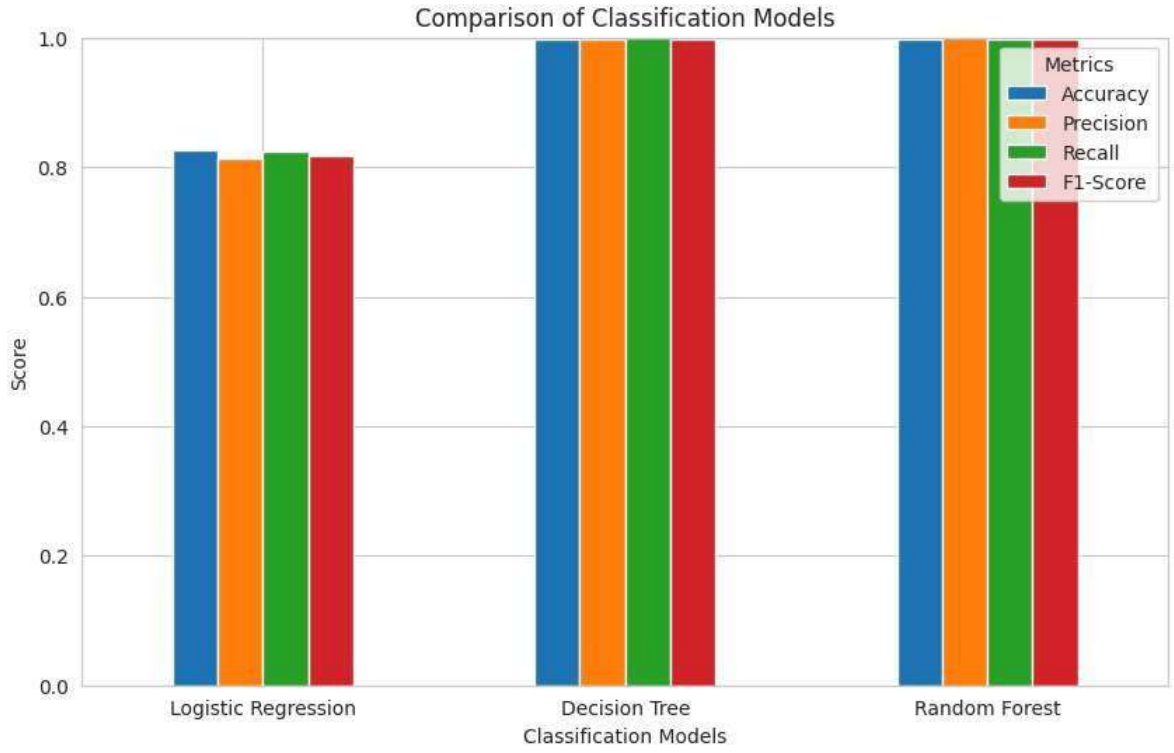
	Model	Accuracy	Precision	Recall	F1-Score
0	Logistic Regression	0.827184	0.813938	0.823414	0.818649
1	Decision Tree	0.998058	0.997380	0.998524	0.997952
2	Random Forest	0.998136	0.999014	0.997049	0.998031

```
results.set_index('Model').plot(
    kind='bar',
    figsize=(10,6)
)

plt.title("Comparison of Classification Models")
plt.xlabel("Classification Models")
plt.ylabel("Score")
plt.ylim(0, 1)

plt.xticks(rotation=0)
plt.legend(title="Metrics")

plt.show()
```




```
best_model = results.loc[
    results['F1-Score'].idxmax()
]

print("Best Performing Model:")
print(best_model['Model'])

print(
    "Best F1-Score:",
    round(best_model['F1-Score'], 4)
)
```

```
Best Performing Model:
Random Forest
Best F1-Score: 0.998
```

```
feature_importance = pd.DataFrame({
    'Feature': X.columns,
    'Importance': random_forest_model.feature_importances_
})

feature_importance = feature_importance.sort_values(
    by='Importance',
    ascending=False
```

```
)

display(feature_importance)
```

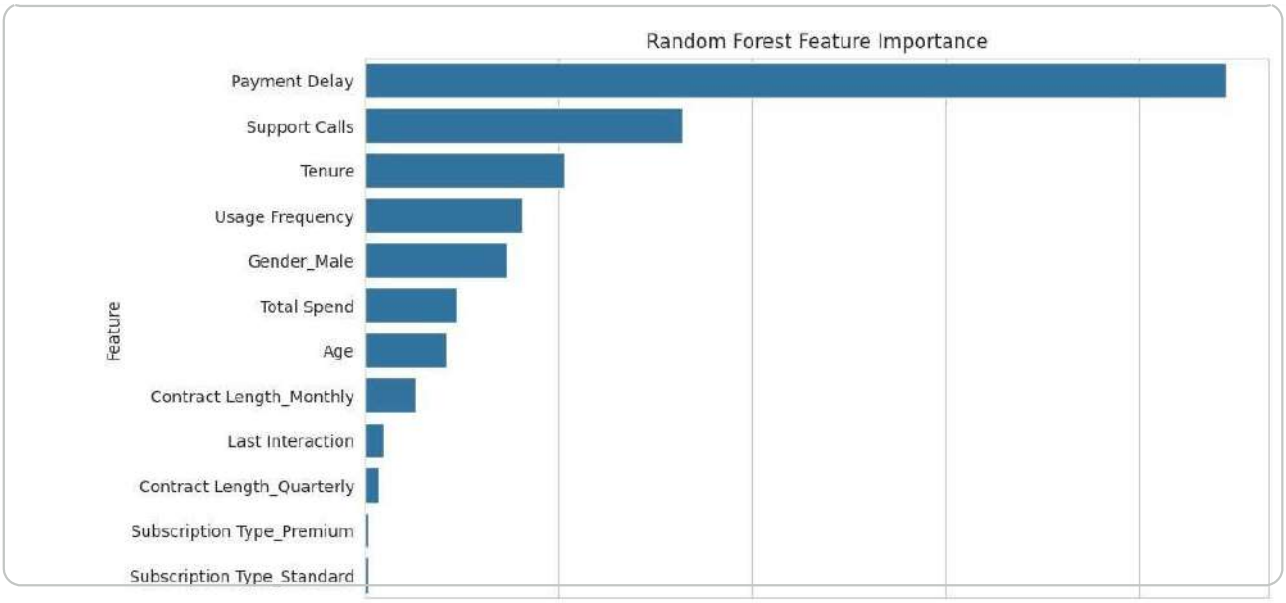
	Feature	Importance
4	Payment Delay	0.444717
3	Support Calls	0.163500
1	Tenure	0.102799
2	Usage Frequency	0.080960
7	Gender_Male	0.073298
5	Total Spend	0.047130
0	Age	0.041765
10	Contract Length_Monthly	0.025525
6	Last Interaction	0.009657
11	Contract Length_Quarterly	0.007026
8	Subscription Type_Premium	0.001850
9	Subscription Type_Standard	0.001772

```
plt.figure(figsize=(10,6))

sns.barplot(
    data=feature_importance,
    x='Importance',
    y='Feature'
)

plt.title("Random Forest Feature Importance")
plt.xlabel("Importance")
plt.ylabel("Feature")

plt.show()
```




```
print("BUSINESS RECOMMENDATIONS")
print("=" * 50)

print("1. Identify high-risk customers using the best-performing "
      "churn prediction model and provide targeted retention offers.")

print("2. Customers with frequent support calls or payment delays "
      "should receive proactive customer support.")

print("3. Focus retention campaigns on customer groups and contract "
      "types that show higher churn rates.")
```

```
BUSINESS RECOMMENDATIONS
==================================================
1. Identify high-risk customers using the best-performing churn prediction mo
2. Customers with frequent support calls or payment delays should receive pro
3. Focus retention campaigns on customer groups and contract types that show
```

```
print("EXPERIMENT SUMMARY")
print("=" * 50)

print("Dataset Shape:", df.shape)
print("Training Samples:", X_train.shape[0])
print("Testing Samples:", X_test.shape[0])

print("\nModel Performance:")
display(results)
```

LEARNING OUTCOMES

After completing this experiment, I was able to:

EXPERIMENT 3A: MODEL PLANNING AND BUILDING – CLASSIFICATION

- Prepare the customer churn dataset for classification.
- Divide the dataset into training and testing data.
- Implement Logistic Regression, Decision Tree, and Random Forest classifiers.
- Evaluate model performance using a confusion matrix and suitable classification metrics such as accuracy, precision, recall, and F1-score.
- Compare the performance of the classification models.
- Identify the best-performing model for customer churn prediction.
- Understand the important features that influence customer churn.
- Derive suitable business recommendations from the model results.

Arun S (241801027)

AIM:

To build and evaluate regression models using the Advertising dataset to predict Sales based on TV, Radio, and Newspaper advertising expenditure.

LEARNING OBJECTIVES:

To understand the fundamentals of regression analysis.

To load and explore the Advertising dataset.

To identify the relationship between advertising expenditure and sales.

To visualize relationships using Pair Plot and Correlation Heatmap.

To split the dataset into 80% training and 20% testing data.

To implement Linear Regression.

To implement Polynomial Regression with degree 2.

To implement Support Vector Regression (SVR) with RBF kernel.

To evaluate and compare the regression models using appropriate performance metrics.

To identify the best-performing regression model.

FUNCTIONS / METHODS USED:

Function / Method Purpose

pd.read_csv() Load the Advertising dataset

df.head() Display first records

df.tail() Display last records

df.shape Find dataset dimensions

df.dtypes Display data types

df.isnull().sum() Check missing values

df.duplicated() Check duplicate records

df.describe() Generate summary statistics

df.corr() Calculate correlation

sns.pairplot() Visualize relationships between variables

sns.heatmap() Display correlation heatmap

train_test_split() Split data into training and testing sets

LinearRegression() Create Linear Regression model

PolynomialFeatures() Generate polynomial features

Pipeline() Combine preprocessing and model steps

StandardScaler() Standardize numerical features

SVR() Create Support Vector Regression model

fit() Train the regression model

predict() Predict Sales values

mean_absolute_error() Calculate MAE

mean_squared_error() Calculate MSE

np.sqrt() Calculate RMSE

r2_score() Calculate R^2 score

plot(kind='bar') Compare model performance

plt.scatter() Plot actual vs predicted values

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.svm import SVR

from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)

sns.set_style("whitegrid")
```

```
df = pd.read_csv("/content/Advertising.csv")

print("Dataset Loaded Successfully")
print("Shape:", df.shape)

display(df.head())
```

```
Dataset Loaded Successfully
Shape: (200, 5)
```

	Unnamed: 0	TV	Radio	Newspaper	Sales
0	1	230.1	37.8	69.2	22.1
1	2	44.5	39.3	45.1	10.4
2	3	17.2	45.9	69.3	9.3
3	4	151.5	41.3	58.5	18.5
4	5	180.8	10.8	58.4	12.9

```
import pandas as pd

df = pd.read_csv("/content/Advertising.csv")

print("Advertising Dataset Loaded Successfully")
print("Shape:", df.shape)

display(df.head())
```

```
Advertising Dataset Loaded Successfully
Shape: (200, 5)
```

	Unnamed: 0	TV	Radio	Newspaper	Sales
0	1	230.1	37.8	69.2	22.1
1	2	44.5	39.3	45.1	10.4
2	3	17.2	45.9	69.3	9.3
3	4	151.5	41.3	58.5	18.5
4	5	180.8	10.8	58.4	12.9

```
print("First 5 Records:")
display(df.head(5))
```

```
First 5 Records:
```

	Unnamed: 0	TV	Radio	Newspaper	Sales
0	1	230.1	37.8	69.2	22.1
1	2	44.5	39.3	45.1	10.4
2	3	17.2	45.9	69.3	9.3
3	4	151.5	41.3	58.5	18.5
4	5	180.8	10.8	58.4	12.9

```
print("Last 5 Records:")
display(df.tail(5))
```

Last 5 Records:

	Unnamed: 0	TV	Radio	Newspaper	Sales
195	196	38.2	3.7	13.8	7.6
196	197	94.2	4.9	8.1	9.7
197	198	177.0	9.3	6.4	12.8
198	199	283.6	42.0	66.2	25.5
199	200	232.1	8.6	8.7	13.4

```
print("Shape of Dataset:")
print(df.shape)
```

```
Shape of Dataset:
(200, 5)
```

```
print("Data Types:")
print(df.dtypes)
```

```
Data Types:
Unnamed: 0      int64
TV            float64
Radio         float64
Newspaper     float64
Sales         float64
dtype: object
```

```
print("Missing Values:")
print(df.isnull().sum())
```

```
Missing Values:
Unnamed: 0    0
TV            0
Radio         0
Newspaper     0
Sales         0
dtype: int64
```

```
print("Number of Duplicate Records:")
print(df.duplicated().sum())
```

```
Number of Duplicate Records:
0
```

```
print("Summary Statistics:")
display(df.describe())
```

Summary Statistics:

	Unnamed: 0	TV	Radio	Newspaper	Sales
count	200.000000	200.000000	200.000000	200.000000	200.000000
mean	100.500000	147.042500	23.264000	30.554000	14.022500
std	57.879185	85.854236	14.846809	21.778621	5.217457
min	1.000000	0.700000	0.000000	0.300000	1.600000
25%	50.750000	74.375000	9.975000	12.750000	10.375000
50%	100.500000	149.750000	22.900000	25.750000	12.900000
75%	150.250000	218.825000	36.525000	45.100000	17.400000
max	200.000000	296.400000	49.600000	114.000000	27.000000

```
X = df[['TV', 'Radio', 'Newspaper']]

y = df['Sales']

print("Input Features:")
display(X.head())

print("\nTarget Variable:")
display(y.head())
```

Input Features:

	TV	Radio	Newspaper
0	230.1	37.8	69.2
1	44.5	39.3	45.1
2	17.2	45.9	69.3
3	151.5	41.3	58.5
4	180.8	10.8	58.4

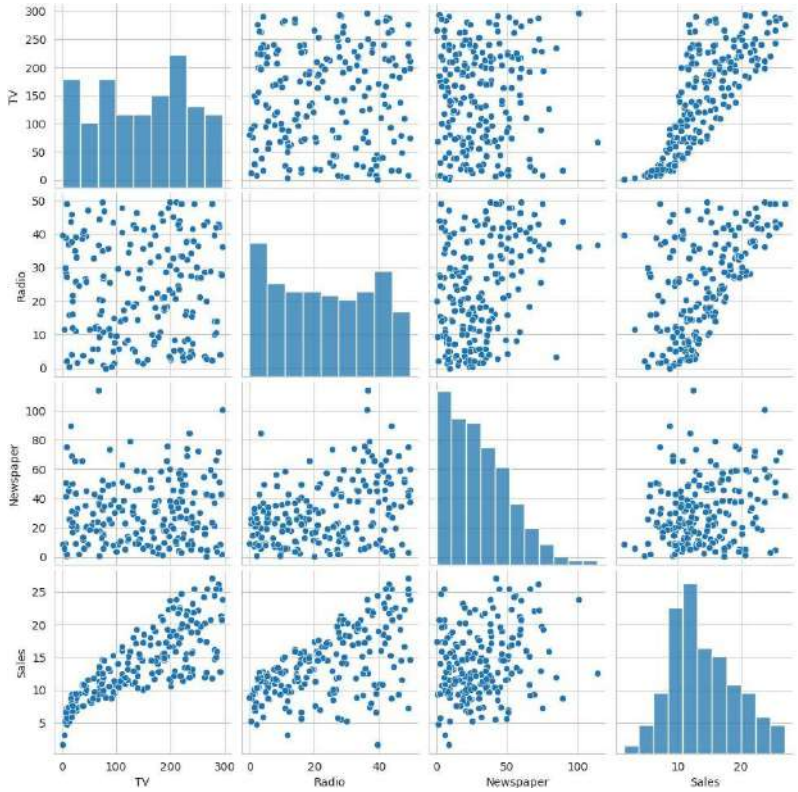
Target Variable:

	Sales
0	22.1
1	10.4
2	9.3
3	18.5
4	12.9

dtype: float64

```
sns.pairplot(df[['TV', 'Radio', 'Newspaper', 'Sales']])
```

```
plt.show()
```



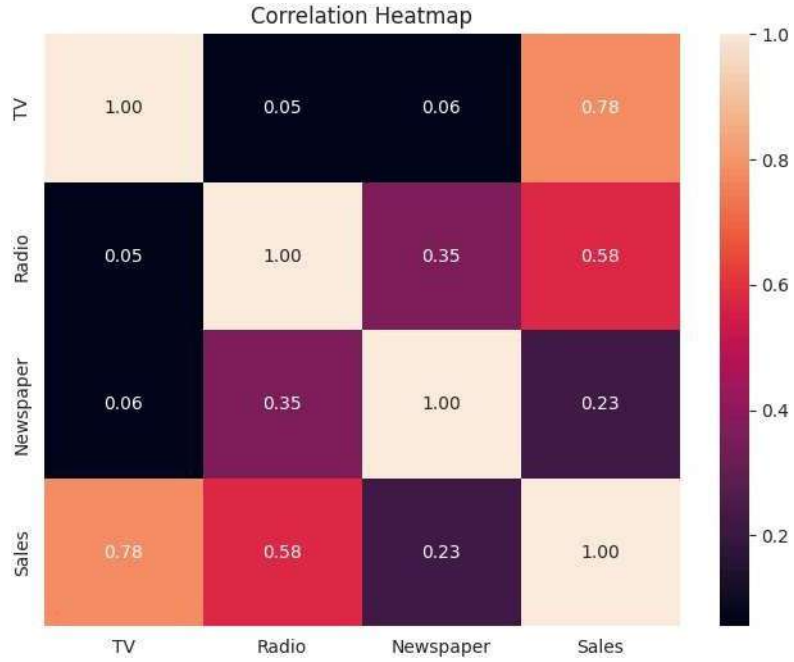
```
plt.figure(figsize=(8,6))

correlation = df[['TV', 'Radio', 'Newspaper', 'Sales']].corr()

sns.heatmap(
    correlation,
    annot=True,
    fmt=".2f"
)

plt.title("Correlation Heatmap")

plt.show()
```




```
sales_correlation = correlation['Sales'].drop('Sales')

print("Correlation with Sales:")
print(sales_correlation)

strongest_feature = sales_correlation.abs().idxmax()

print("\nFeature Most Strongly Correlated with Sales:")
print(strongest_feature)
```

```
Correlation with Sales:
TV           0.782224
Radio        0.576223
```

```
Newspaper    0.228299
Name: Sales, dtype: float64

Feature Most Strongly Correlated with Sales:
TV
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

print("Training Data:", X_train.shape)
print("Testing Data:", X_test.shape)
```

```
Training Data: (160, 3)
Testing Data: (40, 3)
```

```
linear_model = LinearRegression()

linear_model.fit(X_train, y_train)

y_pred_linear = linear_model.predict(X_test)

print("Linear Regression completed successfully.")
```

```
Linear Regression completed successfully.
```

```
polynomial_model = Pipeline([
    ('poly', PolynomialFeatures(degree=2)),
    ('linear', LinearRegression())
])

polynomial_model.fit(X_train, y_train)

y_pred_polynomial = polynomial_model.predict(X_test)

print("Polynomial Regression completed successfully.")
```

```
Polynomial Regression completed successfully.
```

```
svr_model = Pipeline([
    ('scaler', StandardScaler()),
    ('svr', SVR(kernel='rbf'))
])

svr_model.fit(X_train, y_train)

y_pred_svr = svr_model.predict(X_test)

print("SVR completed successfully.")
```

```
SVR completed successfully.
```

```
def evaluate_model(model_name, y_test, y_pred):

    mae = mean_absolute_error(y_test, y_pred)

    mse = mean_squared_error(y_test, y_pred)

    rmse = np.sqrt(mse)

    r2 = r2_score(y_test, y_pred)

    print("\n" + "=" * 50)
    print(model_name)
    print("=" * 50)

    print("MAE :", round(mae, 4))
    print("MSE :", round(mse, 4))
    print("RMSE:", round(rmse, 4))
    print("R²  :", round(r2, 4))

    return mae, mse, rmse, r2
```

```
linear_results = evaluate_model(
    "Linear Regression",
    y_test,
    y_pred_linear
)
```

```
==================================================
Linear Regression
==================================================
MAE : 1.4608
MSE : 3.1741
RMSE: 1.7816
R²  : 0.8994
```

```
polynomial_results = evaluate_model(
    "Polynomial Regression",
    y_test,
    y_pred_polynomial
)
```

```
==================================================
Polynomial Regression
==================================================
MAE : 0.5262
MSE : 0.4129
RMSE: 0.6426
R²  : 0.9869
```

```
svr_results = evaluate_model(
    "Support Vector Regression",
    y_test,
    y_pred_svr
)
```

```
==================================================
Support Vector Regression
==================================================
MAE : 1.1621
MSE : 2.8621
RMSE: 1.6918
R²  : 0.9093
```

```
results = pd.DataFrame({

    'Model': [
        'Linear Regression',
        'Polynomial Regression',
        'SVR'
    ],

    'MAE': [
        linear_results[0],
        polynomial_results[0],
        svr_results[0]
    ],

    'MSE': [
        linear_results[1],
        polynomial_results[1],
        svr_results[1]
    ],

    'RMSE': [
        linear_results[2],
        polynomial_results[2],
        svr_results[2]
    ],

    'R2 Score': [
        linear_results[3],
        polynomial_results[3],
        svr_results[3]
    ]
})

display(results)
```

	Model	MAE	MSE	RMSE	R2 Score
0	Linear Regression	1.460757	3.174097	1.781600	0.899438
1	Polynomial Regression	0.526179	0.412910	0.642581	0.986918
2	SVR	1.162139	2.862109	1.691777	0.909322

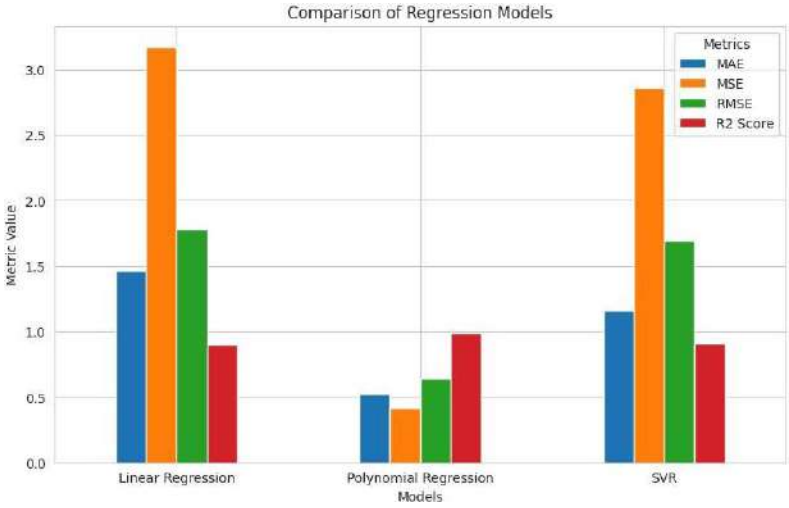
```
results.set_index('Model').plot(
    kind='bar',
    figsize=(10,6)
)

plt.title("Comparison of Regression Models")
plt.xlabel("Models")
plt.ylabel("Metric Value")

plt.xticks(rotation=0)

plt.legend(title="Metrics")

plt.show()
```




```
best_model = results.loc[
    results['RMSE'].idxmin()
```

```
]

print("Best Performing Model:")
print(best_model['Model'])

print("\nLowest RMSE:")
print(round(best_model['RMSE'], 4))

print("\nR² Score:")
print(round(best_model['R2 Score'], 4))
```

```
Best Performing Model:
Polynomial Regression

Lowest RMSE:
0.6426

R² Score:
0.9869
```

```
model_predictions = {
    'Linear Regression': y_pred_linear,
    'Polynomial Regression': y_pred_polynomial,
    'SVR': y_pred_svr
}

best_model_name = best_model['Model']

best_predictions = model_predictions[best_model_name]

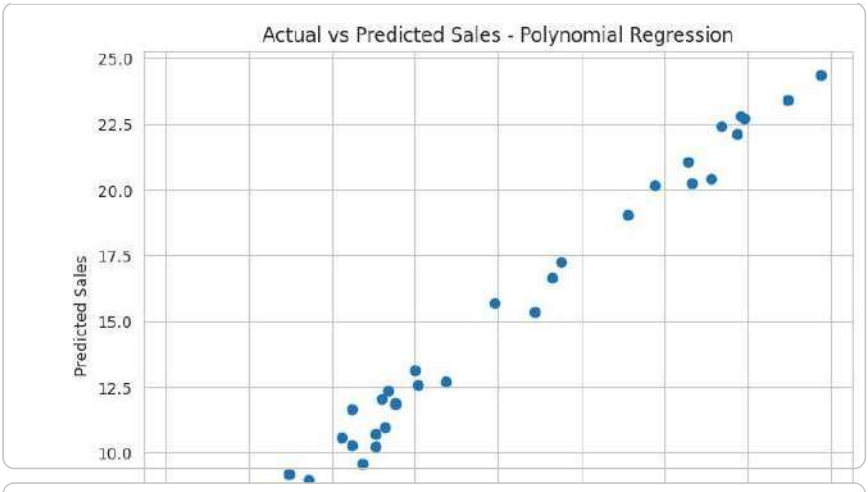
plt.figure(figsize=(8,6))

plt.scatter(
    y_test,
    best_predictions
)

plt.xlabel("Actual Sales")
plt.ylabel("Predicted Sales")

plt.title(
    "Actual vs Predicted Sales - " + best_model_name
)

plt.show()
```




```
print("BUSINESS RECOMMENDATIONS")
print("=" * 50)

print("1. Allocate a larger portion of the advertising budget "
      "to the advertising channel that has the strongest "
      "relationship with Sales.")

print("2. Use the selected regression model to estimate expected "
      "Sales before making major advertising investments.")

print("3. Regularly analyze advertising performance and adjust "
      "the budget across TV, Radio, and Newspaper based on "
      "their contribution to Sales.")
```

```
BUSINESS RECOMMENDATIONS
==================================================
1. Allocate a larger portion of the advertising budget to the advertising chan
2. Use the selected regression model to estimate expected Sales before making
3. Regularly analyze advertising performance and adjust the budget across TV,
```

LEARNING OUTCOMES:

After completing this experiment, I was able to:

Understand the concept of regression analysis. Load and preprocess the Advertising dataset using Python. Analyze the relationship between advertising expenditure and Sales. Implement Linear Regression, Polynomial Regression, and SVR models. Understand the use of RBF kernel in Support Vector Regression. Evaluate regression

LEARNING OUTCOMES

After completing this experiment, I was able to:

- Understand the fundamentals of regression analysis.
- Explore and prepare the Advertising dataset for analysis.
- Identify relationships between advertising expenditure
- and sales using plots and correlation analysis.
- Split the dataset into training and testing sets.
- Implement Linear and Polynomial Regression models.
- Apply Support Vector Regression using an RBF kernel.
- Evaluate and compare models using suitable metrics.
- Identify the best-performing regression model.

Arun S (241801027)

LEARNING OBJECTIVES:

To understand the basic concepts of Information Retrieval.

To preprocess customer review text using NLTK.

To remove punctuation, numbers, special characters, and stopwords from reviews.

To tokenize review text into individual words.

To convert cleaned text into TF-IDF feature vectors.

To calculate Cosine Similarity between a search query and customer reviews.

To retrieve the Top 5 most relevant reviews for a given query.

To evaluate the relevance of retrieved reviews using different sample queries.

BUILT-IN METHODS / FUNCTIONS USED:

pd.read_csv() Load the Amazon Fine Food Reviews dataset

df[['Text']] Select the review text column

isnull() Check missing values

dropna() Remove missing/null reviews

duplicated() Find duplicate reviews

drop_duplicates() Remove duplicate reviews

head() Display first records

len() Find number of reviews

lower() Convert text to lowercase

re.sub() Remove punctuation, numbers and special characters

word_tokenize() Tokenize text into words

stopwords.words() Obtain English stopwords

join() Combine cleaned tokens into a string

TfidfVectorizer() Convert cleaned reviews into TF-IDF vectors

fit_transform() Learn vocabulary and create TF-IDF matrix

transform() Convert the search query into a TF-IDF vector

cosine_similarity() Calculate similarity between query and reviews

argsort() Sort similarity scores

enumerate() Display ranking of retrieved reviews

```
!pip install nltk scikit-learn pandas numpy
```

```
Requirement already satisfied: nltk in /usr/local/lib/python3.12
Requirement already satisfied: scikit-learn in /usr/local/lib/py
Requirement already satisfied: pandas in /usr/local/lib/python3.
Requirement already satisfied: numpy in /usr/local/lib/python3.1
Requirement already satisfied: click in /usr/local/lib/python3.1
Requirement already satisfied: joblib in /usr/local/lib/python3.
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib
Requirement already satisfied: tqdm in /usr/local/lib/python3.12
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/py
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/loca
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/lo
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/py
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/
Requirement already satisfied: six>=1.5 in /usr/local/lib/python
```

```
import pandas as pd
import numpy as np
import re
import string
import nltk

from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
```

```
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('stopwords')
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
True
```

```
import os

print(os.listdir('/content'))
```

```
['.config', 'Reviews.csv.zip', 'sample_data']
```

```
df = pd.read_csv('/content/Reviews.csv.zip')

print("Dataset Loaded Successfully")
print("Shape:", df.shape)

display(df.head())
```

```
Dataset Loaded Successfully
Shape: (568454, 10)
```

	Id	ProductId	UserId	ProfileName	Helpfulness
0	1	B001E4KFG0	A3SGXH7AUHU8GW	delmartian	
1	2	B00813GRG4	A1D87F6ZCVE5NK	dll pa	

```
print(df.columns.tolist())
```

```
['Id', 'ProductId', 'UserId', 'ProfileName', 'HelpfulnessNumerat
```

```
df = df[['Text']]

print("Selected Text Column")
display(df.head())
```

Selected Text Column

	Text
0	I have bought several of the Vitality canned d...
1	Product arrived labeled as Jumbo Salted Peanut...
2	This is a confection that has been around a fe...
3	If you are looking for the secret ingredient i...
4	Great taffy at a great price. There was a wid...

```
print("Missing values before cleaning:")
print(df['Text'].isnull().sum())

df = df.dropna(subset=['Text'])

print("\nMissing values after cleaning:")
print(df['Text'].isnull().sum())
```

```
Missing values before cleaning:
0

Missing values after cleaning:
0
```

```
print("Duplicates before removal:")
print(df.duplicated().sum())

df = df.drop_duplicates()

print("Duplicates after removal:")
print(df.duplicated().sum())
```

```
Duplicates before removal:
174875
Duplicates after removal:
0
```

```
df = df.head(10000).copy()

print("Number of reviews:", len(df))
```

```
Number of reviews: 10000
```

```
stop_words = set(stopwords.words('english'))
```

```
def preprocess_text(text):

    # Convert to lowercase
    text = text.lower()

    # Remove punctuation, numbers and special characters
    text = re.sub(r'[^a-zA-Z\s]', '', text)

    # Tokenize
    tokens = word_tokenize(text)

    # Remove stopwords
    tokens = [
        word for word in tokens
        if word not in stop_words
    ]

    # Join tokens
    cleaned_text = ' '.join(tokens)

    return cleaned_text
```

```
df['Cleaned_Text'] = df['Text'].apply(preprocess_text)

print("Text preprocessing completed.")

display(df.head())
```

```
Text preprocessing completed.
```

	Text	Cleaned_Text
0	I have bought several of the Vitality canned d...	bought several vitality canned dog food produc...
1	Product arrived labeled as Jumbo Salted Peanut...	product arrived labeled jumbo salted peanutsth...
2	This is a confection that has been around a fe...	confection around centuries light pillowy citr...
3	If you are looking for the secret	looking secret ingredient robitussin

```
display(
    df[['Text', 'Cleaned_Text']].head(5)
)
```

	Text	Cleaned_Text
0	I have bought several of the Vitality canned d...	bought several vitality canned dog food produc...
1	Product arrived labeled as Jumbo Salted Peanut...	product arrived labeled jumbo salted peanutsth...
2	This is a confection that has been around a fe...	confection around centuries light pillowy citr...
	If you are looking for the secret	looking secret ingredient robitussin

```
vectorizer = TfidfVectorizer()

tfidf_matrix = vectorizer.fit_transform(
    df['Cleaned_Text']
)

print("TF-IDF Matrix Shape:")
print(tfidf_matrix.shape)

print("\nVocabulary Size:")
print(len(vectorizer.vocabulary_))
```

```
TF-IDF Matrix Shape:
(10000, 24267)

Vocabulary Size:
24267
```

```
def retrieve_reviews(query, top_n=5):

    # Preprocess query
    cleaned_query = preprocess_text(query)

    # Convert query to TF-IDF
    query_vector = vectorizer.transform([cleaned_query])

    # Calculate cosine similarity
    similarity_scores = cosine_similarity(
        query_vector,
        tfidf_matrix
    ).flatten()

    # Get top 5 indices
    top_indices = similarity_scores.argsort()[-top_n:][::-1]

    # Display results
    print("\nSearch Query:", query)
```

```
    print("=" * 80)

    for rank, index in enumerate(top_indices, start=1):

        print("\nRank:", rank)
        print("Similarity Score:",
              round(similarity_scores[index], 4))

        print("Original Review:")
        print(df.iloc[index]['Text'])

        print("\nCleaned Review:")
        print(df.iloc[index]['Cleaned_Text'])

        print("-" * 80)
```

```
retrieve_reviews(
    "great product with fast shipping"
)
```

```
Search Query: great product with fast shipping
================================================================

Rank: 1
Similarity Score: 0.499
Original Review:
Enjoyed the product and they also provided very fast shipping.

Cleaned Review:
enjoyed product also provided fast shipping im need order
----------------------------------------------------------------

Rank: 2
Similarity Score: 0.4417
Original Review:
The tea is good and fresh. We enjoy it. The shipping is fast and

Cleaned Review:
tea good fresh enjoy shipping fast cost reasonablebr recommend p
----------------------------------------------------------------

Rank: 3
Similarity Score: 0.4011
Original Review:
The energy drink is a great product. The shipping price is crazy

Cleaned Review:
energy drink great product shipping price crazy tho paid shippin
----------------------------------------------------------------
```

```
Rank: 4
Similarity Score: 0.3929
Original Review:
My daughter lives in Hawaii and sent me some great coffee from t

Cleaned Review:
daughter lives hawaii sent great coffee keurig sure use new coff
------------------------------------------------------------

Rank: 5
Similarity Score: 0.3882
Original Review:
Fast shipping, items were packaged nicely and are as described.

Cleaned Review:
fast shipping items packaged nicely described also tasty highly
------------------------------------------------------------
```

```
retrieve_reviews(
    "disappointed"
)
```

```
Search Query: disappointed
============================================================

Rank: 1
Similarity Score: 0.4871
Original Review:
I am a bit disappointed.  The flavor was not what I wanted or ex

Cleaned Review:
bit disappointed flavor wanted expected
------------------------------------------------------------

Rank: 2
Similarity Score: 0.3599
Original Review:
The product is very good. Way too expensive and at almost 5.00 a

Cleaned Review:
product good way expensive almost box get pancakes biscuits disa
------------------------------------------------------------

Rank: 3
Similarity Score: 0.3216
Original Review:
Disappointed.  The big boxes had a very different flavor than th

Cleaned Review:
disappointed big boxes different flavor smaller boxes sure good
------------------------------------------------------------
```

```
Rank: 4
Similarity Score: 0.3129
Original Review:
Just plain nasty!!! This item tasted like cardboard, very watere

Cleaned Review:
plain nasty item tasted like cardboard watered flavor disappoint
----------------------------------------------------------------

Rank: 5
Similarity Score: 0.3121
Original Review:
I was a bit disappointed in these.  They crumble very easily and

Cleaned Review:
bit disappointed crumble easily unusual texture son like
----------------------------------------------------------------
```

```
retrieve_reviews(
    "excellent taste"
)
```

```
Search Query: excellent taste
================================================================

Rank: 1
Similarity Score: 0.4532
Original Review:
The Popchips were excellent - had excellent flavor and low in ca

Cleaned Review:
popchips excellent excellent flavor low calories eat dont need a
----------------------------------------------------------------

Rank: 2
Similarity Score: 0.4073
Original Review:
Excellent product. Exactly what they said it would be. Superior

Cleaned Review:
excellent product exactly said would superior chamomile ordered
----------------------------------------------------------------

Rank: 3
Similarity Score: 0.3863
Original Review:
We recieved the product on time and in excellent shape. It taste

Cleaned Review:
recieved product time excellent shape tastes good meets expectat
```

```
------------------------------------------------------------

Rank: 4
Similarity Score: 0.3833
Original Review:
Title says it all, a Excellent quality product and Excellent ser

Cleaned Review:
title says excellent quality product excellent service fair pric
------------------------------------------------------------

Rank: 5
Similarity Score: 0.357
Original Review:
This is an excellent tea, very flavorful - the cinnamon is fresh

Cleaned Review:
excellent tea flavorful cinnamon fresh powerful overall slightly
------------------------------------------------------------
```

```
retrieve_reviews(
    "poor packaging"
)
```

```
Search Query: poor packaging
============================================================

Rank: 1
Similarity Score: 0.3245
Original Review:
I was delighted to find this product online, because I cannot al

Cleaned Review:
delighted find product online always find stores however package
------------------------------------------------------------

Rank: 2
Similarity Score: 0.309
Original Review:
This review is for the packaging and not the actual mix, since t

Cleaned Review:
review packaging actual mix since mix subjective thing taste mat
------------------------------------------------------------

Rank: 3
Similarity Score: 0.3062
Original Review:
The soup is rich and tasty, but DON'T buy it from Amazon.  The p

Cleaned Review:
```

```
soup rich tasty dont buy amazon packaging poor cans arive dented
----------------------------------------------------------------------

Rank: 4
Similarity Score: 0.2989
Original Review:
Didnt really enjoy these after wanting to try something other th

Cleaned Review:
didnt really enjoy wanting try something aunt jemimas dominicks
----------------------------------------------------------------------

Rank: 5
Similarity Score: 0.2591
Original Review:
was really dissappointed in product... too salty and poor flavor

Cleaned Review:
really dissappointed product salty poor flavorconvenience makes
----------------------------------------------------------------------
```

```
queries = [
    "great product with fast shipping",
    "disappointed",
    "excellent taste",
    "poor packaging"
]

for query in queries:

    print("\n")
    print("#" * 100)
    print("QUERY:", query)
    print("#" * 100)

    retrieve_reviews(query)
```

```
Rank: 1
Similarity Score: 0.3245
Original Review:
I was delighted to find this product online, because I cannot a

Cleaned Review:
delighted find product online always find stores however packag
------------------------------------------------------------

Rank: 2
Similarity Score: 0.309
Original Review:
This review is for the packaging and not the actual mix, since

Cleaned Review:
review packaging actual mix since mix subjective thing taste ma
------------------------------------------------------------

Rank: 3
Similarity Score: 0.3062
Original Review:
The soup is rich and tasty, but DON'T buy it from Amazon.  The

Cleaned Review:
soup rich tasty dont buy amazon packaging poor cans arive dente
------------------------------------------------------------

Rank: 4
Similarity Score: 0.2989
Original Review:
Didnt really enjoy these after wanting to try something other t

Cleaned Review:
didnt really enjoy wanting try something aunt jemimas dominicks
------------------------------------------------------------

Rank: 5
Similarity Score: 0.2591
Original Review:
was really dissappointed in product... too salty and poor flavc

Cleaned Review:
really dissappointed product salty poor flavorconvenience makes
```

LEARNING OUTCOMES:

After completing this experiment, I was able to:

Understand the basic Information Retrieval process. Clean and preprocess customer review text using NLTK. Apply tokenization and stopword removal to text data. Convert textual data into numerical

representations using TF-IDF. Calculate Cosine Similarity between a query and review documents. Build a simple information retrieval system. Retrieve and rank the Top 5 most relevant customer reviews. Test the retrieval system using different search queries. Analyze the relevance of retrieved reviews for different queries.

Arun S (241801027)

LEARNING OUTCOMES

- Understand the basic concepts of Information Retrieval.
- Preprocess customer review text using NLTK.
- Clean reviews by removing punctuation, numbers, special characters, and stopwords.
- Tokenize review text into individual words.
- Convert cleaned review text into TF-IDF feature vectors.
- Calculate cosine similarity between a search query and customer reviews.
- Retrieve the five most relevant reviews for a given query.
- Evaluate the relevance of retrieved reviews using different sample queries.

Arun S
241801027

Learning Objectives:

Load and clean Amazon Fine Food Reviews dataset.

Apply spaCy NLP pipeline to customer reviews.

Extract Tokens, Lemmas and POS tags.

Identify Nouns, Verbs, Adjectives and Adverbs.

Calculate POS category frequencies. Perform NER and identify entities such as PERSON, ORG, DATE, MONEY, QUANTITY and PRODUCT, wherever recognized by the selected model.

Develop a reusable information extraction function. Organize extracted information into structured format.

Built-in Methods / Functions:

Function / Method Purpose

pd.read_csv() Load dataset

dropna() Remove missing values

drop_duplicates() Remove duplicate reviews

head() Select/display first records

spacy.load() Load spaCy model

nlp() Process text using spaCy pipeline

token.text Extract tokens

token.lemma_ Extract lemmas

token.pos_ Extract POS tags

ent.text Extract entity text

ent.label_ Extract entity label

Counter() Count POS/entity frequencies

pd.DataFrame() Create structured tables

value_counts() Count categorical values

enumerate() Number reviews

display() Display DataFrames

```
!pip install spacy pandas
```

```
Requirement already satisfied: spacy in /usr/local/lib/python3.13/dist-packages (3.8.16)
Requirement already satisfied: pandas in /usr/local/lib/python3.13/dist-packages (2.2.3)
Requirement already satisfied: spacy-legacy<3.1.0,>=3.0.11 in /usr/local/lib/python3.13/dist-packages (fr
Requirement already satisfied: spacy-loggers<2.0.0,>=1.0.0 in /usr/local/lib/python3.13/dist-packages (fr
Requirement already satisfied: murmurhash<1.1.0,>=0.28.0 in /usr/local/lib/python3.13/dist-packages (fror
Requirement already satisfied: cymem<2.1.0,>=2.0.2 in /usr/local/lib/python3.13/dist-packages (from spacy
Requirement already satisfied: preshed<3.1.0,>=3.0.2 in /usr/local/lib/python3.13/dist-packages (from spa
Requirement already satisfied: thinc<8.4.0,>=8.3.12 in /usr/local/lib/python3.13/dist-packages (from spac
Requirement already satisfied: wasabi<1.2.0,>=0.9.1 in /usr/local/lib/python3.13/dist-packages (from spac
Requirement already satisfied: srsly<3.0.0,>=2.5.3 in /usr/local/lib/python3.13/dist-packages (from spacy
Requirement already satisfied: catalogue<2.1.0,>=2.0.6 in /usr/local/lib/python3.13/dist-packages (from s
Requirement already satisfied: weasel<2.0.0,>=1.0.0 in /usr/local/lib/python3.13/dist-packages (from spac
Requirement already satisfied: confection<2.0.0,>=1.3.2 in /usr/local/lib/python3.13/dist-packages (from
Requirement already satisfied: typer<1.0.0,>=0.3.0 in /usr/local/lib/python3.13/dist-packages (from spacy
Requirement already satisfied: click<9.0.0,>=8.2.1 in /usr/local/lib/python3.13/dist-packages (from spacy
Requirement already satisfied: tqdm<5.0.0,>=4.38.0 in /usr/local/lib/python3.13/dist-packages (from spacy
Requirement already satisfied: numpy>=1.19.0 in /usr/local/lib/python3.13/dist-packages (from spacy) (2.1
Requirement already satisfied: requests<3.0.0,>=2.13.0 in /usr/local/lib/python3.13/dist-packages (from s
```

```
Requirement already satisfied: pydantic<3.0.0,>=2.0.0 in /usr/local/lib/python3.13/dist-packages (from sp
Requirement already satisfied: jinja2 in /usr/local/lib/python3.13/dist-packages (from spacy) (3.1.6)
Requirement already satisfied: setuptools in /usr/local/lib/python3.13/dist-packages (from spacy) (80.10
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.13/dist-packages (from spacy) (2
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.13/dist-packages (from pa
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.13/dist-packages (from pandas) (202
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.13/dist-packages (from pandas) (2
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.13/dist-packages (from py
Requirement already satisfied: pydantic-core==2.46.5 in /usr/local/lib/python3.13/dist-packages (from pyd
Requirement already satisfied: typing-extensions>=4.14.1 in /usr/local/lib/python3.13/dist-packages (fron
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.13/dist-packages (from
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-packages (from python-dateutil>
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.13/dist-packages (from
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.13/dist-packages (from requests<3.0
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.13/dist-packages (from reques
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.13/dist-packages (from reques
Requirement already satisfied: blis<1.4.0,>=1.3.0 in /usr/local/lib/python3.13/dist-packages (from thinc<
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.13/dist-packages (from typer<
Requirement already satisfied: rich>=13.8.0 in /usr/local/lib/python3.13/dist-packages (from typer<1.0.0,
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.13/dist-packages (from type
Requirement already satisfied: cloudpathlib>=0.7.0 in /usr/local/lib/python3.13/dist-packages (from wease
Requirement already satisfied: smart-open>=5.2.1 in /usr/local/lib/python3.13/dist-packages (from weasel<
Requirement already satisfied: httpx>=0.24.0 in /usr/local/lib/python3.13/dist-packages (from weasel<2.0
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.13/dist-packages (from jinja2->s
Requirement already satisfied: anyio in /usr/local/lib/python3.13/dist-packages (from httpx>=0.24.0->weas
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.13/dist-packages (from httpx>=0.24
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.13/dist-packages (from httpcore==1.*->
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.13/dist-packages (from ric
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.13/dist-packages (from r
Requirement already satisfied: wrapt in /usr/local/lib/python3.13/dist-packages (from smart-open>=5.2.1->
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.13/dist-packages (from markdown-it-py
```

```
!python -m spacy download en_core_web_sm
```

```
Collecting en-core-web-sm==3.8.0
  Downloading https://github.com/explosion/spacy-models/releases/download/en_core_web_sm-3.8.0/en_core_we
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.8/12.8 MB 104.5 MB/s eta 0:00:00
✓ Download and installation successful
You can now load the package via spacy.load('en_core_web_sm')
⚠ Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Python in
order to load all the package's dependencies. You can do this by selecting the
'Restart kernel' or 'Restart runtime' option.
```

```
import pandas as pd
import spacy
from collections import Counter
```

```
nlp = spacy.load("en_core_web_sm")

print("spaCy model loaded successfully!")
```

```
spaCy model loaded successfully!
```

```
import pandas as pd

df = pd.read_csv("/content/Reviews.csv.zip")

print("Dataset Loaded Successfully")
print("Shape:", df.shape)

display(df.head())
```

```
Dataset Loaded Successfully
Shape: (568454, 10)
```

	Id	ProductId	UserId	ProfileName	HelpfulnessNumerator	HelpfulnessDenominator	Score
0	1	B001E4KFG0	A3SGXH7AUHU8GW	delmartian	1	1	5
1	2	B00813GRG4	A1D87F6ZCVE5NK	dll pa	0	0	1
2	3	B000LQOCH0	ABXLMWJIXXAIN	Natalia Corres "Natalia Corres"	1	1	4
3	4	B000UA0QIQ	A395BORC6FGVXV	Karl	3	3	2
4	5	B006K2ZZ7K	A1UQRSCLF8GW1T	Michael D. Bigham "M. Wassir"	0	0	5

```
print("Column Names:")
print(df.columns.tolist())
```

```
Column Names:
['Id', 'ProductId', 'UserId', 'ProfileName', 'HelpfulnessNumerator', 'HelpfulnessDenominator', 'Score',
```

```
df = df[['Text']].copy()

print("Text column selected successfully!")

display(df.head())
```

```
Text column selected successfully!
```

	Text
0	I have bought several of the Vitality canned d...
1	Product arrived labeled as Jumbo Salted Peanut...
2	This is a confection that has been around a fe...
3	If you are looking for the secret ingredient i...
4	Great taffy at a great price. There was a wid...

```
print("Missing values before:")
print(df['Text'].isnull().sum())

df = df.dropna(subset=['Text'])

print("\nMissing values after:")
print(df['Text'].isnull().sum())
```

```
Missing values before:
0

Missing values after:
0
```

```
print("Duplicate reviews before:")
print(df.duplicated().sum())

df = df.drop_duplicates()

print("Duplicate reviews after:")
print(df.duplicated().sum())
```

```
Duplicate reviews before:
174875
Duplicate reviews after:
0
```

```
df = df.head(10000).copy()

print("Number of reviews:", len(df))
```

```
Number of reviews: 10000
```

```
for i, review in enumerate(df['Text'].head(5), start=1):

    doc = nlp(review)

    print("\n" + "=" * 80)
    print("REVIEW", i)
    print("=" * 80)

    print("\nOriginal Review:")
    print(review[:500])

    print("\nTokens:")
    print([token.text for token in doc])

    print("\nLemmas:")
    print([token.lemma_ for token in doc])

    print("\nPOS Tags:")
    print([(token.text, token.pos_) for token in doc])
```

```
Tokens:
['Product', 'arrived', 'labeled', 'as', 'Jumbo', 'Salted', 'Peanuts', '...', 'the', 'peanuts', 'were',

Lemmas:
['product', 'arrived', 'label', 'as', 'Jumbo', 'Salted', 'Peanuts', '...', 'the', 'peanut', 'be', 'actu
```

```
Original Review:
If you are looking for the secret ingredient in Robitussin I believe I have found it.  I got this in ad

Tokens:
['If', 'you', 'are', 'looking', 'for', 'the', 'secret', 'ingredient', 'in', 'Robitussin', 'I', 'believe

Lemmas:
['if', 'you', 'be', 'look', 'for', 'the', 'secret', 'ingredient', 'in', 'Robitussin', 'I', 'believe', '

POS Tags:
[('If', 'SCONJ'), ('you', 'PRON'), ('are', 'AUX'), ('looking', 'VERB'), ('for', 'ADP'), ('the', 'DET'),

================================================================================
REVIEW 5
================================================================================

Original Review:
Great taffy at a great price.  There was a wide assortment of yummy taffy.  Delivery was very quick.  I

Tokens:
['Great', 'taffy', 'at', 'a', 'great', 'price', '.', ' ', 'There', 'was', 'a', 'wide', 'assortment', 'o

Lemmas:
['great', 'taffy', 'at', 'a', 'great', 'price', '.', ' ', 'there', 'be', 'a', 'wide', 'assortment', 'of

POS Tags:
[('Great', 'ADJ'), ('taffy', 'NOUN'), ('at', 'ADP'), ('a', 'DET'), ('great', 'ADJ'), ('price', 'NOUN'),
```

```
for i, review in enumerate(df['Text'].head(5), start=1):

    doc = nlp(review)

    nouns = [
        token.text for token in doc
        if token.pos_ == "NOUN"
    ]

    verbs = [
        token.text for token in doc
        if token.pos_ == "VERB"
    ]

    adjectives = [
        token.text for token in doc
        if token.pos_ == "ADJ"
    ]

    adverbs = [
        token.text for token in doc
        if token.pos_ == "ADV"
    ]

    print("\n" + "=" * 80)
    print("REVIEW", i)
    print("=" * 80)

    print("Nouns:", nouns[:20])
    print("Verbs:", verbs[:20])
    print("Adjectives:", adjectives[:20])
    print("Adverbs:", adverbs[:20])
```

```
================================================================================
REVIEW 1
================================================================================
Nouns: ['dog', 'food', 'products', 'quality', 'product', 'stew', 'meat', 'product']
Verbs: ['bought', 'canned', 'found', 'looks', 'processed', 'smells', 'appreciates']
Adjectives: ['several', 'good', 'finicky', 'most']
Adverbs: ['more', 'better', 'better']

================================================================================
REVIEW 2
================================================================================
Nouns: ['Product', 'peanuts', 'error', 'vendor', 'product']
Verbs: ['labeled', 'unsalted', 'intended', 'represent']
Adjectives: ['small', 'sized', 'sure']
```

```
Adverbs: ['actually']

================================================================================
REVIEW 3
================================================================================
Nouns: ['confection', 'centuries', 'citrus', 'nuts', 'case', 'Filberts', 'squares', 'sugar', 'chewy', 'tr
Verbs: ['gelatin', 'cut', 'coated', 'recommend', 'seduces', 'selling']
Adjectives: ['few', 'light', 'pillowy', 'tiny', 'powdered', 'tiny', 'mouthful', 'flavorful', 'yummy', 'fa
Adverbs: ['then', 'liberally', 'too', 'very', 'highly']

================================================================================
REVIEW 4
================================================================================
Nouns: ['ingredient', 'addition', 'cherry', 'soda', 'flavor']
Verbs: ['looking', 'believe', 'found', 'got', 'ordered', 'made']
Adjectives: ['secret', 'good', 'medicinal']
Adverbs: ['very']

================================================================================
REVIEW 5
================================================================================
Nouns: ['taffy', 'price', 'assortment', 'taffy', 'lover', 'deal']
Verbs: ['was']
Adjectives: ['Great', 'great', 'wide', 'yummy', 'quick']
Adverbs: ['very']
```

```
pos_counter = Counter()

for review in df['Text']:

    doc = nlp(review)

    for token in doc:
        if token.pos_ in ["NOUN", "VERB", "ADJ", "ADV"]:
            pos_counter[token.pos_] += 1

print("POS Category Frequencies:")

for category, count in pos_counter.items():
    print(category, ":", count)
```

```
POS Category Frequencies:
VERB : 95797
ADJ : 71482
NOUN : 144487
ADV : 50615
```

```
pos_df = pd.DataFrame(
    pos_counter.items(),
    columns=['POS Category', 'Frequency']
)

pos_df = pos_df.sort_values(
    by='Frequency',
    ascending=False
)

display(pos_df)
```

	POS Category	Frequency
2	NOUN	144487
0	VERB	95797
1	ADJ	71482
3	ADV	50615

Next steps: Generate code with pos_df | New interactive sheet

```
entity_data = []

for review in df['Text']:
```

```
    doc = nlp(review)

    for ent in doc.ents:

        if ent.label_ in [
            "PERSON",
            "ORG",
            "DATE",
            "MONEY",
            "QUANTITY",
            "PRODUCT"
        ]:

            entity_data.append({
                "Entity": ent.text,
                "Label": ent.label_,
                "Review": review[:200]
            })
```

```
entity_df = pd.DataFrame(entity_data)

print("Extracted Named Entities:")
display(entity_df.head(20))
```

Extracted Named Entities:

	Entity	Label	Review
0	Vitality	ORG	I have bought several of the Vitality canned d...
1	My Labrador	PERSON	I have bought several of the Vitality canned d...
2	Jumbo Salted	ORG	Product arrived labeled as Jumbo Salted Peanut...
3	around a few centuries	DATE	This is a confection that has been around a fe...
4	pillowy citrus gelatin	ORG	This is a confection that has been around a fe...
5	C.S. Lewis'	ORG	This is a confection that has been around a fe...
6	Edmund	PRODUCT	This is a confection that has been around a fe...
7	Sisters	PERSON	This is a confection that has been around a fe...
8	Witch	ORG	This is a confection that has been around a fe...
9	Delivery	PERSON	Great taffy at a great price. There was a wid...
10	five pound	QUANTITY	I got a wild hair for taffy and ordered this f...
11	only two weeks	DATE	I got a wild hair for taffy and ordered this f...
12	Fralinger's	ORG	This saltwater taffy had great flavors and was...
13	Wheatgrass	PERSON	Right now I'm mostly just sprouting this so my...
14	Tequila Picante Gourmet de Inclan	ORG	I don't know if it's the cactus or the tequila...
15	Felidae Platinum	ORG	My cats have been happily eating Felidae Plati...
16	more than two years	DATE	My cats have been happily eating Felidae Plati...
17	Twizzlers	ORG	good flavor! these came securely packed... the...
18	The Strawberry Twizzlers	ORG	The Strawberry Twizzlers are my guilty pleasur...
19	Six pounds	QUANTITY	The Strawberry Twizzlers are my guilty pleasur...

```
entity_counts = entity_df['Label'].value_counts()

print("Entity Type Frequencies:")
display(entity_counts)
```

```
Entity Type Frequencies:
```

Label	count
ORG	6704
PERSON	4410
DATE	3056
MONEY	896
QUANTITY	671
PRODUCT	275

dtype: int64

```python
def extract_information(review):

    doc = nlp(review)

    tokens = [
        token.text
        for token in doc
    ]

    lemmas = [
        token.lemma_
        for token in doc
    ]

    pos_tags = [
        token.pos_
        for token in doc
    ]

    named_entities = [
        ent.text
        for ent in doc.ents
    ]

    entity_labels = [
        ent.label_
        for ent in doc.ents
    ]

    return {
        "Tokens": tokens,
        "Lemmas": lemmas,
        "POS Tags": pos_tags,
        "Named Entities": named_entities,
        "Entity Labels": entity_labels
    }
```

```python
review1 = """
I bought Nestle coffee from Amazon on January 10, 2025.
The package cost $25 and contained 2 kilograms of coffee.
"""

result1 = extract_information(review1)

print("REVIEW 1")
print("=" * 60)

for key, value in result1.items():
    print("\n", key, ":")
    print(value)
```

```
REVIEW 1
============================================================

 Tokens :
```

```
['\n', 'I', 'bought', 'Nestle', 'coffee', 'from', 'Amazon', 'on', 'January', '10', ',', '2025', '.', '\n

 Lemmas :
['\n', 'I', 'buy', 'Nestle', 'coffee', 'from', 'Amazon', 'on', 'January', '10', ',', '2025', '.', '\n',

 POS Tags :
['SPACE', 'PRON', 'VERB', 'PROPN', 'NOUN', 'ADP', 'PROPN', 'ADP', 'PROPN', 'NUM', 'PUNCT', 'NUM', 'PUNCT

 Named Entities :
['Nestle', 'Amazon', 'January 10, 2025', '25', '2 kilograms']

 Entity Labels :
['ORG', 'ORG', 'DATE', 'MONEY', 'QUANTITY']
```

```
review2 = """
Apple juice from the Organic Valley brand arrived in New York
last Monday. The bottle contained 1 liter and cost $12.
"""

result2 = extract_information(review2)

print("REVIEW 2")
print("=" * 60)

for key, value in result2.items():
    print("\n", key, ":")
    print(value)
```

```
REVIEW 2
============================================================

 Tokens :
['\n', 'Apple', 'juice', 'from', 'the', 'Organic', 'Valley', 'brand', 'arrived', 'in', 'New', 'York', '\r

 Lemmas :
['\n', 'Apple', 'juice', 'from', 'the', 'Organic', 'Valley', 'brand', 'arrive', 'in', 'New', 'York', '\n

 POS Tags :
['SPACE', 'PROPN', 'NOUN', 'ADP', 'DET', 'PROPN', 'PROPN', 'NOUN', 'VERB', 'ADP', 'PROPN', 'PROPN', 'SPAC

 Named Entities :
['Apple', 'Organic Valley', 'New York', 'last Monday', '1 liter', '12']

 Entity Labels :
['ORG', 'LOC', 'GPE', 'DATE', 'QUANTITY', 'MONEY']
```

```
review3 = """
I ordered a Samsung product from Amazon in March 2025.
The package weighed 5 kilograms and the price was $100.
"""

result3 = extract_information(review3)

print("REVIEW 3")
print("=" * 60)

for key, value in result3.items():
    print("\n", key, ":")
    print(value)
```

```
REVIEW 3
============================================================

 Tokens :
['\n', 'I', 'ordered', 'a', 'Samsung', 'product', 'from', 'Amazon', 'in', 'March', '2025', '.', '\n', 'Th

 Lemmas :
['\n', 'I', 'order', 'a', 'Samsung', 'product', 'from', 'Amazon', 'in', 'March', '2025', '.', '\n', 'the

 POS Tags :
['SPACE', 'PRON', 'VERB', 'DET', 'PROPN', 'NOUN', 'ADP', 'PROPN', 'ADP', 'PROPN', 'NUM', 'PUNCT', 'SPACE

 Named Entities :
['Samsung', 'Amazon', 'March 2025', '5 kilograms', '100']
```

```
 Entity Labels :
['ORG', 'ORG', 'DATE', 'QUANTITY', 'MONEY']
```

```
structured_results = pd.DataFrame([
    {
        "Review": review1,
        "Tokens": result1["Tokens"],
        "Lemmas": result1["Lemmas"],
        "POS Tags": result1["POS Tags"],
        "Named Entities": result1["Named Entities"],
        "Entity Labels": result1["Entity Labels"]
    },
    {
        "Review": review2,
        "Tokens": result2["Tokens"],
        "Lemmas": result2["Lemmas"],
        "POS Tags": result2["POS Tags"],
        "Named Entities": result2["Named Entities"],
        "Entity Labels": result2["Entity Labels"]
    },
    {
        "Review": review3,
        "Tokens": result3["Tokens"],
        "Lemmas": result3["Lemmas"],
        "POS Tags": result3["POS Tags"],
        "Named Entities": result3["Named Entities"],
        "Entity Labels": result3["Entity Labels"]
    }
])

display(structured_results)
```

	Review	Tokens	Lemmas	POS Tags	Named Entities	Entity Labels
0	\nI bought Nestle coffee from Amazon on Januar...	[\n, I, bought, Nestle, coffee, from, Amazon, ...	[\n, I, buy, Nestle, coffee, from, Amazon, on,...	[SPACE, PRON, VERB, PROPN, NOUN, ADP, PROPN, A...	[Nestle, Amazon, January 10, 2025, 25, 2 kilog...	[ORG, ORG, DATE, MONEY, QUANTITY]
1	\nApple juice from the Organic Valley brand ar...	[\n, Apple, juice, from, the, Organic, Valley,...	[\n, Apple, juice, from, the, Organic, Valley,...	[SPACE, PROPN, NOUN, ADP, DET, PROPN, PROPN, N...	[Apple, Organic Valley, New York, last Monday,...	[ORG, LOC, GPE, DATE, QUANTITY, MONEY]
	\nI ordered a	[\n, I, ordered, a,	[\n, I, order, a,	[SPACE, PRON,	[Samsung,	[ORG, ORG,

Next steps: Generate code with structured_results | New interactive sheet

Learning Outcomes:

Understand information extraction from unstructured text.

Apply spaCy for NLP processing.

Extract tokens, lemmas and POS tags.

Analyze major POS categories and their frequencies.

Build a reusable information extraction function.

Use extracted information for customer feedback analysis.

Arun S

241801027

Learning Objectives Understand and perform EDA on the Netflix dataset.

Analyze content, ratings, countries, genres, and trends.

Use statistical techniques and visualizations to identify patterns.

Functions Used

pd.read_csv() Load dataset

head() View records

shape Rows & columns

info() Dataset details

describe() Statistics

isnull() Missing values

drop_duplicates() Remove duplicates

value_counts() Count categories

groupby() Group data

plt.plot() / plt.hist() Visualizations

sns.countplot() / sns.heatmap() Visualizations

WordCloud() Word cloud

```
!pip install wordcloud -q

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import re

from collections import Counter
from wordcloud import WordCloud

sns.set_style("whitegrid")
```

```
df = pd.read_csv('/content/netflix_titles.csv.zip')

print("Dataset Loaded Successfully")
print("Shape:", df.shape)

display(df.head())
```

```
Dataset Loaded Successfully
Shape: (8807, 12)
```

	show_id	type	title	director	cast	country	date_added	release_year	rating	duration	listed_in	description
0	s1	Movie	Dick Johnson Is Dead	Kirsten Johnson	NaN	United States	September 25, 2021	2020	PG-13	90 min	Documentaries	As her father nears the end of his life, filmm...
1	s2	TV Show	Blood & Water	NaN	Ama Qamata, Khosi Ngema, Gail Mabalane, Thaban...	South Africa	September 24, 2021	2021	TV-MA	2 Seasons	International TV Shows, TV Dramas, TV Mysteries	After crossing paths at a party, a Cape Town t...
2	s3	TV Show	Ganglands	Julien Leclercq	Sami Bouajila, Tracy Gotoas,	NaN	September 24, 2021	2021	TV-MA	1 Season	Crime TV Shows, International	To protect his family from a

```
print("Shape:", df.shape)

print("\nColumns:")
print(df.columns.tolist())

print("\nData Types:")
print(df.dtypes)

print("\nMissing Values:")
print(df.isnull().sum())

print("\nDuplicate Records:")
print(df.duplicated().sum())

print("\nStatistical Summary:")
display(df.describe(include='all').T)
```

```
Shape: (8807, 12)

Columns:
['show_id', 'type', 'title', 'director', 'cast', 'country', 'date_added', 'release_year', 'rating', 'duration', 'listed_in', '
```

```
Data Types:
show_id         object
type            object
title           object
director        object
cast            object
country         object
date_added      object
release_year     int64
rating          object
duration        object
listed_in       object
description     object
dtype: object
```

```
Missing Values:
show_id            0
type               0
title              0
director        2634
cast             825
country          831
date_added        10
release_year       0
rating             4
duration           3
listed_in          0
description        0
dtype: int64
```

```
Duplicate Records:
0
```

```
Statistical Summary:
```

	count	unique	top	freq	mean	std	min	25%	50%	75%	max
show_id	8807	8807	s8807	1	NaN	NaN	NaN	NaN	NaN	NaN	NaN
type	8807	2	Movie	6131	NaN	NaN	NaN	NaN	NaN	NaN	NaN
title	8807	8807	Zubaan	1	NaN	NaN	NaN	NaN	NaN	NaN	NaN
director	6173	4528	Rajiv Chilaka	19	NaN	NaN	NaN	NaN	NaN	NaN	NaN
cast	7982	7692	David Attenborough	19	NaN	NaN	NaN	NaN	NaN	NaN	NaN
country	7976	748	United States	2818	NaN	NaN	NaN	NaN	NaN	NaN	NaN
date_added	8797	1767	January 1, 2020	109	NaN	NaN	NaN	NaN	NaN	NaN	NaN
release_year	8807.0	NaN	NaN	NaN	2014.180198	8.819312	1925.0	2013.0	2017.0	2019.0	2021.0
rating	8803	17	TV-MA	3207	NaN	NaN	NaN	NaN	NaN	NaN	NaN
duration	8804	220	1 Season	1793	NaN	NaN	NaN	NaN	NaN	NaN	NaN
listed_in	8807	514	Dramas, International Movies	362	NaN	NaN	NaN	NaN	NaN	NaN	NaN
description	8807	8775	Paranormal activity at a lush,	4	NaN	NaN	NaN	NaN	NaN	NaN	NaN

```
print(df['type'].value_counts())

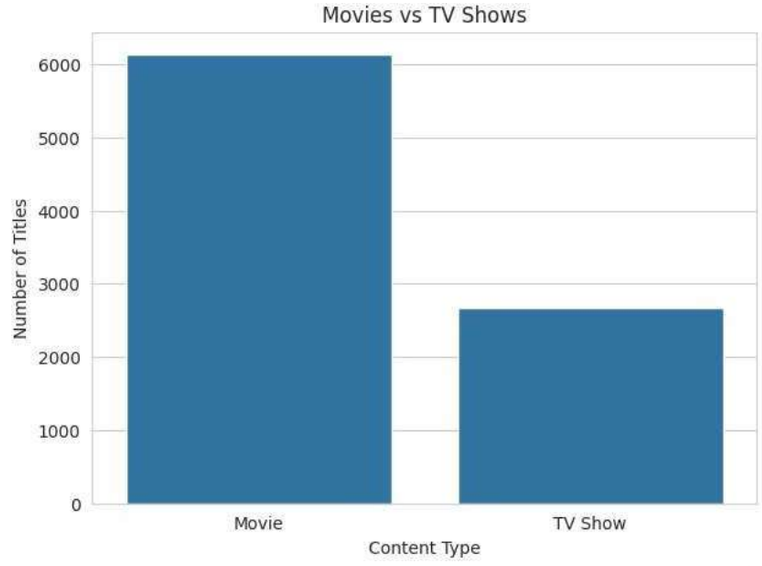
plt.figure(figsize=(7,5))

sns.countplot(data=df, x='type')

plt.title("Movies vs TV Shows")
plt.xlabel("Content Type")
plt.ylabel("Number of Titles")

plt.show()
```

```
type
Movie      6131
TV Show    2676
Name: count, dtype: int64
```



```
rating_counts = df['rating'].value_counts()

display(rating_counts.to_frame("Number of Titles"))

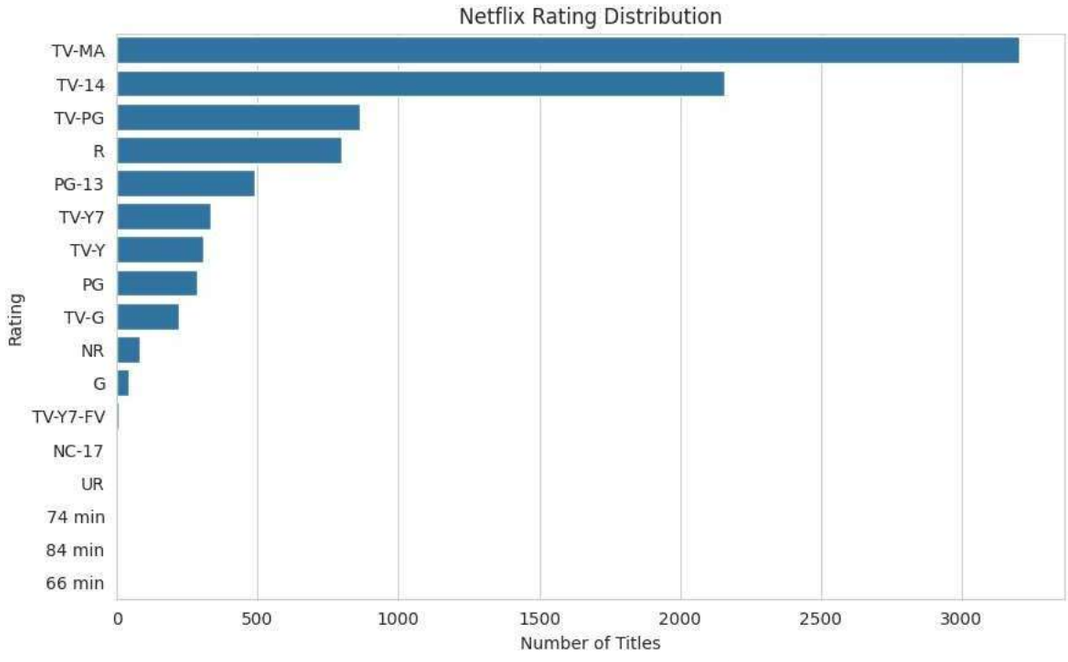
plt.figure(figsize=(10,6))

sns.countplot(
    data=df,
    y='rating',
    order=rating_counts.index
)

plt.title("Netflix Rating Distribution")
plt.xlabel("Number of Titles")
plt.ylabel("Rating")

plt.show()
```

rating	Number of Titles
TV-MA	3207
TV-14	2160
TV-PG	863
R	799
PG-13	490
TV-Y7	334
TV-Y	307
PG	287
TV-G	220
NR	80
G	41
TV-Y7-FV	6
NC-17	3
UR	3
74 min	1
84 min	1
66 min	1




```
country_series = (
    df['country']
    .str.split(',')
    .explode()
    .str.strip()
)

country_counts = country_series.value_counts()

display(
    country_counts.head(15).to_frame("Number of Titles")
)

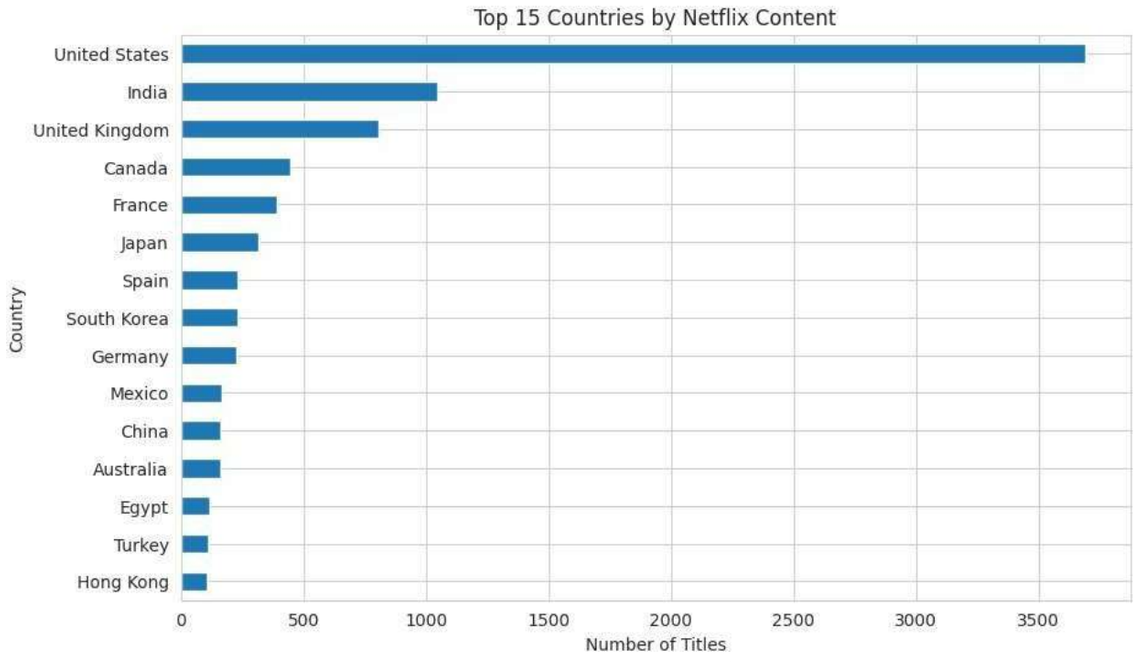
plt.figure(figsize=(10,6))

country_counts.head(15).sort_values().plot(
    kind='barh'
)
```

```
plt.title("Top 15 Countries by Netflix Content")
plt.xlabel("Number of Titles")
plt.ylabel("Country")

plt.show()
```

country	Number of Titles
United States	3690
India	1046
United Kingdom	806
Canada	445
France	393
Japan	318
Spain	232
South Korea	231
Germany	226
Mexico	169
China	162
Australia	160
Egypt	117
Turkey	113
Hong Kong	105




```
release_counts = df['release_year'].value_counts().sort_index()

display(
    df['release_year']
    .value_counts()
    .head(15)
    .to_frame("Number of Titles")
)

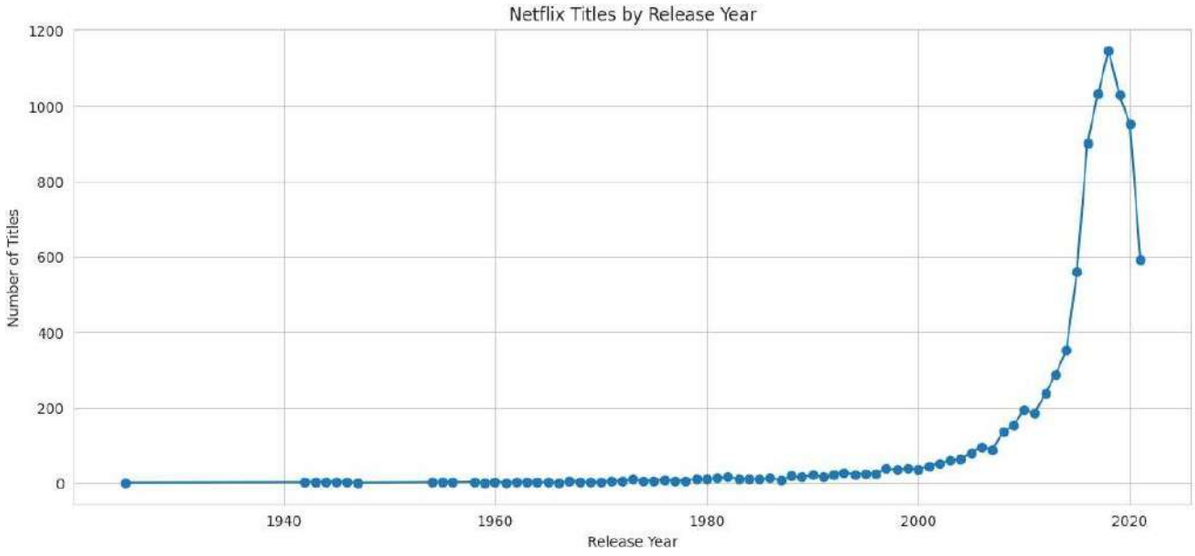
plt.figure(figsize=(14,6))

plt.plot(
    release_counts.index,
    release_counts.values,
    marker='o'
)

plt.title("Netflix Titles by Release Year")
plt.xlabel("Release Year")
plt.ylabel("Number of Titles")

plt.show()
```

release_year	Number of Titles
2018	1147
2017	1032
2019	1030
2020	953
2016	902
2021	592
2015	560
2014	352
2013	288
2012	237
2010	194
2011	185
2009	152
2008	136
2006	96




```
# STEP 10: TITLES ADDED TO NETFLIX BY YEAR

# Convert date_added column to datetime
df['date_added'] = pd.to_datetime(
    df['date_added'],
    errors='coerce'
)

# Extract year
df['added_year'] = df['date_added'].dt.year

# Count titles added per year
added_year_counts = df['added_year'].value_counts().sort_index()

print("Titles Added by Year:")
display(
    added_year_counts.to_frame("Number of Titles")
)

# Plot
plt.figure(figsize=(12,6))

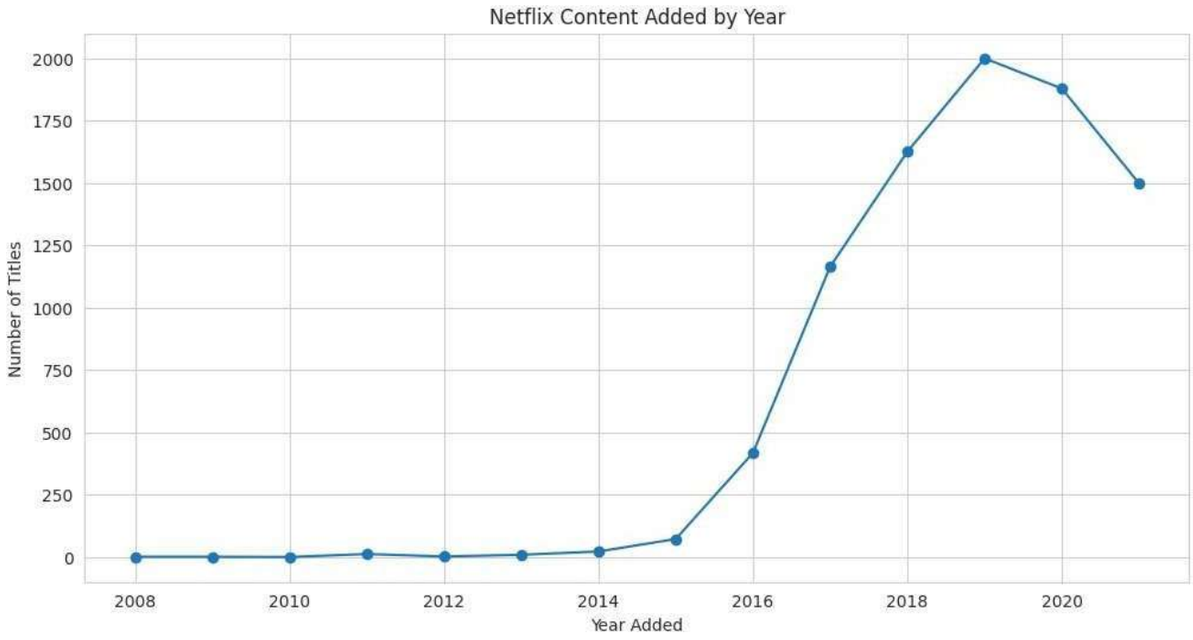
plt.plot(
    added_year_counts.index,
    added_year_counts.values,
    marker='o'
)

plt.title("Netflix Content Added by Year")
plt.xlabel("Year Added")
plt.ylabel("Number of Titles")

plt.show()
```

```
Titles Added by Year:
```

added_year	Number of Titles
2008.0	2
2009.0	2
2010.0	1
2011.0	13
2012.0	3
2013.0	10
2014.0	23
2015.0	73
2016.0	418
2017.0	1164
2018.0	1625
2019.0	1999
2020.0	1878
2021.0	1498




```
# STEP 11: NETFLIX CONTENT GROWTH TREND

yearly_growth = (
    df.dropna(subset=['date_added'])
    .groupby(df.dropna(subset=['date_added'])['date_added'].dt.year)
    .size()
)

print("Yearly Content Growth:")
display(
    yearly_growth.to_frame("Titles Added")
)

plt.figure(figsize=(12,6))

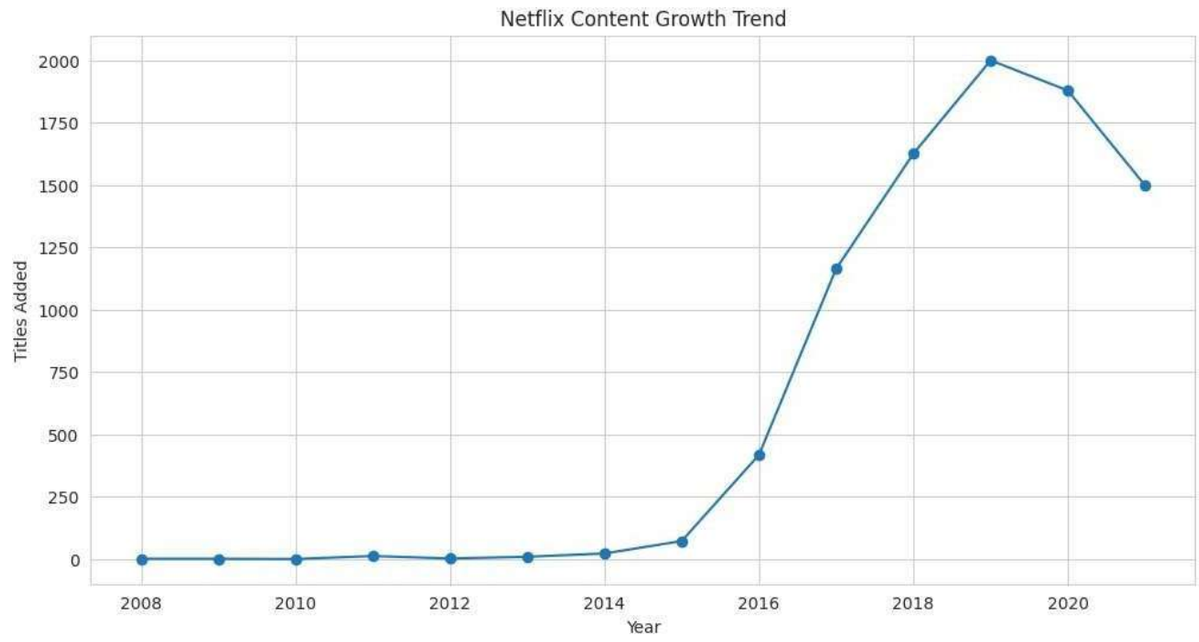
plt.plot(
    yearly_growth.index,
    yearly_growth.values,
    marker='o'
)

plt.title("Netflix Content Growth Trend")
plt.xlabel("Year")
plt.ylabel("Titles Added")

plt.show()
```

```
Yearly Content Growth:
```

date_added	Titles Added
2008	2
2009	2
2010	1
2011	13
2012	3
2013	10
2014	23
2015	73
2016	418
2017	1164
2018	1625
2019	1999
2020	1878
2021	1498




```
# STEP 12: MOVIE DURATION ANALYSIS

movies = df[df['type'] == 'Movie'].copy()

movies['duration_minutes'] = (
    movies['duration']
    .str.extract(r'(\d+)')[0]
    .astype(float)
)

print("Movie Duration Statistics:")
display(
    movies['duration_minutes'].describe()
)

plt.figure(figsize=(10,6))

plt.hist(
    movies['duration_minutes'].dropna(),
    bins=30
)

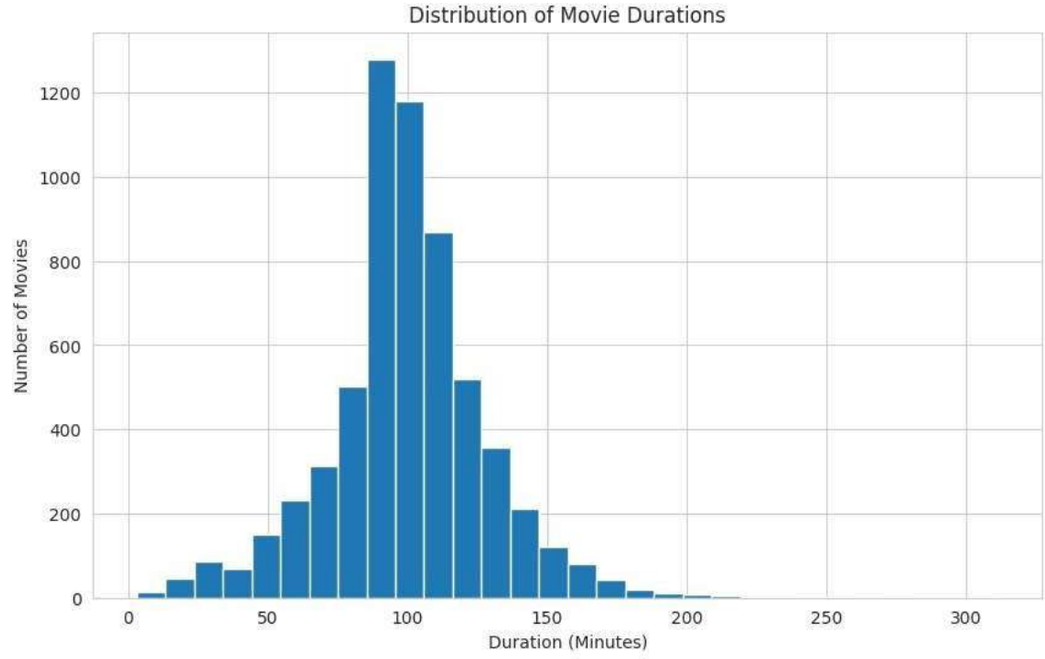
plt.title("Distribution of Movie Durations")
plt.xlabel("Duration (Minutes)")
plt.ylabel("Number of Movies")

plt.show()
```

```
Movie Duration Statistics:
```

	duration_minutes
count	6128.000000
mean	99.577187
std	28.290593
min	3.000000
25%	87.000000
50%	98.000000
75%	114.000000
max	312.000000

dtype: float64




```
# STEP 13: TV SHOW SEASONS ANALYSIS

tv_shows = df[df['type'] == 'TV Show'].copy()

tv_shows['seasons'] = (
    tv_shows['duration']
    .str.extract(r'(\d+)')[0]
    .astype(float)
)

season_counts = (
    tv_shows['seasons']
    .value_counts()
    .sort_index()
)

print("TV Shows by Number of Seasons:")
display(
    season_counts.to_frame("Number of TV Shows")
)

plt.figure(figsize=(10,6))

season_counts.plot(kind='bar')

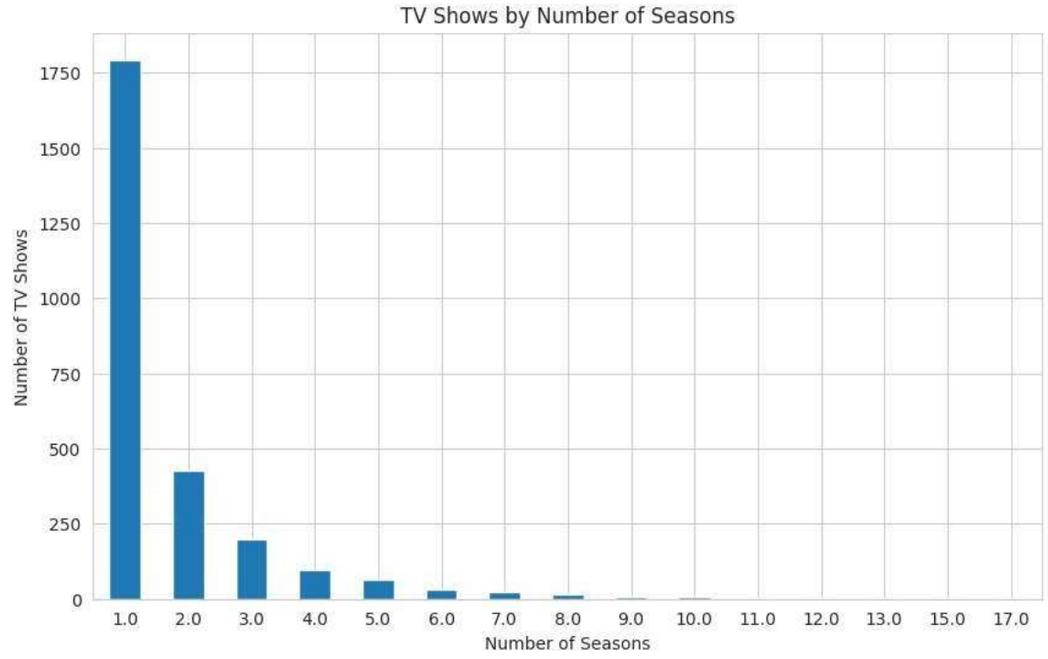
plt.title("TV Shows by Number of Seasons")
plt.xlabel("Number of Seasons")
plt.ylabel("Number of TV Shows")

plt.xticks(rotation=0)

plt.show()
```

```
TV Shows by Number of Seasons:
```

seasons	Number of TV Shows
1.0	1793
2.0	425
3.0	199
4.0	95
5.0	65
6.0	33
7.0	23
8.0	17
9.0	9
10.0	7
11.0	2
12.0	2
13.0	3
15.0	2
17.0	1




```
# STEP 14: TOP 10 GENRES

genre_series = (
    df['listed_in']
    .str.split(',')
    .explode()
    .str.strip()
)

genre_counts = genre_series.value_counts()

print("Top 10 Genres:")
display(
    genre_counts.head(10).to_frame("Number of Titles")
)

plt.figure(figsize=(10,6))

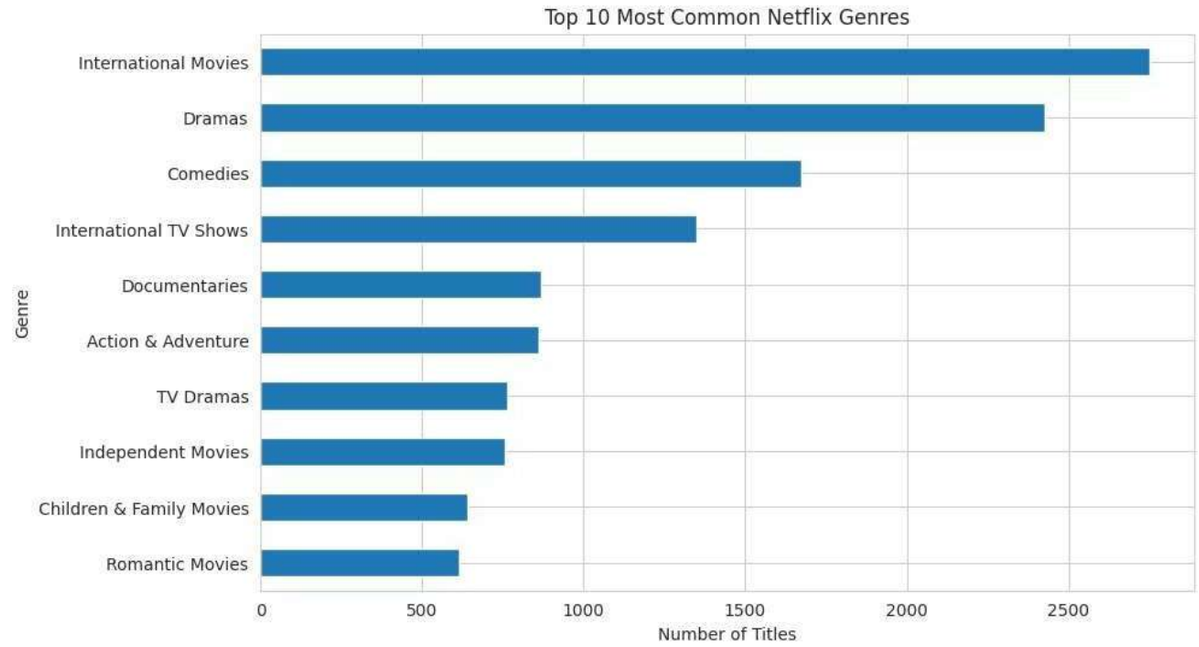
genre_counts.head(10).sort_values().plot(
    kind='barh'
)

plt.title("Top 10 Most Common Netflix Genres")
plt.xlabel("Number of Titles")
plt.ylabel("Genre")

plt.show()
```

```
Top 10 Genres:
```

listed_in	Number of Titles
International Movies	2752
Dramas	2427
Comedies	1674
International TV Shows	1351
Documentaries	869
Action & Adventure	859
TV Dramas	763
Independent Movies	756
Children & Family Movies	641
Romantic Movies	616




```
# STEP 15: DIRECTOR ANALYSIS

director_series = (
    df[df['director'] != 'Unknown']['director']
    .str.split(',')
    .explode()
    .str.strip()
)

director_counts = director_series.value_counts()

print("Top 15 Directors:")
display(
    director_counts.head(15).to_frame("Number of Titles")
)

plt.figure(figsize=(10,6))

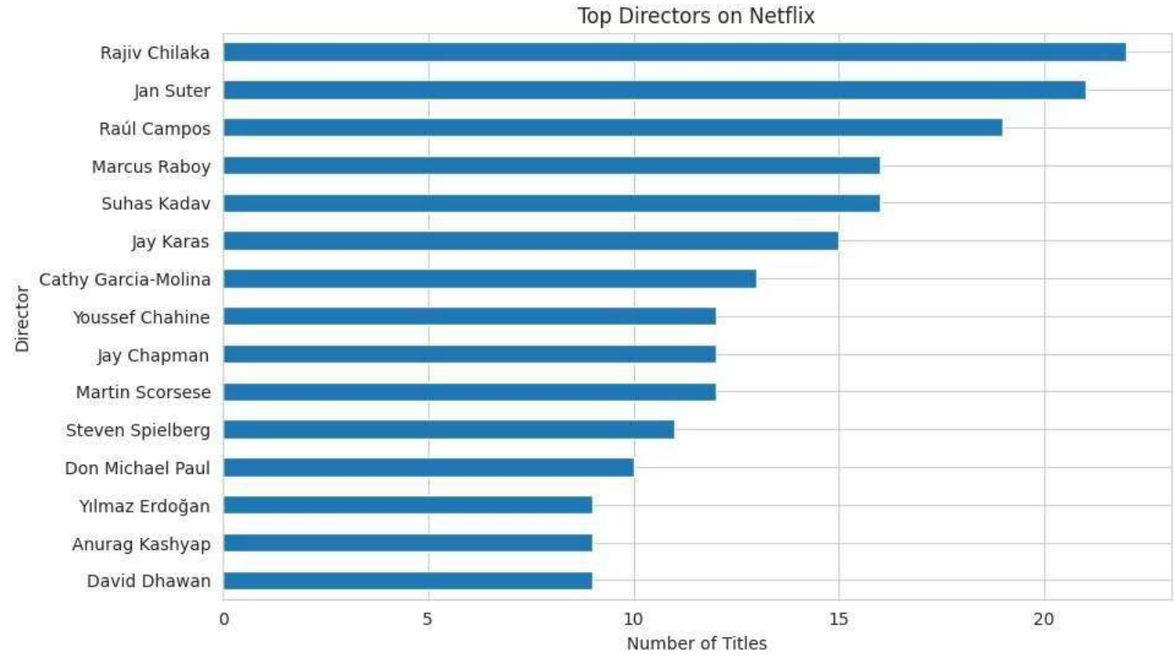
director_counts.head(15).sort_values().plot(
    kind='barh'
)

plt.title("Top Directors on Netflix")
plt.xlabel("Number of Titles")
plt.ylabel("Director")

plt.show()
```

```
Top 15 Directors:
```

director	Number of Titles
Rajiv Chilaka	22
Jan Suter	21
Raúl Campos	19
Suhas Kadav	16
Marcus Raboy	16
Jay Karas	15
Cathy Garcia-Molina	13
Martin Scorsese	12
Youssef Chahine	12
Jay Chapman	12
Steven Spielberg	11
Don Michael Paul	10
Anurag Kashyap	9
Yılmaz Erdoğan	9
David Dhawan	9




```
# STEP 16: ACTOR ANALYSIS

cast_series = (
    df[df['cast'] != 'Unknown']['cast']
    .str.split(',')
    .explode()
    .str.strip()
)

actor_counts = cast_series.value_counts()

print("Top 15 Actors:")
display(
    actor_counts.head(15).to_frame("Number of Appearances")
)

plt.figure(figsize=(10,6))

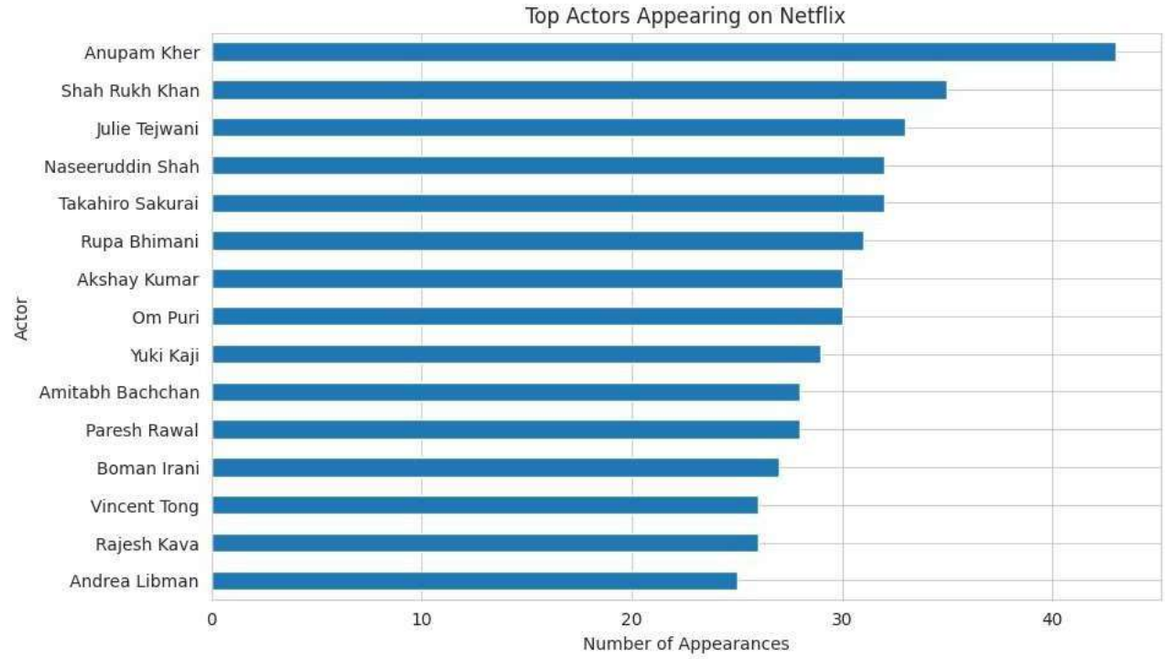
actor_counts.head(15).sort_values().plot(
    kind='barh'
)

plt.title("Top Actors Appearing on Netflix")
plt.xlabel("Number of Appearances")
plt.ylabel("Actor")

plt.show()
```

```
Top 15 Actors:
```

cast	Number of Appearances
Anupam Kher	43
Shah Rukh Khan	35
Julie Tejwani	33
Takahiro Sakurai	32
Naseeruddin Shah	32
Rupa Bhimani	31
Om Puri	30
Akshay Kumar	30
Yuki Kaji	29
Amitabh Bachchan	28
Paresh Rawal	28
Boman Irani	27
Rajesh Kava	26
Vincent Tong	26
Andrea Libman	25




```
# STEP 17: WORD CLOUD

text = ' '.join(
    df['description'].astype(str)
)

text = re.sub(
    r'[^a-zA-Z\s]',
    ' ',
    text
)

text = re.sub(
    r'\s+',
    ' ',
    text
).lower()

wordcloud = WordCloud(
    width=1200,
    height=600,
    background_color='white'
).generate(text)

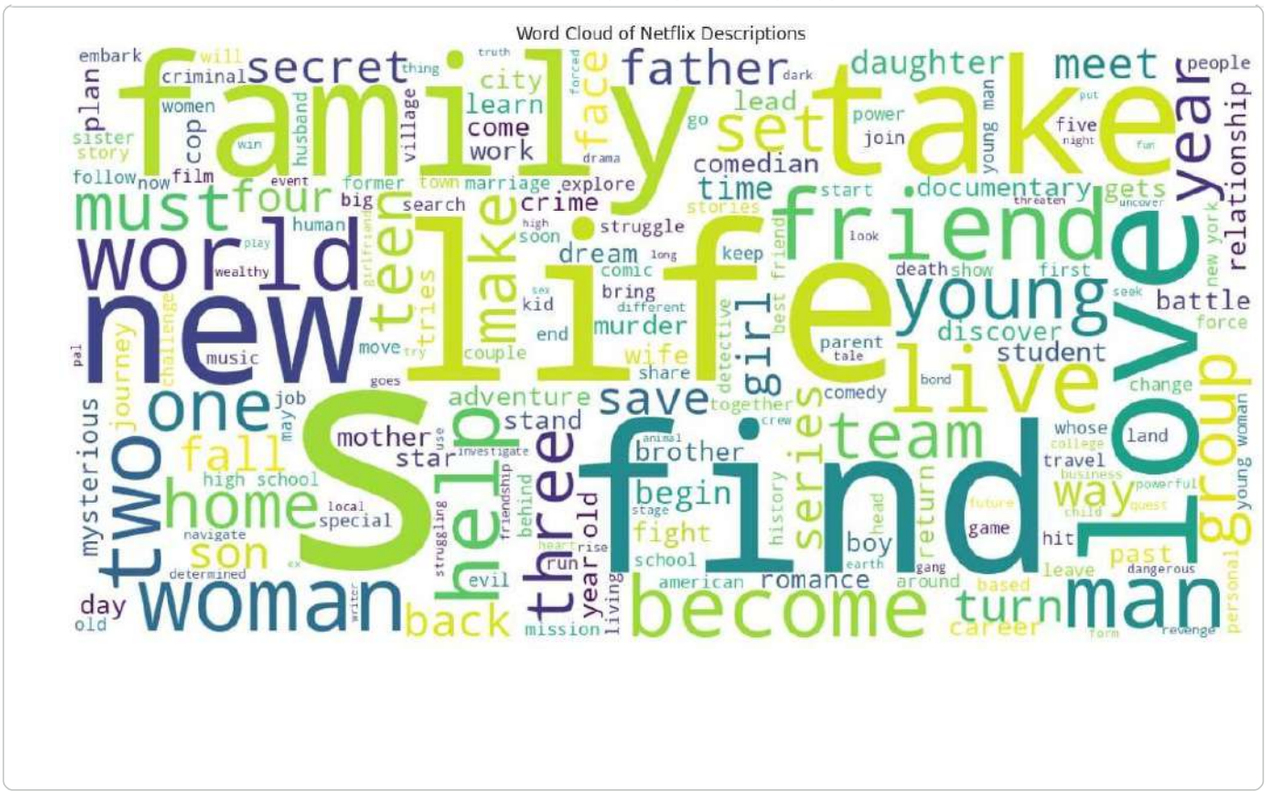
plt.figure(figsize=(15,8))

plt.imshow(
    wordcloud,
    interpolation='bilinear'
)

plt.axis('off')

plt.title("Word Cloud of Netflix Descriptions")

plt.show()
```




```
# STEP 18: MOST COMMON WORDS IN DESCRIPTION

words = re.findall(
    r'\b[a-zA-Z]{3,}\b',
    text
)

common_words = Counter(words).most_common(20)

common_words_df = pd.DataFrame(
    common_words,
    columns=['Word', 'Frequency']
)

print("Most Common Words:")
display(common_words_df)
```

Most Common Words:

	Word	Frequency
0	the	8183
1	and	6371
2	his	3362
3	with	2270
4	her	2167
5	for	1802
6	their	1671
7	when	1517
8	this	1396
9	from	1292
10	after	994
11	who	930
12	that	876
13	life	845
14	but	807
15	young	729
16	into	715
17	new	709
18	family	635
19	they	635

Next steps: Generate code with common_words_df | New interactive sheet

```
# STEP 19: MONTHLY CONTENT ADDITION

df['added_month'] = df['date_added'].dt.month

month_counts = (
    df['added_month']
    .value_counts()
    .sort_index()
```

```
)

month_names = {
    1: 'January',
    2: 'February',
    3: 'March',
    4: 'April',
    5: 'May',
    6: 'June',
    7: 'July',
    8: 'August',
    9: 'September',
    10: 'October',
    11: 'November',
    12: 'December'
}

monthly_data = month_counts.rename(
    index=month_names
)

print("Content Added by Month:")
display(
    monthly_data.to_frame("Number of Titles")
)

plt.figure(figsize=(12,6))

monthly_data.plot(kind='bar')

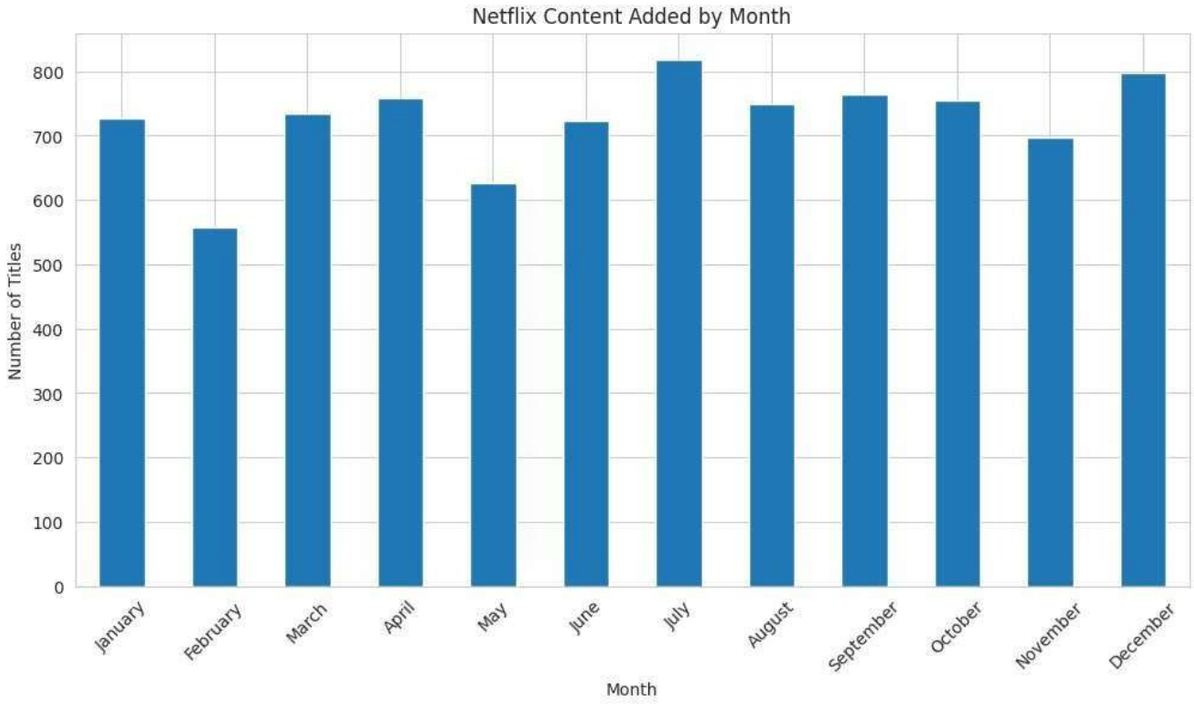
plt.title("Netflix Content Added by Month")
plt.xlabel("Month")
plt.ylabel("Number of Titles")

plt.xticks(rotation=45)

plt.show()
```

```
Content Added by Month:
```

added_month	Number of Titles
January	727
February	557
March	734
April	759
May	626
June	724
July	819
August	749
September	765
October	755
November	697
December	797




```
# STEP 20: COUNTRY vs CONTENT TYPE

country_type = df[
    ['country', 'type']
].copy()
```

```
country_type['country'] = (
    country_type['country']
    .str.split(',')
)

country_type = country_type.explode(
    'country'
)

country_type['country'] = (
    country_type['country']
    .str.strip()
)

top_countries = (
    country_type['country']
    .value_counts()
    .head(10)
    .index
)

country_type_top = country_type[
    country_type['country'].isin(top_countries)
]

country_type_counts = pd.crosstab(
    country_type_top['country'],
    country_type_top['type']
)

print("Movies vs TV Shows by Country:")
display(country_type_counts)

country_type_counts.plot(
    kind='bar',
    figsize=(14,7)
)

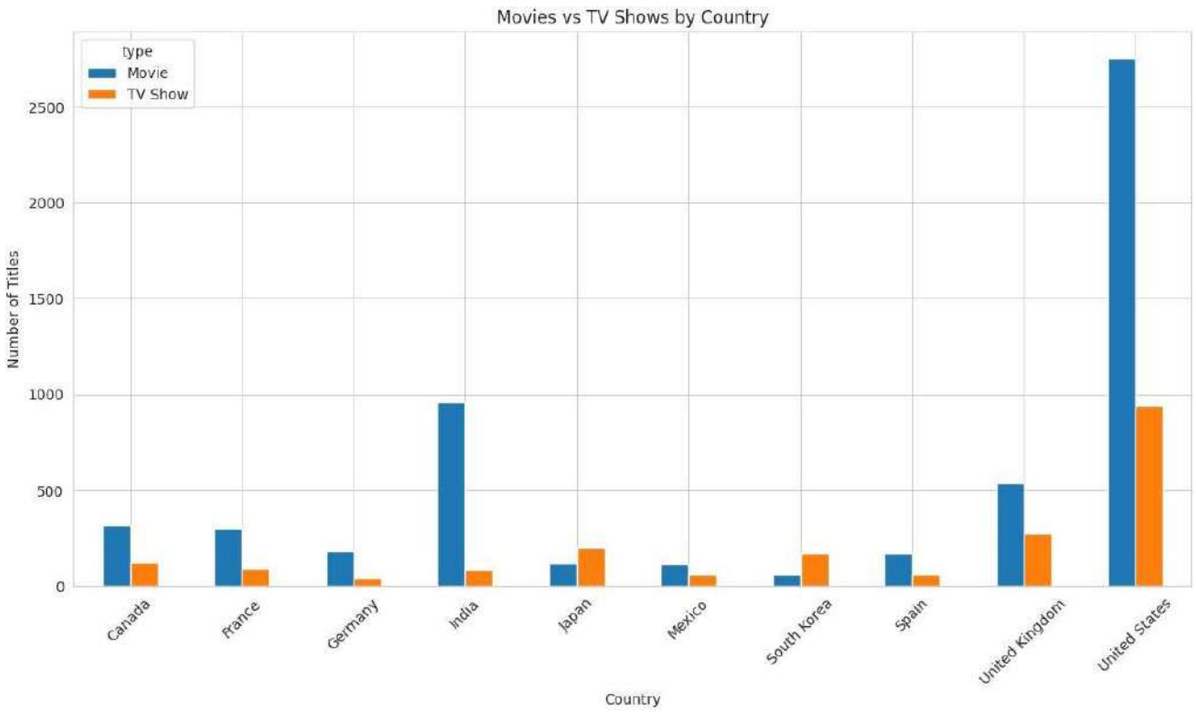
plt.title("Movies vs TV Shows by Country")
plt.xlabel("Country")
plt.ylabel("Number of Titles")

plt.xticks(rotation=45)

plt.show()
```

```
Movies vs TV Shows by Country:
```

country	type Movie	TV Show
Canada	319	126
France	303	90
Germany	182	44
India	962	84
Japan	119	199
Mexico	111	58
South Korea	61	170
Spain	171	61
United Kingdom	534	272
United States	2752	938



Next steps: Generate code with country_type_counts | New interactive sheet

```
# STEP 21: RATING vs CONTENT TYPE

rating_type = pd.crosstab(
    df['rating'],
    df['type']
)

print("Ratings for Movies vs TV Shows:")
display(rating_type)

rating_type.plot(
    kind='bar',
    figsize=(14,7)
)

plt.title("Ratings for Movies vs TV Shows")
plt.xlabel("Rating")
plt.ylabel("Number of Titles")

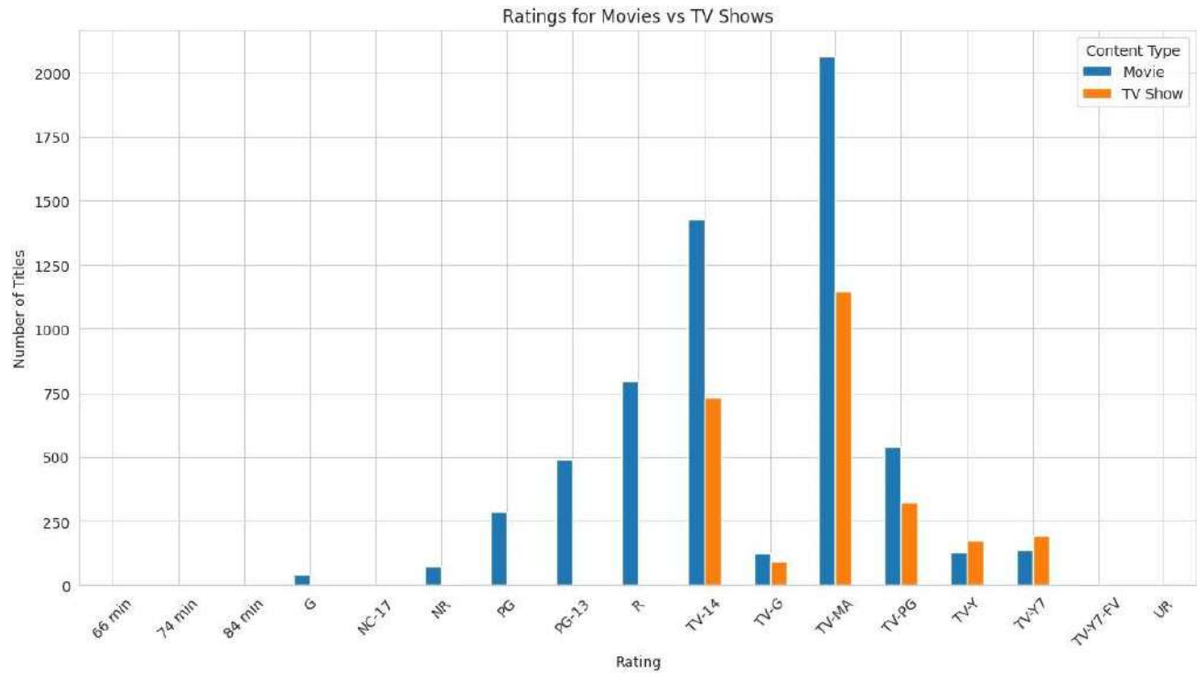
plt.xticks(rotation=45)

plt.legend(title="Content Type")

plt.show()
```

```
Ratings for Movies vs TV Shows:
```

rating	type Movie	TV Show
66 min	1	0
74 min	1	0
84 min	1	0
G	41	0
NC-17	3	0
NR	75	5
PG	287	0
PG-13	490	0
R	797	2
TV-14	1427	733
TV-G	126	94
TV-MA	2062	1145
TV-PG	540	323
TV-Y	131	176
TV-Y7	139	195
TV-Y7-FV	5	1
UR	3	0



Next steps: Generate code with rating_type | New interactive sheet

```
# STEP 22: OLDEST AND NEWEST TITLES

oldest_year = df['release_year'].min()
newest_year = df['release_year'].max()

print("Oldest Release Year:", oldest_year)
print("Newest Release Year:", newest_year)

print("\nOldest Titles:")

display(
    df[
        df['release_year'] == oldest_year
    ][
        [
            'title',
            'type',
            'release_year',
            'country',
            'rating'
        ]
    ]
)

print("\nNewest Titles:")

display(
    df[
```

```
        df['release_year'] == newest_year
    ][
        [
            'title',
            'type',
            'release_year',
            'country',
            'rating'
        ]
    ]
)
```

```
Oldest Release Year: 1925
Newest Release Year: 2021

Oldest Titles:
```

	title	type	release_year	country	rating
4250	Pioneers: First Women Filmmakers*	TV Show	1925	NaN	TV-14

```
Newest Titles:
```

	title	type	release_year	country	rating
1	Blood & Water	TV Show	2021	South Africa	TV-MA
2	Ganglands	TV Show	2021	NaN	TV-MA
3	Jailbirds New Orleans	TV Show	2021	NaN	TV-MA
4	Kota Factory	TV Show	2021	India	TV-MA
5	Midnight Mass	TV Show	2021	NaN	TV-MA
...	...	...	...	...	...
1468	What Happened to Mr. Cha?	Movie	2021	South Korea	TV-MA
1551	Hilda	TV Show	2021	United Kingdom, Canada, United States	TV-Y7
1696	Polly Pocket	TV Show	2021	Canada, United States, Ireland	TV-Y
2920	Love Is Blind	TV Show	2021	United States	TV-MA
8437	The Netflix Afterparty	TV Show	2021	United States	TV-MA

592 rows × 5 columns

```
# STEP 23: GENRE CO-OCCURRENCE HEATMAP

top_genres = genre_counts.head(15).index.tolist()

genre_matrix = pd.DataFrame(
    0,
    index=top_genres,
    columns=top_genres
)

for genres in df['listed_in'].dropna():

    genre_list = [
        g.strip()
        for g in genres.split(',')
        if g.strip() in top_genres
    ]

    for i in range(len(genre_list)):

        for j in range(i + 1, len(genre_list)):

            g1 = genre_list[i]
            g2 = genre_list[j]

            genre_matrix.loc[g1, g2] += 1
            genre_matrix.loc[g2, g1] += 1

print("Genre Co-occurrence Matrix:")
display(genre_matrix)

plt.figure(figsize=(14,10))

sns.heatmap(
    genre_matrix,
    annot=True,
    fmt='g'
)

plt.title("Genre Co-occurrence Heatmap")
plt.xlabel("Genre")
plt.ylabel("Genre")

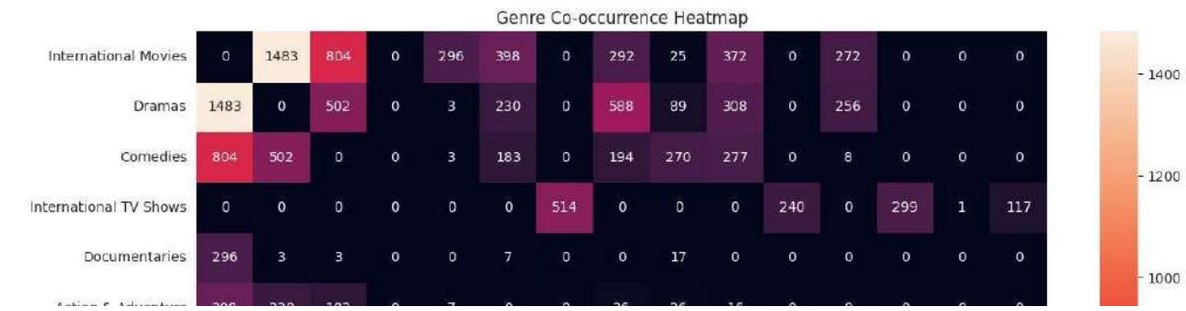
plt.xticks(
    rotation=45,
    ha='right'
)

plt.yticks(rotation=0)

plt.show()
```

Genre Co-occurrence Matrix:

	International Movies	Dramas	Comedies	International TV Shows	Documentaries	Action & Adventure	TV Dramas	Independent Movies	Children & Family Movies	Romantic Movies
International Movies	0	1483	804	0	296	398	0	292	25	372
Dramas	1483	0	502	0	3	230	0	588	89	308
Comedies	804	502	0	0	3	183	0	194	270	277
International TV Shows	0	0	0	0	0	0	514	0	0	0
Documentaries	296	3	3	0	0	7	0	0	17	0
Action & Adventure	398	230	183	0	7	0	0	36	26	16
TV Dramas	0	0	0	514	0	0	0	0	0	0
Independent Movies	292	588	194	0	0	36	0	0	6	32
Children & Family Movies	25	89	270	0	17	26	0	6	0	17
Romantic Movies	372	308	277	0	0	16	0	32	17	0
TV Comedies	0	0	0	240	0	0	106	0	0	0
Thrillers	272	256	8	0	0	9	0	70	0	4
Crime TV Shows	0	0	0	299	0	0	180	0	0	0
Kids' TV	0	0	0	1	0	0	13	0	0	0
Docuseries	0	0	0	117	0	0	3	0	0	0



LEARNING OUTCOMES

Arun S

- Understand the purpose and importance of Exploratory Data Analysis (EDA).
- Load and inspect the Netflix titles dataset using Pandas.
- Examine dataset dimensions, data types, summary statistics, and missing values.
- Identify and handle missing values and duplicate records.
- Analyze content types, ratings, countries, genres, and release trends.
- Use grouping and value counts to summarize categorical data.
- Create visualizations such as plots, count plots, heatmaps, and a word cloud.
- Interpret EDA findings to describe patterns and characteristics in the dataset.

Arun S

241801027

LEARNING OBJECTIVES

Understand time series data and seasonal patterns.

Perform stationarity testing using the ADF test.

Apply differencing to make the series stationary.

Build an ARIMA model and forecast future passengers.

Evaluate forecasts using MSE, MAE, and RMSE.

BUILT-IN METHODS / FUNCTIONS

Function Purpose

pd.read_csv() Load dataset

parse_dates Convert date column

set_index() / index_col Set time index

head() / info() Explore dataset

plt.plot() Plot time series

adfuller() Check stationarity

diff() Apply differencing

train_test_split() / slicing Split data

ARIMA() Build forecasting model

fit() Train model

forecast() Predict future values

mean_squared_error() Calculate MSE

mean_absolute_error() Calculate MAE

np.sqrt() Calculate RMSE

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
import zipfile
import os

with zipfile.ZipFile('airline passengar.zip', 'r') as zip_ref:
    zip_ref.extractall('/content/airline_data')

os.listdir('/content/airline_data')
```

```
['airline-passengers.csv']
```

```
import os

for root, dirs, files_list in os.walk('/content/airline_data'):
    for file in files_list:
        print(os.path.join(root, file))
```

```
/content/airline_data/airline-passengers.csv
```

```
df = pd.read_csv(
    '/content/airline_data/airline-passengers.csv',
    parse_dates=['Month'],
    index_col='Month'
)

df.head()
```

Month	Passengers
1949-01-01	112
1949-02-01	118
1949-03-01	132
1949-04-01	129
1949-05-01	121

Next steps: Generate code with df | New interactive sheet

```
from google.colab import files

uploaded = files.upload()
```

Choose Files airline passengar.zip

airline passengar.zip(application/x-zip-compressed) - 755 bytes, last modified: 9/25/2026 - 100% done

```
Saving airline passengar.zip to airline passengar.zip
```

```
df = pd.read_csv(
    '/content/airline passengar.zip',
    parse_dates=['Month'],
    index_col='Month'
)

df.head()
```

Month	Passengers
1949-01-01	112
1949-02-01	118
1949-03-01	132
1949-04-01	129
1949-05-01	121

Next steps: Generate code with df | New interactive sheet

```
url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/airline-passengers.csv"

df = pd.read_csv(
    url,
    parse_dates=['Month'],
    index_col='Month'
)

df.head()
```

Month	Passengers
1949-01-01	112
1949-02-01	118
1949-03-01	132
1949-04-01	129
1949-05-01	121

Next steps: Generate code with df | New interactive sheet

```
print("Shape:", df.shape)
print("\nColumns:")
print(df.columns)

print("\nData Types:")
print(df.dtypes)

print("\nMissing Values:")
print(df.isnull().sum())

print("\nSummary Statistics:")
print(df.describe())
```

```
Shape: (144, 1)

Columns:
Index(['Passengers'], dtype='object')

Data Types:
Passengers    int64
dtype: object

Missing Values:
Passengers    0
dtype: int64

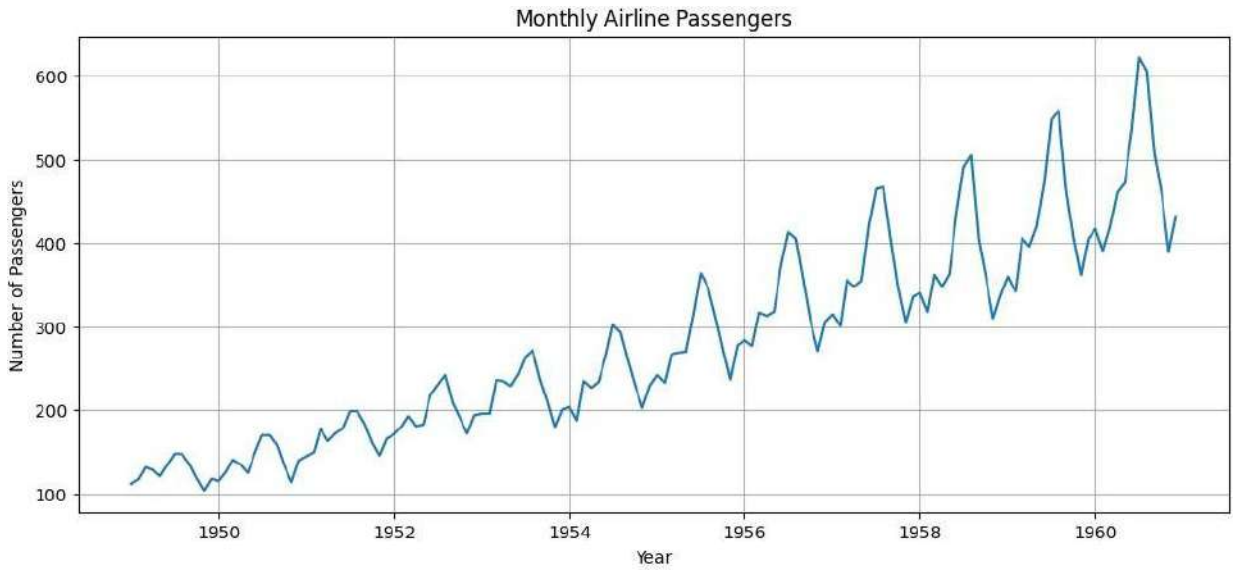
Summary Statistics:
       Passengers
count  144.000000
mean   280.298611
std    119.966317
min    104.000000
25%    180.000000
50%    265.500000
75%    360.500000
max    622.000000
```

```
plt.figure(figsize=(12,5))

plt.plot(df.index, df['Passengers'])

plt.title("Monthly Airline Passengers")
plt.xlabel("Year")
plt.ylabel("Number of Passengers")

plt.grid()
plt.show()
```




```
from statsmodels.tsa.stattools import adfuller
```

```
result = adfuller(df['Passengers'])

print("ADF Statistic:", result[0])
print("p-value:", result[1])

if result[1] <= 0.05:
    print("The series is Stationary")
else:
    print("The series is Non-Stationary")
```

```
ADF Statistic: 0.8153688792060498
p-value: 0.991880243437641
The series is Non-Stationary
/tmp/ipykernel_4969/1410127404.py:1: FutureWarning: adfuller currently returns a plain tuple whose leng
  result = adfuller(df['Passengers'])
```

```
series = df['Passengers']

train_size = int(len(series) * 0.8)

train = series[:train_size]
test = series[train_size:]

print("Training Data:", len(train))
print("Testing Data:", len(test))
```

```
Training Data: 115
Testing Data: 29
```

```
from statsmodels.tsa.arima.model import ARIMA
```

```
model = ARIMA(train, order=(5, 1, 0))

model_fit = model.fit()

print(model_fit.summary())
```

```
                               SARIMAX Results
==============================================================================
Dep. Variable:             Passengers   No. Observations:                  115
Model:                 ARIMA(5, 1, 0)   Log Likelihood                -524.549
Date:                Fri, 25 Sep 2026   AIC                           1061.098
Time:                        05:31:37   BIC                           1077.515
```

```
Sample:                    01-01-1949   HQIC                          1067.760
                         - 07-01-1958
Covariance Type:                 opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.L1          0.3558      0.092      3.854      0.000       0.175       0.537
ar.L2         -0.2644      0.109     -2.422      0.015      -0.478      -0.050
ar.L3          0.0847      0.078      1.089      0.276      -0.068       0.237
ar.L4         -0.3179      0.088     -3.621      0.000      -0.490      -0.146
ar.L5          0.0748      0.106      0.704      0.482      -0.133       0.283
sigma2       578.0868     98.904      5.845      0.000     384.239     771.935
===================================================================================
Ljung-Box (L1) (Q):                   0.01   Jarque-Bera (JB):                 4.12
Prob(Q):                              0.94   Prob(JB):                         0.13
Heteroskedasticity (H):               5.78   Skew:                             0.29
Prob(H) (two-sided):                  0.00   Kurtosis:                         2.27
===================================================================================

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
/usr/local/lib/python3.13/dist-packages/statsmodels/tsa/base/tsa_model.py:480: ValueWarning: No frequer
  self._init_dates(dates, freq)
/usr/local/lib/python3.13/dist-packages/statsmodels/tsa/base/tsa_model.py:480: ValueWarning: No frequer
  self._init_dates(dates, freq)
/usr/local/lib/python3.13/dist-packages/statsmodels/tsa/base/tsa_model.py:480: ValueWarning: No frequer
  self._init_dates(dates, freq)
```

```
forecast = model_fit.forecast(steps=len(test))

forecast = pd.Series(
    forecast,
    index=test.index
)

print(forecast)
```

```
Month
1958-08-01    500.902643
1958-09-01    489.904481
1958-10-01    466.349010
1958-11-01    449.296639
1958-12-01    449.565681
1959-01-01    456.411345
1959-02-01    463.997445
1959-03-01    468.568886
1959-04-01    467.408711
1959-05-01    464.273767
1959-06-01    461.952615
1959-07-01    460.971351
1959-08-01    461.681083
1959-09-01    462.906268
1959-10-01    463.574875
1959-11-01    463.687326
1959-12-01    463.355327
1960-01-01    462.927691
1960-02-01    462.751916
1960-03-01    462.788567
1960-04-01    462.925812
1960-05-01    463.061184
1960-06-01    463.100065
1960-07-01    463.064937
1960-08-01    463.012736
1960-09-01    462.973972
1960-10-01    462.968769
1960-11-01    462.986820
1960-12-01    463.005302
Name: predicted_mean, dtype: float64
```

```
from sklearn.metrics import mean_squared_error, mean_absolute_error
import numpy as np
```

```
mse = mean_squared_error(test, forecast)
mae = mean_absolute_error(test, forecast)
rmse = np.sqrt(mse)

print("MSE  :", mse)
print("MAE  :", mae)
print("RMSE :", rmse)
```

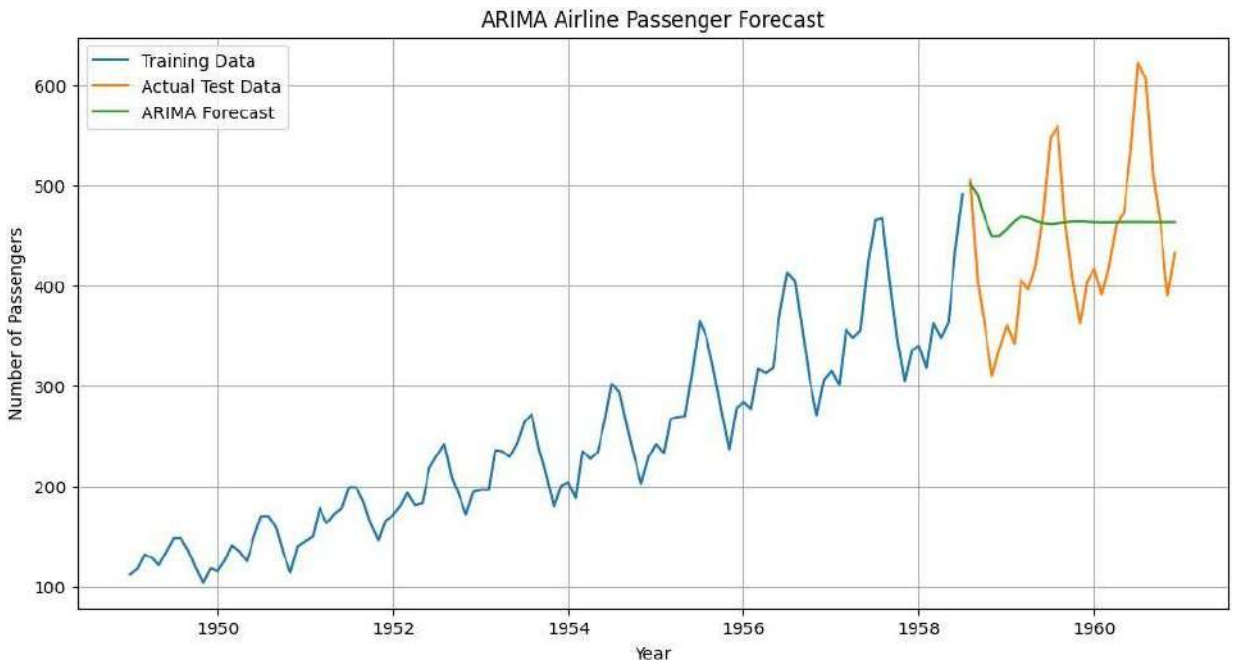
```
MSE  : 6506.672066539828
MAE  : 67.38350057393653
RMSE : 80.66394527011326
```

```
plt.figure(figsize=(12,6))

plt.plot(train.index, train, label="Training Data")
plt.plot(test.index, test, label="Actual Test Data")
plt.plot(test.index, forecast, label="ARIMA Forecast")

plt.title("ARIMA Airline Passenger Forecast")
plt.xlabel("Year")
plt.ylabel("Number of Passengers")

plt.legend()
plt.grid()
plt.show()
```



LEARNING OUTCOMES

Analyze time series trends and seasonality.

Test and achieve stationarity using differencing.

Build an ARIMA forecasting model.

Generate and evaluate passenger forecasts using error metrics.

LEARNING OUTCOMES

After completing this experiment, students will be able to:

- Understand time series data and identify trend and seasonal patterns.
- Load, inspect, and visualize airline passenger data over time.
- Check time series stationarity using the Augmented Dickey-Fuller (ADF) test.
- Apply differencing to help make a time series stationary.
- Prepare training and testing data for time series forecasting.
- Build and fit an ARIMA model to forecast future passenger values.
- Generate forecasts and compare predicted values with actual observations.
- Evaluate forecasting performance using MSE, MAE, and RMSE.

Double-click (or enter) to edit

Arun S

241801027

LEARNING OBJECTIVES

Understand the concept of K-Means Clustering for customer segmentation.

Select and standardize important customer features.

Determine the optimal number of clusters using the Elbow Method.

Apply K-Means and visualize customer clusters.

Analyze clusters and suggest suitable marketing strategies.

BUILT-IN METHODS / FUNCTIONS

Function Purpose pd.read_csv() Load dataset

head() / tail() View records

shape / dtypes Dataset details

isnull().sum() Check missing values

duplicated().sum() Check duplicates

describe() Summary statistics

StandardScaler() Standardize feature

KMeans() Perform clustering

fit_predict() Train and assign clusters

groupby() Analyze clusters

plt.scatter() Visualize clusters

plt.plot() Plot Elbow curve

inverse_transform() Convert centroids back to original scale

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
```

```
from google.colab import files

uploaded = files.upload()
```

Choose files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving Mall_Customers.csv to Mall_Customers.csv
```

```
df = pd.read_csv('Mall_Customers.csv')

df.head()
```

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

```
df.head()
```

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

```
df.tail()
```

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
195	196	Female	35	120	79
196	197	Female	45	126	28
197	198	Male	32	126	74
198	199	Male	32	137	18
199	200	Male	30	137	83

```
print("Shape:", df.shape)
```

```
Shape: (200, 5)
```

```
print(df.dtypes)
```

```
CustomerID                int64
Gender                   object
Age                       int64
Annual Income (k$)        int64
Spending Score (1-100)    int64
dtype: object
```

```
print(df.isnull().sum())
```

```
CustomerID                0
Gender                    0
Age                       0
Annual Income (k$)        0
Spending Score (1-100)    0
dtype: int64
```

```
print("Duplicate records:", df.duplicated().sum())
```

```
Duplicate records: 0
```

```
df.describe()
```

	CustomerID	Age	Annual Income (k$)	Spending Score (1-100)
count	200.000000	200.000000	200.000000	200.000000
mean	100.500000	38.850000	60.560000	50.200000
std	57.879185	13.969007	26.264721	25.823522
min	1.000000	18.000000	15.000000	1.000000
25%	50.750000	28.750000	41.500000	34.750000
50%	100.500000	36.000000	61.500000	50.000000
75%	150.250000	49.000000	78.000000	73.000000
max	200.000000	70.000000	137.000000	99.000000

```
X = df[['Annual Income (k$)', 'Spending Score (1-100)']]

X.head()
```

	Annual Income (k$)	Spending Score (1-100)
0	15	39
1	15	81
2	16	6
3	16	77
4	17	40

```
scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

X_scaled[:5]
```

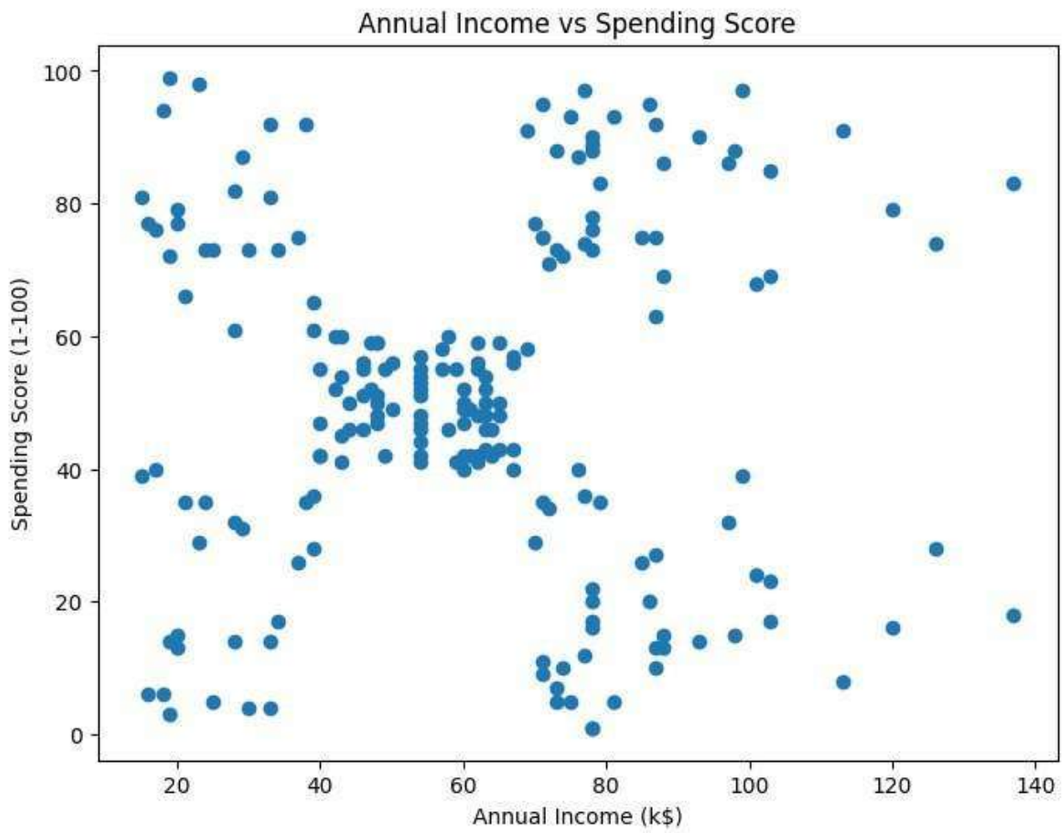
```
array([[-1.73899919, -0.43480148],
       [-1.73899919,  1.19570407],
       [-1.70082976, -1.71591298],
       [-1.70082976,  1.04041783],
       [-1.66266033, -0.39597992]])
```

```
plt.figure(figsize=(8,6))

plt.scatter(
    X['Annual Income (k$)'],
    X['Spending Score (1-100)']
)

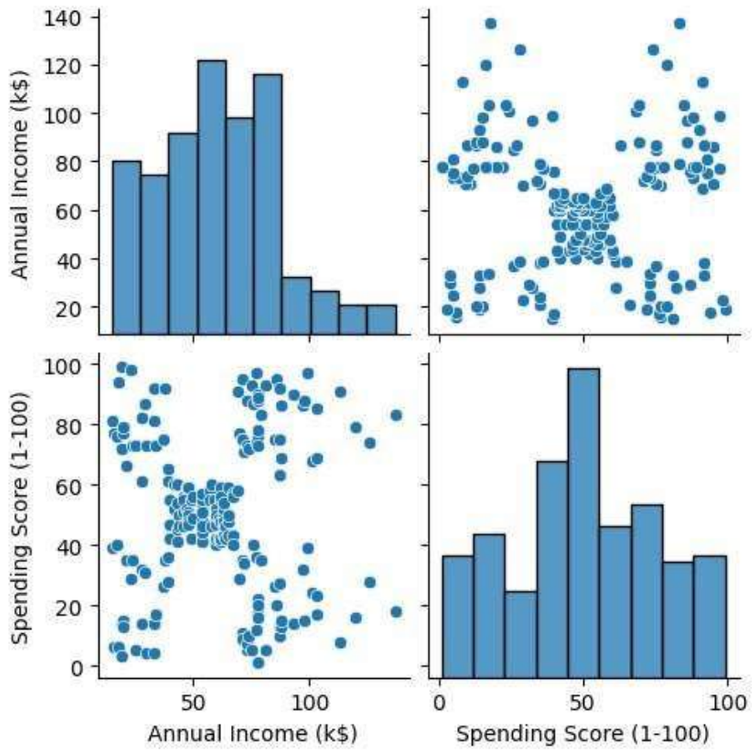
plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score (1-100)')
plt.title('Annual Income vs Spending Score')

plt.show()
```




```
sns.pairplot(
    df[['Annual Income (k$)', 'Spending Score (1-100)']]
)

plt.show()
```




```
inertia = []

for k in range(1, 11):
    kmeans = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10
    )

    kmeans.fit(X_scaled)
    inertia.append(kmeans.inertia_)
```

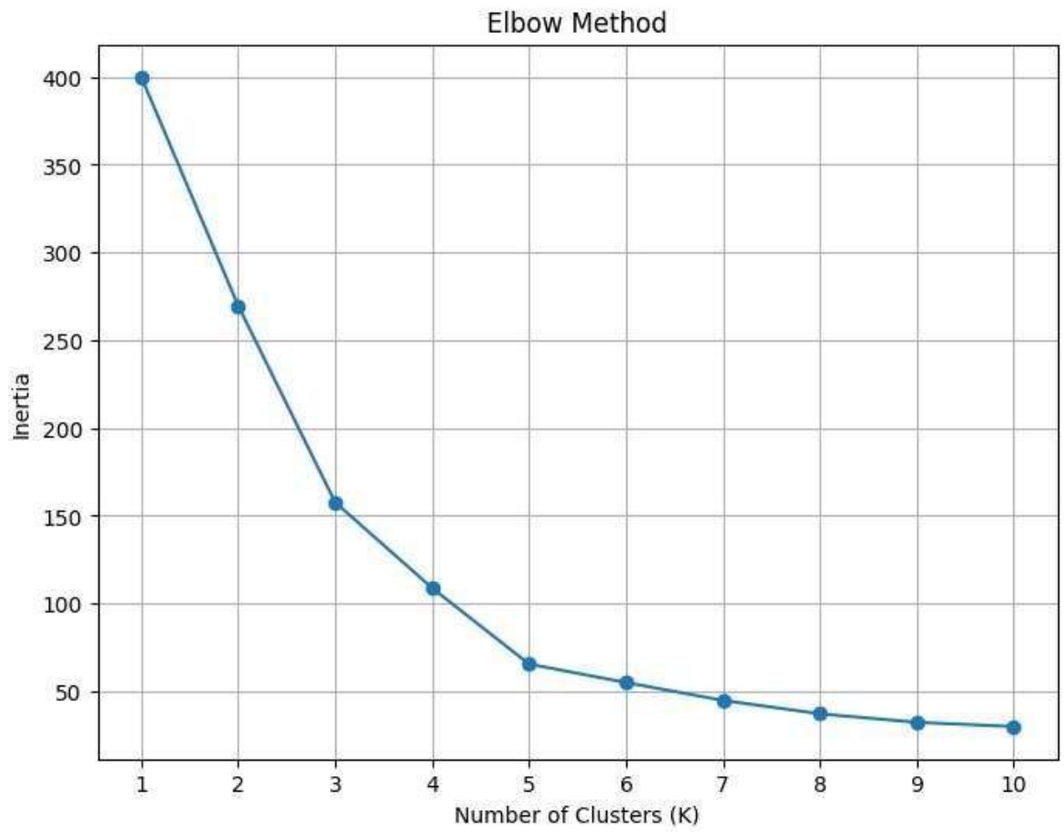
```
plt.figure(figsize=(8,6))

plt.plot(range(1, 11), inertia, marker='o')

plt.xlabel('Number of Clusters (K)')
plt.ylabel('Inertia')
plt.title('Elbow Method')

plt.xticks(range(1, 11))
plt.grid()

plt.show()
```




```
kmeans = KMeans(
    n_clusters=5,
    random_state=42,
    n_init=10
)

df['Cluster'] = kmeans.fit_predict(X_scaled)

df.head()
```

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)	Cluster
0	1	Male	19	15	39	4
1	2	Male	21	15	81	2
2	3	Female	20	16	6	4
3	4	Female	23	16	77	2
4	5	Female	31	17	40	4

```
df.to_csv('Mall_Customers_Clustered.csv', index=False)

print("Clustered dataset saved successfully.")
```

```
Clustered dataset saved successfully.
```

```
from google.colab import files

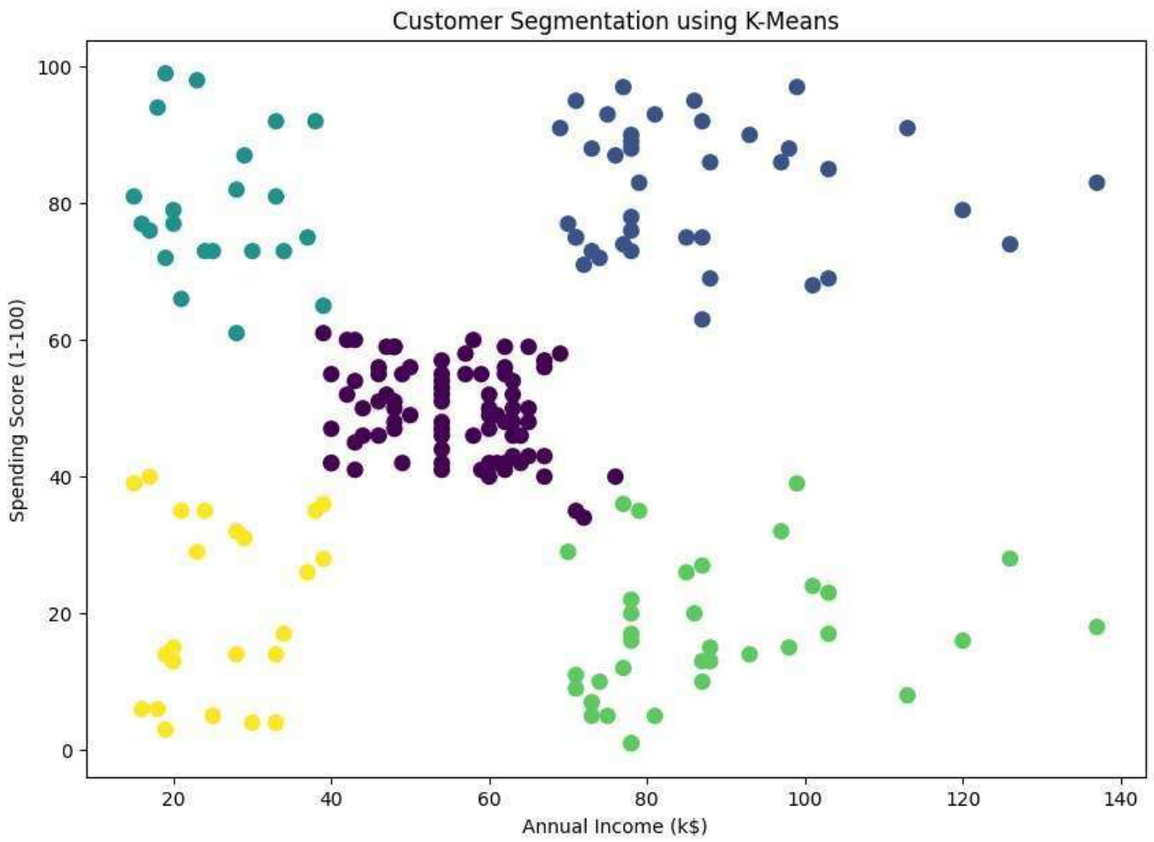
files.download('Mall_Customers_Clustered.csv')
```

```
plt.figure(figsize=(10,7))

plt.scatter(
    df['Annual Income (k$)'],
    df['Spending Score (1-100)'],
    c=df['Cluster'],
    s=60
)

plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score (1-100)')
plt.title('Customer Segmentation using K-Means')
```

```
plt.show()
```




```
centroids = scaler.inverse_transform(kmeans.cluster_centers_)

print("Cluster Centroids:")

for i, centroid in enumerate(centroids):
    print(
        f"Cluster {i}: "
        f"Annual Income = {centroid[0]:.2f}, "
        f"Spending Score = {centroid[1]:.2f}"
    )
```

```
Cluster Centroids:
Cluster 0: Annual Income = 55.30, Spending Score = 49.52
Cluster 1: Annual Income = 86.54, Spending Score = 82.13
Cluster 2: Annual Income = 25.73, Spending Score = 79.36
Cluster 3: Annual Income = 88.20, Spending Score = 17.11
Cluster 4: Annual Income = 26.30, Spending Score = 20.91
```

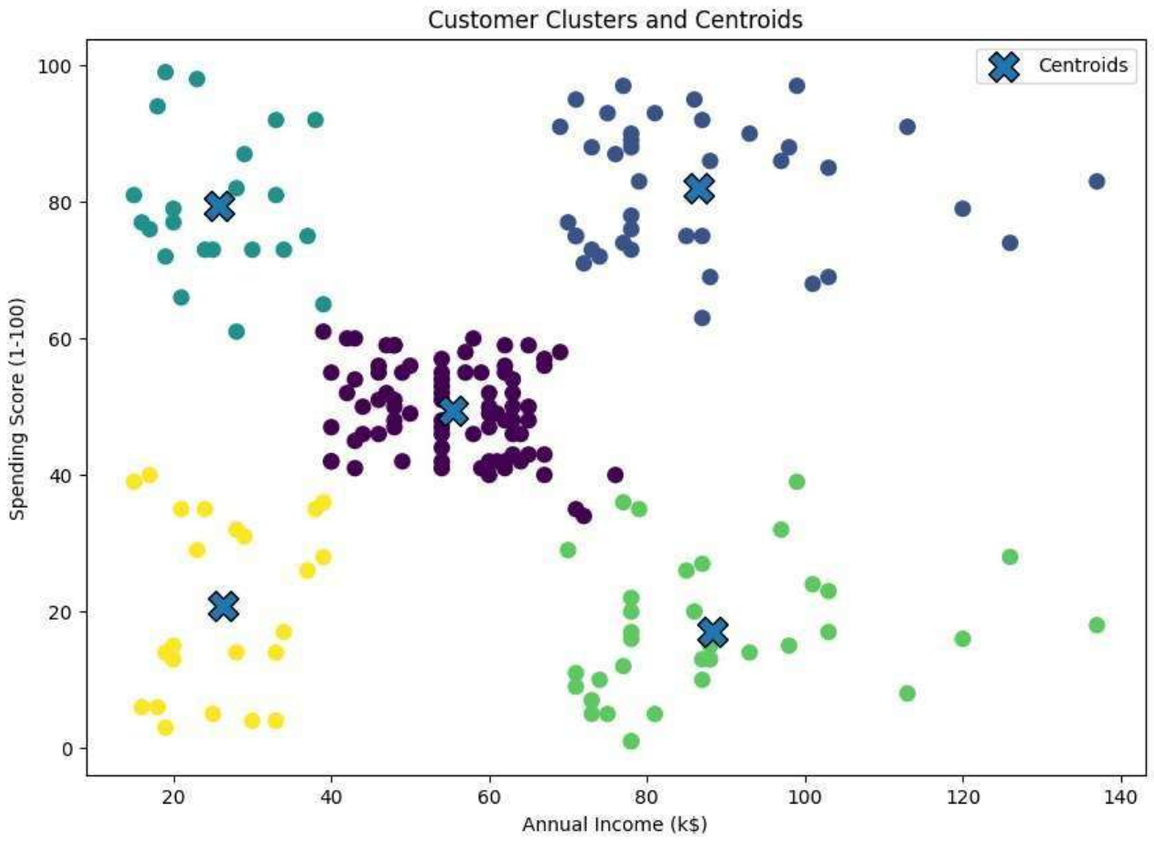
```
plt.figure(figsize=(10,7))

plt.scatter(
    df['Annual Income (k$)'],
    df['Spending Score (1-100)'],
    c=df['Cluster'],
    s=60
)

plt.scatter(
    centroids[:, 0],
    centroids[:, 1],
    marker='X',
    s=250,
    edgecolor='black',
    label='Centroids'
)

plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score (1-100)')
plt.title('Customer Clusters and Centroids')

plt.legend()
plt.show()
```




```
cluster_counts = df['Cluster'].value_counts().sort_index()

print(cluster_counts)
```

```
Cluster
0    81
1    39
2    22
3    35
4    23
Name: count, dtype: int64
```

```
cluster_summary = df.groupby('Cluster')[
    ['Annual Income (k$)', 'Spending Score (1-100)']
].mean()

print(cluster_summary)
```

```
         Annual Income (k$)  Spending Score (1-100)
Cluster
0                 55.296296               49.518519
1                 86.538462               82.128205
2                 25.727273               79.363636
3                 88.200000               17.114286
4                 26.304348               20.913043
```

```
cluster_summary
```

Cluster	Annual Income (k$)	Spending Score (1-100)
0	55.296296	49.518519
1	86.538462	82.128205
2	25.727273	79.363636
3	88.200000	17.114286
4	26.304348	20.913043

```
for cluster in cluster_summary.index:
    income = cluster_summary.loc[cluster, 'Annual Income (k$)']
    spending = cluster_summary.loc[cluster, 'Spending Score (1-100)']
```

```
    print(f"\nCluster {cluster}")
    print(f"Average Income: {income:.2f} k$")
    print(f"Average Spending Score: {spending:.2f}")
```

```
Cluster 0
Average Income: 55.30 k$
Average Spending Score: 49.52

Cluster 1
Average Income: 86.54 k$
Average Spending Score: 82.13

Cluster 2
Average Income: 25.73 k$
Average Spending Score: 79.36

Cluster 3
Average Income: 88.20 k$
```

LEARNING OUTCOMES

After completing this experiment, the learner will be able to:

1. Explain the purpose of K-Means clustering in customer segmentation.
2. Prepare customer data by checking missing values, duplicates, and feature scales.
3. Standardize numerical features before applying a clustering algorithm.
4. Use the Elbow Method to help select a suitable number of clusters.
5. Apply K-Means to group customers based on annual income and spending score.
6. Visualize and interpret the resulting customer clusters.
7. Describe how cluster characteristics can inform suitable marketing strategies.

EXPERIMENT 8

Arun S

241801027

1. AIM

To analyse the Sample Superstore dataset using Python, Pandas, Matplotlib and Seaborn, and visualize sales and profit across product categories, regions and months.

2. LEARNING OBJECTIVES

• Load a CSV dataset and inspect its rows, columns, data types and missing values.

• Summarize the dataset using descriptive statistics.

• Group and aggregate sales and profit by category, region and month.

• Create bar charts, line charts and scatter plots to understand business trends.

• Use filtering functions to analyse a selected region or product category.

3. BUILT-IN METHODS / FUNCTIONS USED

Method / Function	Purpose
pd.read_csv()	Reads the CSV dataset into a DataFrame.
df.head(), df.shape, df.columns	Displays sample rows, dataset dimensions and column names.
df.info(), df.isnull().sum()	Shows data types and counts missing values.
df.describe()	Generates descriptive statistical summaries.
pd.to_datetime()	Converts order dates into date-time values.
groupby(), sum(), sort_values(), reset_i	ndGerxo(u) ps records and calculates aggregated sales/profit.
plt.figure(), plt.title(), plt.xlabel(), plt.ylab	Displays matplotlib charts
sns.barplot(), sns.lineplot(), sns.scatter	loCt(r)eates bar, line and scatter visualizations using Seaborn.
round(), iterrows(), .dt.to_period()	Rounds values, iterates through rows and extracts monthly periods.

4. PROGRAM (CONTINUOUS FORMAT)

```
import pandas as pd import
matplotlib.pyplot as plt import
seaborn as sns


 df = pd.read_csv("/content/Sample - Superstore.csv",
encoding="latin1") print("Dataset loaded successfully!") print("Dataset
Shape:", df.shape) print("\nFirst 5 rows:") print(df.head())
print("\nColumn Names:") print(df.columns) print("\nDataset
Information:") df.info() print("\nMissing Values:")
print(df.isnull().sum()) print("\nStatistical Summary:")
print(df.describe())


 df["Order Date"] = pd.to_datetime(df["Order Date"]) category_sales =
df.groupby("Category")["Sales"].sum().sort_values(ascending=False) print("\nSales
by Category:") print(category_sales)
```

```
plt.figure(figsize=(8, 5))
  sns.barplot(x=category_sales.index,
  y=category_sales.values) plt.title("Sales by Category")
  plt.xlabel("Category") plt.ylabel("Total Sales")
  plt.tight_layout() plt.show()


category_region = df.groupby(["Category", "Region"])["Sales"].sum().reset_index()
  plt.figure(figsize=(10, 6)) sns.barplot(data=category_region, x="Category",
  y="Sales", hue="Region") plt.title("Sales by Category and Region")
  plt.xlabel("Category") plt.ylabel("Sales") plt.tight_layout() plt.show()


df["Month"] = df["Order Date"].dt.to_period("M").astype(str) monthly_sales = df.groupby(["Month",
  "Region"])["Sales"].sum().reset_index()                plt.figure(figsize=(12,                6))
  sns.lineplot(data=monthly_sales, x="Month", y="Sales", hue="Region", marker="o") plt.title("Sales
  Trend    Over    Time")    plt.xlabel("Month")    plt.ylabel("Sales")    plt.xticks(rotation=45)
  plt.tight_layout()   plt.show()   scatter_data   =   df.groupby(["Sub-Category",   "Category"],
  as_index=False)[["Sales",  "Profit"]].sum()  print("\nProfit  vs  Sales:")  print(scatter_data)
  plt.figure(figsize=(10, 7))
  sns.scatterplot(data=scatter_data, x="Sales", y="Profit", hue="Category", s=150)
  for _, row in scatter_data.iterrows():
      plt.text(row["Sales"], row["Profit"], row["Sub-Category"], fontsize=8)
  plt.title("Profit vs Sales by Sub-Category")
  plt.xlabel("Sales") plt.ylabel("Profit")
  plt.axhline(0, linestyle="--")
  plt.tight_layout() plt.show()



  summary = df.groupby("Category")[["Sales", "Profit"]].sum()
  print("\nSales and Profit by Category:") print(summary)
def filter_by_region(region):
      filtered_df = df[df["Region"] == region] print("\nSelected Region:", region)
      print("\nSales                          by                          Category:")
      print(filtered_df.groupby("Category")["Sales"].sum().sort_values(ascending=Fals
      e))  print("\nTotal Sales:", round(filtered_df["Sales"].sum(), 2)) print("Total
      Profit:", round(filtered_df["Profit"].sum(), 2)) filter_by_region("East")

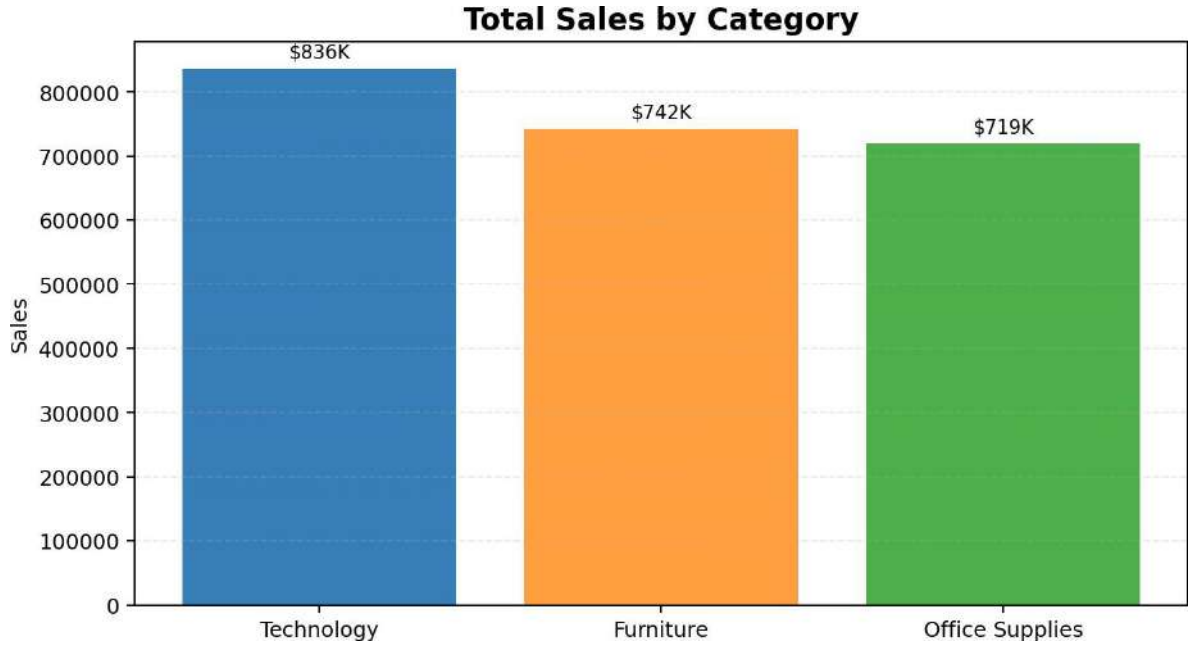

  # Filter and analyse a selected category def
  filter_by_category(category):
      filtered_df = df[df["Category"] == category] print("\nSelected Category:",
  category) print("Total Sales:", round(filtered_df["Sales"].sum(), 2)) print("Total
  Profit:", round(filtered_df["Profit"].sum(), 2)) print("\nSales by Region:")
  print(filtered_df.groupby("Region")["Sales"].sum().sort_values(ascending=False))
  filter_by_category("Technology")

  # Overall dashboard summary total_sales = df["Sales"].sum() total_profit
  = df["Profit"].sum()  print("\n" + "=" * 60)  print("SALES AND PROFIT
  DASHBOARD   SUMMARY")   print("="   *   60)   print("Total   Sales   :",
  round(total_sales, 2))  print("Total  Profit:",  round(total_profit,  2))
  print("\nTotal              Sales              by              Region:")
  print(df.groupby("Region")["Sales"].sum().sort_values(ascending=False))
  print("\nTotal             Sales             by             Category:")
  print(df.groupby("Category")["Sales"].sum().sort_values(ascending=False)
  )          print("\nTotal         Profit         by         Category:")
  print(df.groupby("Category")["Profit"].sum().sort_values(ascending=False
  ))
```

5. OUTPUT (COLOUR DIAGRAMS AND RESULTS)

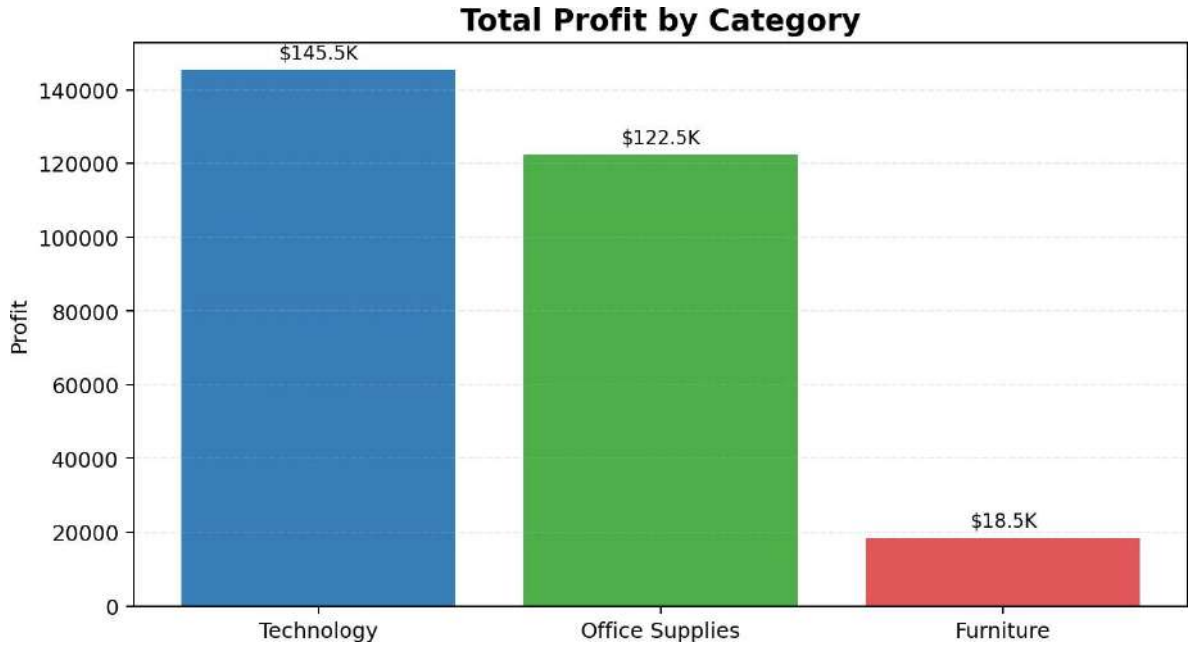
Dataset loaded successfully. Dataset shape: **(9994, 21)**. The dataset contains 9,994 records and 21 columns, and the uploaded output reports no missing values.

Sales by Category



Technology has the highest total sales, followed by Furniture and Office Supplies.

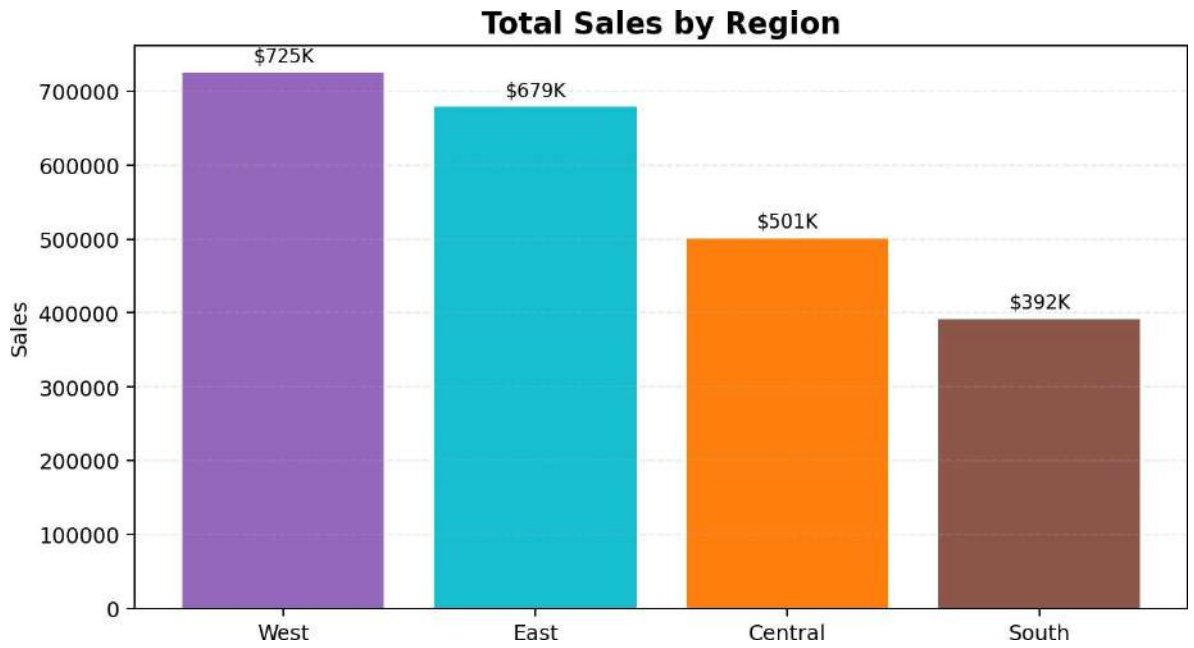
Profit by Category



Technology produces the highest total profit. Furniture has a much smaller profit compared with the other categories.

5. OUTPUT (CONTINUED)

Sales by Region



Overall dashboard summary from the provided output:

Measure	Value
Total Sales	$2,297,200.86
Total Profit	$286,397.02
Highest Sales Region	West — $725,457.82
Highest Sales Category	Technology — $836,154.03
Technology Profit	$145,454.95
East Region Sales	$678,781.24
East Region Profit	$91,522.78

6. LEARNING OUTCOMES

• Loaded and explored the Superstore dataset using Pandas.

• Checked data types, missing values and statistical summaries.

• Used grouping and aggregation to compare sales and profit.

• Visualized category, region and monthly trends with colourful charts.

• Created reusable filtering functions for region-wise and category-wise analysis.

RESULT

Thus, the Superstore dataset was analysed successfully using Python. The charts and summary statistics show that Technology generated the highest sales and profit, while the West region recorded the highest sales.